

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس یادگیری ماشین

استاد: دکتر امیرحسین نیکوفرد

تمرین سری چهارم

گروه جاویدنام سپهر ایران

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	آرین خوشنویس
شماره دانشجویی	۴۰۱۱۷۹۸۳
تاریخ	۱۴۰۵ خرداد



فهرست مطالب

۴	۱ پرسش ۱: شبکه متخاصم مولد
۷	۱.۱ بارگذاری داده ها و شبکه ResNet
۷	۱.۱.۱ دادگان BreastMNIST
۷	۲.۱.۱ مدل ResNet18
۱۷	۲.۱ شبکه Conditional DCGAN
۱۷	۱.۲.۱ بررسی ساختار شبکه اولیه
۲۲	۲.۲.۱ بررسی ساختار شبکه بهبود یافته (رویکرد میانی)
۲۴	۳.۲.۱ بررسی ساختار شبکه نهایی (Final cDCGAN Architecture)
۲۶	۴.۲.۱ بررسی دقیق ساختار شبکه های نهایی
۲۹	۵.۲.۱ بررسی فرایند آموزش و استراتژی ارزیابی شبکه
۳۱	۶.۲.۱ بررسی نتایج نهایی مدل
۳۲	۷.۲.۱ تولید ۲۰۰۰ داده های مصنوعی و توازن بخشی به مجموعه داده آموزش
۳۵	۸.۲.۱ ارزیابی نهایی: تحلیل عملکرد طبقه بند ResNet (داده های اصیل در برابر داده های افزوده با GAN)
۳۷	۳.۱ کد کامل
۷۰	۲ پرسش ۲: شبکه های رمزگذار - رمزگشا مولد
۷۰	۱.۲ مجموعه دادگان
۷۰	۱.۱.۲ معرفی مجموعه داده CIFAR-۱۰
۷۰	۲.۲ انجام PCA و ISOMAP
۷۰	۱.۲.۲ الگوریتم PCA (Principal Component Analysis)
۷۱	۲.۲.۲ الگوریتم ISOMAP (Isometric Mapping)
۷۳	۳.۲.۲ ارزیابی الگوریتم های کاهش ابعاد (PCA و ISOMAP) به همراه طبقه بند KNN
۷۶	۴.۲.۲ بهینه سازی مدل ها با استفاده از جستجوی تصادفی (Random Search) و Pipeline
۷۹	۵.۲.۲ اعمال PCA بر روی کل مجموعه داده ها
۸۱	۳.۲ رمزگذار-رمزگشا (Autoencoder)
۸۱	۱.۳.۲ استخراج ویژگی با استفاده از شبکه های خود رمزگذار (Autoencoders)
۸۷	۴.۲ خود رمزگذار متغیر (Variational Auto Encoder)
۸۷	۱.۴.۲ مروری بر خود رمزگذارهای متغیر (Variational Autoencoders - VAEs)
۸۸	۲.۴.۲ پیاده سازی معماری های VAE
۹۰	۳.۴.۲ نتایج طبقه بندی با KNN روی فضای پنهان
۹۳	۴.۴.۲ تحلیل و مقایسه نتایج
۹۴	۵.۲ کاوش در فضای latent
۹۴	۱.۵.۲ تحلیل نمودار پراکندگی فضای پنهان مدل ConvVAE (فضای ۱۰۰ بعدی)
۹۵	۲.۵.۲ تحلیل تولید منیفولد (Manifold Generation) و پیمایش فضای پنهان



۹۷		کد کامل	۳.۵.۲
۱۱۹		مدل های انتشار (Diffusion Models)	۶.۲
۱۱۹		مدل های انتشار (Diffusion Models) و مقایسه آن ها با VAE	۱.۶.۲



فهرست تصاویر

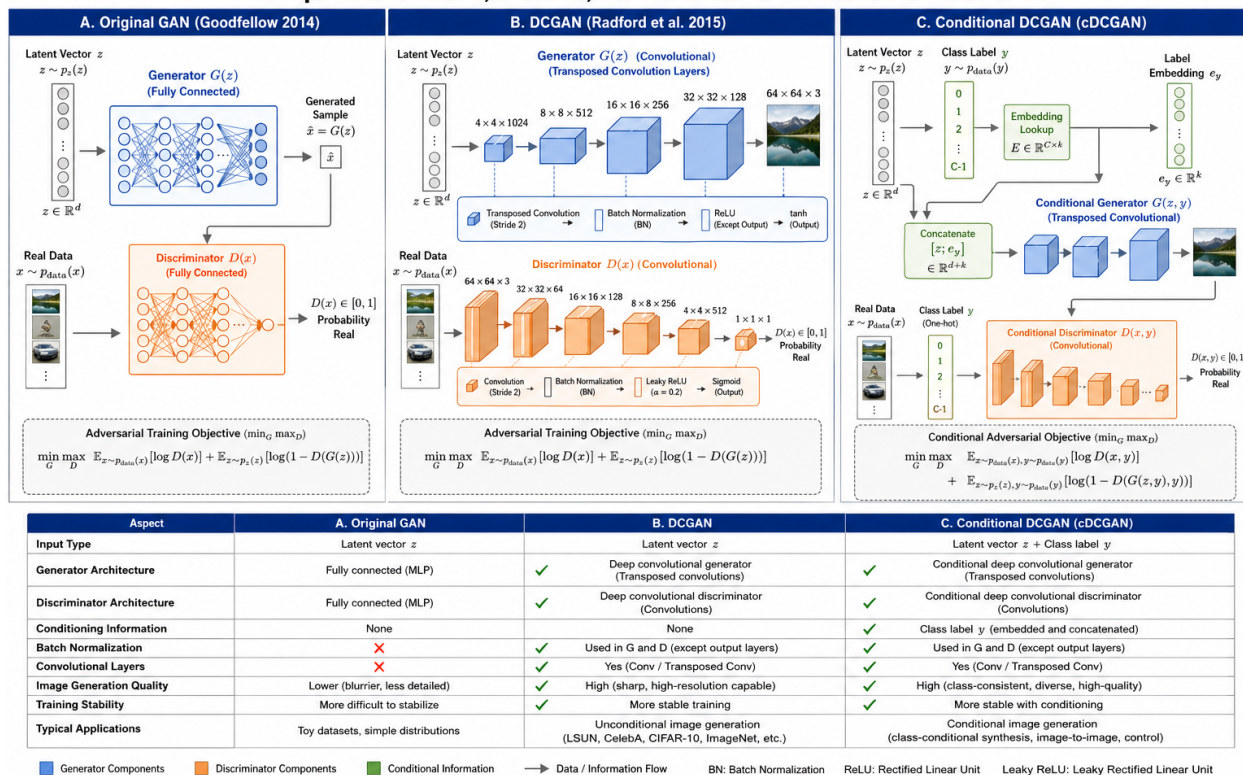
۴	مقایسه ساختار، ابعاد و جزئیات معماری شبکه‌های GAN، DCGAN و Conditional DCGAN	۱
۹	تصاویری از مجموعه داده‌های BreastMNIST	۲
۱۰	وضعیت دسته‌بندی داده‌های Train، Validation و Test	۳
۱۵	ماتریس آشفتگی	۴
۱۶	نمودار آموزش و اعتبارسنجی	۵
۱۶	نتایج مدل جدید	۶
۱۷	ماتریس آشفتگی مدل جدید	۷
۱۹	روند تغییرات تابع زیان مولد و متمایزگر در ۵۰ اپوک اولیه	۸
۲۰	نمونه تصاویر تولید شده توسط مدل اولیه (بروز پدیده فروپاشی حالت و نویزهای شدید رنگی)	۹
۲۳	منحنی تغییرات تابع زیان در معماری میانی؛ تثبیت گرادیان‌ها و رسیدن به مجاورت تعادل نش.	۱۰
۲۴	نمونه خروجی‌های معماری میانی؛ بروز مصنوعات شطرنجی و پدیده فروپاشی حالت نسبی.	۱۱
۲۷	گزارش ساختاری و ابعاد تنسورها در لایه‌های شبکه مولد نهایی.	۱۲
۲۸	گزارش ساختاری و ابعاد تنسورها در لایه‌های شبکه متمایزگر نهایی.	۱۳
۳۱	منحنی تغییرات توابع زیان متمایزگر و مولد در طول ۵۰۰ اپوک در معماری نهایی.	۱۴
۳۲	تصاویر ۲۸×۲۸ سنتز شده توسط شبکه نهایی تفکیک شده بر اساس کلاس‌های خوش خیم و بدخیم.	۱۵
	توزیع کلاس‌ها در بخش‌های مختلف مجموعه داده BreastMNIST پس از اعمال توازن بخشی در فاز آموزش.	۱۶
۳۴	بخش‌های هاشورخورده نمایانگر سهم تصاویر تولید شده توسط GAN هستند.	۳۴
۳۴	نمونه‌هایی تصادفی از مجموعه داده متوازن شده نهایی؛ ترکیبی یکپارچه از تصاویر واقعی و مصنوعی.	۳۴
۸۵	نمودارهای خطا برحسب اپوک برای مدل‌های اتوانکودر	۱۸
۹۳	نمودارهای خطا برحسب اپوک برای مدل‌های خود رمزگذار متغیر	۱۹
۹۴	ترسیم نمودار دو بعدی از ویژگی داده‌ها در فضای latent	۲۰
۹۶	تصویر تولید منیفولد	۲۱
۹۷	نمونه تصاویر تولید شده	۲۲



۱ پرسش ۱: شبکه متخاصم مولد

بررسی شبکه‌های مولد متخاصم (GAN)

Comparison of GAN, DCGAN, and Conditional DCGAN Architectures



شکل ۱: مقایسه ساختار، ابعاد و جزئیات معماری شبکه‌های GAN، DCGAN و Conditional DCGAN

همان‌طور که در تصویر ۱ به تصویر کشیده شده است، ایده مدل‌های مولد متخاصم از یک ساختار ساده مبتنی بر لایه‌های تمام‌متصل به مرور زمان به شبکه‌های پیچیده پیچشی و کنترل‌پذیر (شرطی) تغییر یافته است. در این بخش، جزئیات ریاضی و ساختاری هر یک از این سه معماری را بر اساس مستندات موجود در تصویر تحلیل می‌کنیم.

Original GAN (گودفلو - ۲۰۱۴)

ایده اولیه شبکه‌های متخاصم مولد که در بخش A تصویر دیده می‌شود، بر پایه یک بازی با مجموع صفر (Zero-sum game) بین دو شبکه کاملاً مجزا شکل گرفت:

- شبکه مولد $G(z)$: این شبکه یک بردار نویز تصادفی z را از یک فضای نهان با توزیع مشخص $z \sim p_z(z)$ دریافت می‌کند. ساختار داخلی این شبکه کاملاً مبتنی بر لایه‌های تمام‌متصل (Fully Connected یا MLP) است. وظیفه این لایه‌ها این است که بردار ورودی را گام‌به‌گام نگاشت داده و در نهایت یک نمونه خروجی مصنوعی به نام $\hat{x} = G(z)$ تولید کنند.

• شبکه متمایزگر $D(x)$: این شبکه نیز ساختاری کاملاً تمام‌متصل دارد. ورودی آن می‌تواند یک داده واقعی x (از توزیع اصلی داده‌ها $p_{data}(x)$) یا یک داده جعلی $\hat{x}$ باشد. خروجی این شبکه یک عدد اسکالر در بازه $[0, 1]$ است که نشان‌دهنده احتمال واقعی بودن تصویر ورودی است.

فرآیند آموزش این مدل از طریق یک تابع هدف مشترک به صورت زیر مدیریت می‌شود که در آن متمایزگر به دنبال بیشینه‌سازی دقت خود و مولد به دنبال فریب دادن متمایزگر (کمینه‌سازی لگاریتم خطای متمایزگر روی داده‌های جعلی) است:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

DCGAN (رادفورد - ۲۰۱۵)

نسخه کلاسیک گودفلو به دلیل ساختار تمام‌متصل، در پردازش و تولید تصاویر با ابعاد بزرگ و دارای بافت پیچیده به شدت ناپایدار بود. رادفورد با معرفی DCGAN (بخش B تصویر)، لایه‌های پیچشی را جایگزین ساختار قبلی کرد و چند قانون معماری کلیدی را توسعه داد:

- معماری مولد $G(z)$: در این مدل، لایه‌های تمام‌متصل حذف شده و جای خود را به لایه‌های کانولوشن ترانپوز (Transposed Convolution) با گام حرکت (Stride) دو داده‌اند. فرآیند بازسازی تصویر از یک بردار نویز z شروع شده و طی لایه‌های متوالی با ابعاد فیلتر مشخص (مثلاً ابتدا ابعاد کوچک 4×4 با 10×10 کانال، سپس 8×8 با 512 کانال، 16×16 با 256 کانال و در نهایت 32×32 با 128 کانال) به ابعاد نهایی تصویر ($3 \times 64 \times 64$) می‌رسد. در تمام لایه‌های میانی از تابع فعال‌سازی ReLU و لایه نرمال‌سازی دسته‌ای (Batch Normalization) استفاده می‌شود و لایه نهایی به تابع Tanh مجهز است.
- معماری متمایزگر $D(x)$: این شبکه یک مدل پیچشی عمیق معکوس است. تصویر ورودی 64×64 با عبور از لایه‌های کانولوشن استاندارد با گام ۲، به تدریج فشرده شده و عمق کانال‌های آن افزایش می‌یابد ($64, 128, 256, 512$). در این شبکه لایه‌های نرمال‌سازی دسته‌ای و تابع فعال‌سازی Leaky ReLU با نرخ آلفای ۰.۲ به کار گرفته شده‌اند و لایه آخر با یک تابع Sigmoid وضعیت نهایی را مشخص می‌کند.

(cDCGAN) Conditional DCGAN

اگرچه DCGAN کیفیت تصاویر را به شدت ارتقا داد، اما خروجی آن کاملاً تصادفی بود. در کاربردهای پزشکی (مانند تفکیک تومورهای خوش‌خیم و بدخیم)، نیاز به کنترل نوع تصویر تولیدی وجود دارد. این نیاز در معماری cDCGAN (بخش C تصویر) برطرف شد:

- تزییق شرط در مولد $G(z, y)$: علاوه بر بردار نویز z ، برچسب کلاس مورد نظر y نیز به شبکه داده می‌شود. این برچسب ابتدا از یک لایه جایگذاری (Embedding Lookup) عبور کرده تا به یک بردار متراکم $e_y \in \mathbb{R}^k$ تبدیل شود. سپس این دو بردار با یکدیگر ترکیب (Concatenate) شده و بردار جدیدی را می‌سازند که هم حاوی اطلاعات ساختاری تصادفی است و هم نوع کلاس خروجی را دیکته می‌کند. ادامه فرآیند تولید مشابه DCGAN پیش می‌رود.
- تزییق شرط در متمایزگر $D(x, y)$: برچسب کلاس y به شکل یک ماسک یا کانال فضایی هم‌اندازه با تصویر ورودی بازآرایی می‌شود. این کانال شرطی، در راستای بعد کانال‌ها به تصویر ورودی متصل می‌گردد (مثلاً برای تصویر تک‌کاناله، ورودی متمایزگر ۲ کاناله خواهد شد). متمایزگر در این حالت نه تنها کیفیت تصویر، بلکه تطابق بافت آن با برچسب ورودی را هم ارزیابی می‌کند. ریاضیات این شبکه مستقیماً به شکل شرطی تغییر یافته و به صورت زیر بازنویسی می‌شود:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x), y \sim p_{data}(y)} [\log D(x, y)] + \mathbb{E}_{z \sim p_z(z), y \sim p_{data}(y)} [\log(1 - D(G(z, y), y))]$$

مقایسه ساختاری و تحلیل محدودیت‌ها

با تکیه بر جدول مشخصات فنی ارائه شده در پایین تصویر ۱، می‌توان گفت هر یک از این گام‌های تکاملی برای پاسخ به یک محدودیت جدی در لایه قبلی طراحی شده‌اند:

- نوع ورودی و کنترل‌پذیری: مدل‌های GAN و DCGAN صرفاً بر پایه بردار تصادفی z کار می‌کنند، به این معنی که هیچ کنترلی روی خروجی وجود ندارد. مدل cDCGAN با ترکیب لایه جایگذاری برچسب با بردار نویز، این محدودیت بزرگ را حل کرد و کنترل‌پذیری هدایت‌شده را به شبکه اضافه نمود.
- معماری شبکه‌ها و لایه‌های پیچشی: مدل اولیه به دلیل نداشتن لایه‌های کانولوشن و استفاده از ساختار تمام‌متصل (MLP)، ویژگی‌های فضایی تصاویر را از دست می‌داد که نتیجه آن تولید خروجی‌های مات و بدون جزئیات بود. DCGAN و cDCGAN با ورود لایه‌های کانولوشن ترانهاد و استاندارد، توانستند الگوهای محلی و بافت‌های پیچیده تصاویر را استخراج و بازسازی کنند که منجر به بهبود چشمگیر کیفیت تصاویر شد.
- پایداری فرآیند آموزش: مدل‌های اولیه GAN به شدت مستعد پدیده ناپایداری و واگرایی بودند. در مدل‌های بعدی با اعمال استانداردهایی نظیر استفاده از Batch Normalization در لایه‌های میانی و توابع فعال‌سازی مناسب (Leaky ReLU در متمایزگر و Tanh/ReLU در مولد)، رفتار گرادین‌ها در طول شبکه مهار شد و فرآیند آموزش به ثبات بالایی رسید.

در نهایت، جدول تکاملی زیر خلاصه‌ای از این مقایسه ساختاری را نشان می‌دهد:

جدول ۱: جدول مقایسه ویژگی‌های ساختاری لایه‌ها و رفتار شبکه‌های خانواده GAN

شاخص فنی	Original GAN	DCGAN	Conditional DCGAN
نوع داده ورودی	بردار نهان z	بردار نهان z	بردار z + برچسب کلاس y
ساختار مولد	تمام‌متصل (MLP)	کانولوشن ترانهاد	کانولوشن ترانهاد شرطی
ساختار متمایزگر	تمام‌متصل (MLP)	کانولوشن عمیق	کانولوشن عمیق شرطی
اطلاعات شرطی	ندارد	ندارد	برچسب کلاس (ترکیب لایه‌ای)
نرمال‌سازی دسته‌ای	خیر	بله (به جز لایه‌های مرزی)	بله (به جز لایه‌های مرزی)
لایه‌های پیچشی	خیر	بله	بله
کیفیت تصویر خروجی	پایین و مات	بالا و واضح	بالا و منطبق بر کلاس هدف
پایداری در آموزش	بسیار سخت و ناپایدار	پایدار و استاندارد	بسیار پایدار به دلیل جهت‌دهی شرط

۱.۱ بارگذاری داده ها و شبکه ResNet

ما از مجموعه داده BreastMNIST استفاده میکنیم. این مجموعه داده، تصاویر سونوگرافی پستان زنان را به دو کلاس خوش خیم و بدخیم تقسیم میکند در حالی که تعداد داده های کلاس خوش خیم نسبت به بدخیم بسیار بیشتر است. نامتقارنی در داده های کلاس های پزشکی امری طبیعی و متداول است اما در یادگیری ماشین باعث خطای زیاد مدل در پیشبینی داده های کلاس کوچک تر میشود. در این سوال، از مدل آماده ResNet18 برای آموزش و اعتبارسنجی داده ها استفاده کرده و برای حل مشکل کلاس های نامتقارن، به سراغ شبکه های متخاصم مولد (GAN) میرویم

۱.۱.۱ دادگان BreastMNIST

مجموعه MedMNIST استانداردسازی شده ای از تصاویر پزشکی (شامل تصاویر رادیولوژی، سونوگرافی، سی تی اسکن و غیره) است که ساختاری مشابه با دیتاست مشهور MNIST (اعداد دست نویس) دارد تا پیاده سازی و ارزیابی الگوریتم های بینایی ماشین و یادگیری عمیق در حوزه پزشکی ساده تر و استانداردتر شود.

دادگان BreastMNIST یکی از زیرمجموعه های پلتفرم MedMNIST v2 به شمار میرود. تصاویر موجود در دادگان BreastM-NIST براساس یک مجموعه داده واقعی پزشکی از تصاویر سونوگرافی پستان تولید شده اند. این تصاویر شامل اسکن های سونوگرافی مربوط به زنان در بازه سنی ۲۵ تا ۷۵ سال است.

مشخصات دادگان BreastMNIST

- در این دادگان، داده ها تومور ها و بافت های سینه به دو کلاس خوشخیم (Benign) و بدخیم (Malignant) تقسیم شده اند.
 - ابعاد تمام تصاویر به 28×28 پیکسل تغییر یافته اند
 - تصاویر به صورت خاکستری (Grayscale) یا تک کاناله هستند
 - این دادگان شامل ۷۸۰ تصویر سونوگرافی است (۵۴۶ داده Train، ۷۸ داده اعتبارسنجی val و ۱۵۶ داده test)
- داده های کلاس بدخیم، حدود ۲۷ درصد از کل داده ها را در هر بخش تشکیل میدهد. درصدی که نشان میدهد نامتقارنی زیادی در بین داده های خوش خیم و بدخیم وجود دارد که میتواند پیشبینی مدل را تحت تاثیر قرار دهد.

۲.۱.۱ مدل ResNet18

مدل ResNet18 (مخفف Residual Network با ۱۸ لایه) یکی از کاربردی ترین شبکه های عصبی کانولوشنی (CNN) در دنیای بینایی ماشین است. وزن های پیش فرض آن (Pre-Trained) که روی دیتابیس عظیم ImageNet آموزش دیده اند، به عنوان یک استخراج کننده ویژگی فوق العاده برای تسک های دیگر عمل می کنند. این ویژگی باعث شده این مدل برای دسته بندی تصاویر و همچنین کاربردهای تصویربرداری و تشخیص پزشکی بسیار مناسب باشد.

در شبکه های عصبی کانولوشنی کلاسیک، پردازش تصویر به صورت متوالی و لایه به لایه انجام می شود. به این معنی که هر لایه خروجی لایه قبلی را دریافت کرده، تغییراتی روی آن اعمال می کند و به لایه بعدی منتقل می کند. اما هنگامی که تعداد لایه های شبکه از یک حد مشخص فراتر می رود، عملکرد مدل به جای بهبود، دچار افت شدید می شود که به آن پدیده تخریب (Degradation) گفته

می‌شود. در این حالت نه تنها فرآیند آموزش سخت می‌شود، بلکه جزئیات مهم تصویر در اعماق شبکه محو یا دچار تغییرات مخرب می‌شوند.

معماری ResNet برای حل این مشکل از مفهوم اتصالات میانبر استفاده می‌کند. در هر بلوک ResNet یک مسیر میانبر اضافه می‌شود تا داده‌های ورودی یک بلوک بدون تغییر توسط لایه‌های کانولوشنی، مستقیماً به انتهای بلوک منتقل شوند. در پایان بلوک، اطلاعات مسیر میانبر با خروجی لایه‌های کانولوشنی ادغام می‌شود. از آنجا که ویژگی‌های ورودی از طریق مسیر میانبر حفظ می‌شوند، لایه‌های داخلی نیازی به بازسازی کامل ویژگی‌ها ندارند که این امر باعث کاهش بار محاسباتی و تسهیل فرآیند بهینه‌سازی می‌شود.

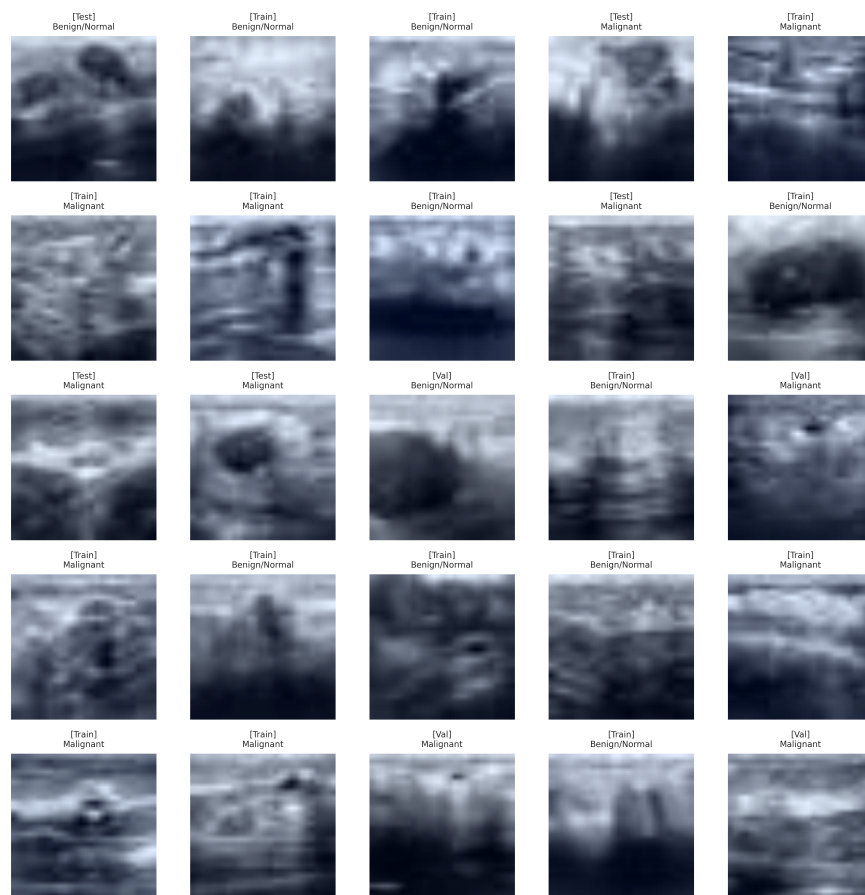
بلوک‌های سازنده و ساختار داخلی **ResNet18** معماری ResNet18 از بلوک‌های ساده‌تری به نام BasicBlock استفاده می‌کند. هر BasicBlock شامل موارد زیر است:

- دو لایه کانولوشن با فیلترهای 3×3
 - لایه BatchNormalization بعد از هر کانولوشن برای پایداری آموزش
 - تابع فعال‌سازی ReLU
 - یک مسیر میانبر که ورودی را به خروجی جمع می‌کند
- معماری ResNet18 دارای 18 لایه دارای وزن است.

پیش‌پردازش داده‌ها همان‌طور که در ساختار داخلی ResNet18 اشاره شد، این مدل داده‌ها را در قالب تصاویر 3 کاناله با ابعاد 224×224 دریافت می‌کند. در حالی که داده‌های ما تصاویر تک کاناله 28×28 هستند. از طرفی تصاویر ImageNet که ResNet18 با آن آموزش دیده است دارای میانگین و انحراف معیار مشخصی هستند.

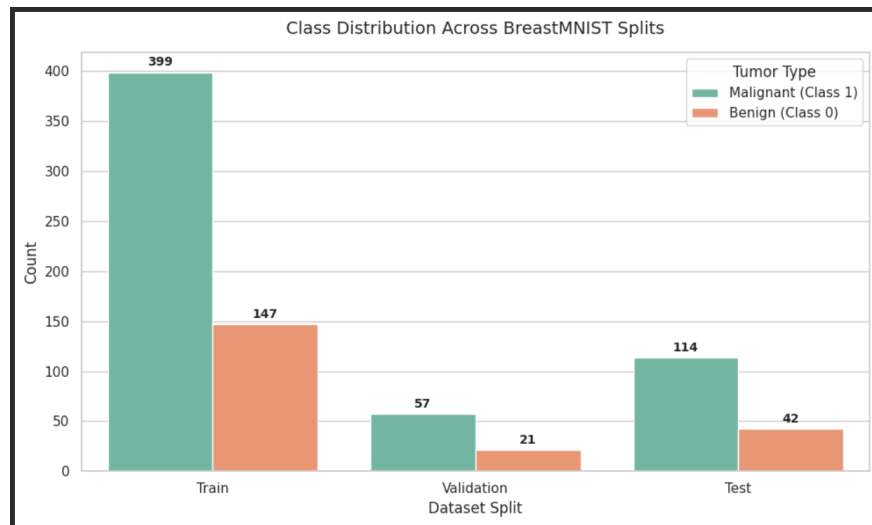
بنابراین در مرحله پیش‌پردازش، تصاویر BreastMNIST ابتدا باید به تصاویر 3 کاناله با ابعاد 224×224 تبدیل شوند. همچنین چون از وزن‌های پیش‌آموزش‌دیده ResNet18 استفاده می‌کنیم، داده‌ها باید مطابق میانگین و انحراف معیار ImageNet نرمال‌سازی شوند.

بصری‌سازی داده‌ها در این مرحله تعدادی از داده‌های دیتاست BreastMNIST به صورت تصادفی نمایش داده شده‌اند:



شکل ۲: تصاویری از مجموعه داده‌های BreastMNIST

مشاهده دسته‌بندی کلاس‌ها در بخش 1.1 جزئیات دیتاست BreastMNIST توضیح داده شد. در این بخش توزیع کلاس‌ها نمایش داده می‌شود:



شکل ۳: وضعیت دسته‌بندی داده‌های Train، Validation و Test

— TRAIN DATASET INFO —

Total Images: 546

Image Tensor Shape: torch.Size([3, 224, 224])

Data Type: torch.float32

Classes Found: ['0', '1']

— VALIDATION DATASET INFO —

Total Images: 78

Image Tensor Shape: torch.Size([3, 224, 224])

Data Type: torch.float32

Classes Found: ['0', '1']

— TEST DATASET INFO —

Total Images: 156

Image Tensor Shape: torch.Size([3, 224, 224])

Data Type: torch.float32

Classes Found: ['0', '1']

Total Images Across All Splits: 780

مشاهده می‌شود که تمامی داده‌ها به صورت تانسورهای 3 کاناله با ابعاد 224×224 هستند که با ورودی ResNet مطابقت دارند.



تعریف مدل **ResNet18**، تابع هزینه و بهینه‌ساز پس از پیش‌پردازش داده‌ها، از مدل ResNet18 به همراه تابع خطای CrossEntropy و بهینه‌ساز Adam با نرخ یادگیری 0.0001 استفاده می‌شود.

نکته مهم این است که مدل ResNet18 به طور پیش‌فرض دارای 1000 کلاس خروجی است، در حالی که مسئله ما 2 کلاس است (خوش خیم و بدخیم). بنابراین لایه آخر حذف شده و یک لایه خطی جدید با خروجی 2 کلاس جایگزین می‌شود.

```

1  basicstyle=\ttfamily\tiny,
2  numbers=none,
3  breaklines=false,
4  backgroundcolor=\color{white}
5  ]
6
=====
7  Layer (type (var_name))          Input Shape      Output Shape      Param #
8                                     Trainable
=====
9  ResNet (ResNet)                  [32, 3, 224, 224] [32, 2]           --
10                                     True
11  Conv2d (conv1)                   [32, 3, 224, 224] [32, 64, 112, 112] 9,408
12                                     True
13  BatchNorm2d (bn1)                [32, 64, 112, 112] [32, 64, 112, 112] 128
14                                     True
15  ReLU (relu)                       [32, 64, 112, 112] [32, 64, 112, 112] --
16                                     --
17  MaxPool2d (maxpool)              [32, 64, 112, 112] [32, 64, 56, 56]   --
18                                     --
19  Sequential (layer1)              [32, 64, 56, 56]   [32, 64, 56, 56]   --
20                                     True
21  BasicBlock (0)                   [32, 64, 56, 56]   [32, 64, 56, 56]   --
22                                     True
23  Conv2d (conv1)                   [32, 64, 56, 56]   [32, 64, 56, 56]   36,864
24                                     True
25  BatchNorm2d (bn1)                [32, 64, 56, 56]   [32, 64, 56, 56]   128
26                                     True
27  ReLU (relu)                       [32, 64, 56, 56]   [32, 64, 56, 56]   --
28                                     --
29  Conv2d (conv2)                   [32, 64, 56, 56]   [32, 64, 56, 56]   36,864
30                                     True
31  BatchNorm2d (bn2)                [32, 64, 56, 56]   [32, 64, 56, 56]   128
32                                     True
33  ReLU (relu)                       [32, 64, 56, 56]   [32, 64, 56, 56]   --
34                                     --

```



22	BasicBlock (1)	[32, 64, 56, 56]	[32, 64, 56, 56]	--
	True			
23	Conv2d (conv1)	[32, 64, 56, 56]	[32, 64, 56, 56]	36,864
	True			
24	BatchNorm2d (bn1)	[32, 64, 56, 56]	[32, 64, 56, 56]	128
	True			
25	ReLU (relu)	[32, 64, 56, 56]	[32, 64, 56, 56]	--
	--			
26	Conv2d (conv2)	[32, 64, 56, 56]	[32, 64, 56, 56]	36,864
	True			
27	BatchNorm2d (bn2)	[32, 64, 56, 56]	[32, 64, 56, 56]	128
	True			
28	ReLU (relu)	[32, 64, 56, 56]	[32, 64, 56, 56]	--
	--			
29	Sequential (layer2)	[32, 64, 56, 56]	[32, 128, 28, 28]	--
	True			
30	BasicBlock (0)	[32, 64, 56, 56]	[32, 128, 28, 28]	--
	True			
31	Conv2d (conv1)	[32, 64, 56, 56]	[32, 128, 28, 28]	73,728
	True			
32	BatchNorm2d (bn1)	[32, 128, 28, 28]	[32, 128, 28, 28]	256
	True			
33	ReLU (relu)	[32, 128, 28, 28]	[32, 128, 28, 28]	--
	--			
34	Conv2d (conv2)	[32, 128, 28, 28]	[32, 128, 28, 28]	147,456
	True			
35	BatchNorm2d (bn2)	[32, 128, 28, 28]	[32, 128, 28, 28]	256
	True			
36	Sequential (downsample)	[32, 64, 56, 56]	[32, 128, 28, 28]	8,448
	True			
37	ReLU (relu)	[32, 128, 28, 28]	[32, 128, 28, 28]	--
	--			
38	BasicBlock (1)	[32, 128, 28, 28]	[32, 128, 28, 28]	--
	True			
39	Conv2d (conv1)	[32, 128, 28, 28]	[32, 128, 28, 28]	147,456
	True			
40	BatchNorm2d (bn1)	[32, 128, 28, 28]	[32, 128, 28, 28]	256
	True			
41	ReLU (relu)	[32, 128, 28, 28]	[32, 128, 28, 28]	--
	--			
42	Conv2d (conv2)	[32, 128, 28, 28]	[32, 128, 28, 28]	147,456
	True			
43	BatchNorm2d (bn2)	[32, 128, 28, 28]	[32, 128, 28, 28]	256
	True			
44	ReLU (relu)	[32, 128, 28, 28]	[32, 128, 28, 28]	--



	--			
45	Sequential (layer3)	[32, 128, 28, 28]	[32, 256, 14, 14]	--
	True			
46	BasicBlock (0)	[32, 128, 28, 28]	[32, 256, 14, 14]	--
	True			
47	Conv2d (conv1)	[32, 128, 28, 28]	[32, 256, 14, 14]	294,912
	True			
48	BatchNorm2d (bn1)	[32, 256, 14, 14]	[32, 256, 14, 14]	512
	True			
49	ReLU (relu)	[32, 256, 14, 14]	[32, 256, 14, 14]	--
	--			
50	Conv2d (conv2)	[32, 256, 14, 14]	[32, 256, 14, 14]	589,824
	True			
51	BatchNorm2d (bn2)	[32, 256, 14, 14]	[32, 256, 14, 14]	512
	True			
52	Sequential (downsample)	[32, 128, 28, 28]	[32, 256, 14, 14]	33,280
	True			
53	ReLU (relu)	[32, 256, 14, 14]	[32, 256, 14, 14]	--
	--			
54	BasicBlock (1)	[32, 256, 14, 14]	[32, 256, 14, 14]	--
	True			
55	Conv2d (conv1)	[32, 256, 14, 14]	[32, 256, 14, 14]	589,824
	True			
56	BatchNorm2d (bn1)	[32, 256, 14, 14]	[32, 256, 14, 14]	512
	True			
57	ReLU (relu)	[32, 256, 14, 14]	[32, 256, 14, 14]	--
	--			
58	Conv2d (conv2)	[32, 256, 14, 14]	[32, 256, 14, 14]	589,824
	True			
59	BatchNorm2d (bn2)	[32, 256, 14, 14]	[32, 256, 14, 14]	512
	True			
60	ReLU (relu)	[32, 256, 14, 14]	[32, 256, 14, 14]	--
	--			
61	Sequential (layer4)	[32, 256, 14, 14]	[32, 512, 7, 7]	--
	True			
62	BasicBlock (0)	[32, 256, 14, 14]	[32, 512, 7, 7]	--
	True			
63	Conv2d (conv1)	[32, 256, 14, 14]	[32, 512, 7, 7]	1,179,648
	True			
64	BatchNorm2d (bn1)	[32, 512, 7, 7]	[32, 512, 7, 7]	1,024
	True			
65	ReLU (relu)	[32, 512, 7, 7]	[32, 512, 7, 7]	--
	--			
66	Conv2d (conv2)	[32, 512, 7, 7]	[32, 512, 7, 7]	2,359,296
	True			



67	BatchNorm2d (bn2)	[32, 512, 7, 7]	[32, 512, 7, 7]	1,024
	True			
68	Sequential (downsample)	[32, 256, 14, 14]	[32, 512, 7, 7]	132,096
	True			
69	ReLU (relu)	[32, 512, 7, 7]	[32, 512, 7, 7]	--
	--			
70	BasicBlock (1)	[32, 512, 7, 7]	[32, 512, 7, 7]	--
	True			
71	Conv2d (conv1)	[32, 512, 7, 7]	[32, 512, 7, 7]	2,359,296
	True			
72	BatchNorm2d (bn1)	[32, 512, 7, 7]	[32, 512, 7, 7]	1,024
	True			
73	ReLU (relu)	[32, 512, 7, 7]	[32, 512, 7, 7]	--
	--			
74	Conv2d (conv2)	[32, 512, 7, 7]	[32, 512, 7, 7]	2,359,296
	True			
75	BatchNorm2d (bn2)	[32, 512, 7, 7]	[32, 512, 7, 7]	1,024
	True			
76	ReLU (relu)	[32, 512, 7, 7]	[32, 512, 7, 7]	--
	--			
77	AdaptiveAvgPool2d (avgpool)	[32, 512, 7, 7]	[32, 512, 1, 1]	--
	--			
78	Linear (fc)	[32, 512]	[32, 2]	1,026
	True			
79	=====			
80	Total params: 11,177,538			
81	Trainable params: 11,177,538			
82	Non-trainable params: 0			
83	Total mult-adds (Units.GIGABYTES): 58.03			
84	=====			
85	Input size (MB): 19.27			
86	Forward/backward pass size (MB): 1271.66			
87	Params size (MB): 44.71			
88	Estimated Total Size (MB): 1335.64			
89	=====			
90				

جزئیات معماری ResNet18 در مجموع این مدل دارای 11177538 پارامتر است.

فرآیند آموزش و اعتبارسنجی مدل برای epoch 10 آموزش داده شده است. در هر epoch مقدار loss و accuracy برای داده‌های train و validation محاسبه می‌شود.

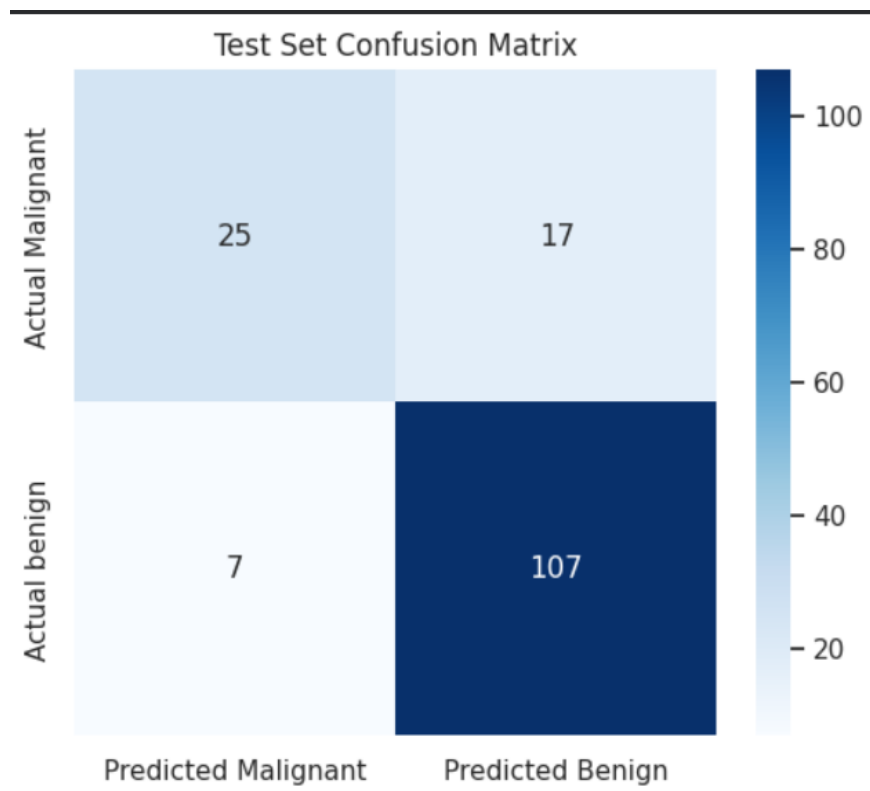
```
1 Epoch [1/10] | Train Loss: 0.4827 | Train Acc: 0.8095 | Val Loss: 0.6081 | Val Acc: 0.7308
2 ...
3 Epoch [10/10] | Train Loss: 0.0539 | Train Acc: 0.9853 | Val Loss: 0.3149 | Val Acc: 0.8974
4
```

بهترین نتیجه در epoch 7 حاصل شده است.

ارزیابی روی داده‌های Test

Test Accuracy (Best Model): 0.8462

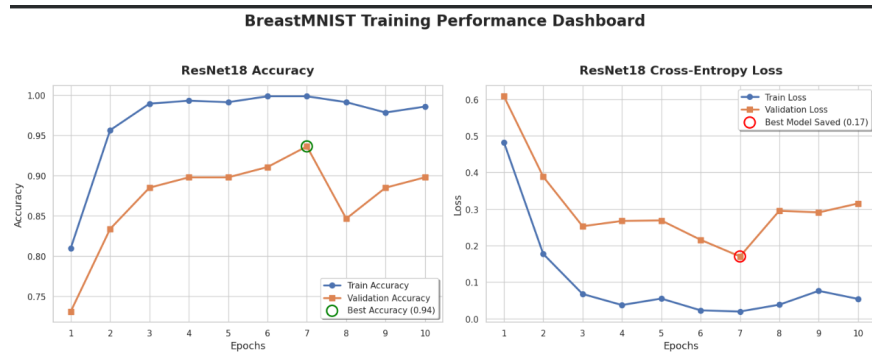
```
1 precision recall f1-score support
2
3 Benign (0) 0.78 0.60 0.68 42
4 Malignant (1) 0.86 0.94 0.90 114
5
6 accuracy 0.85 156
7
```



شکل ۴: ماتریس آشفتگی



ماتریس آشفتگی نتایج نشان می‌دهد مدل در تشخیص کلاس بدخیم عملکرد بهتری دارد، اما در کلاس خوش خیم خطای بیشتری دارد. این موضوع ناشی از عدم توازن داده‌ها است. برای رفع این مشکل از شبکه‌های GAN استفاده می‌شود.

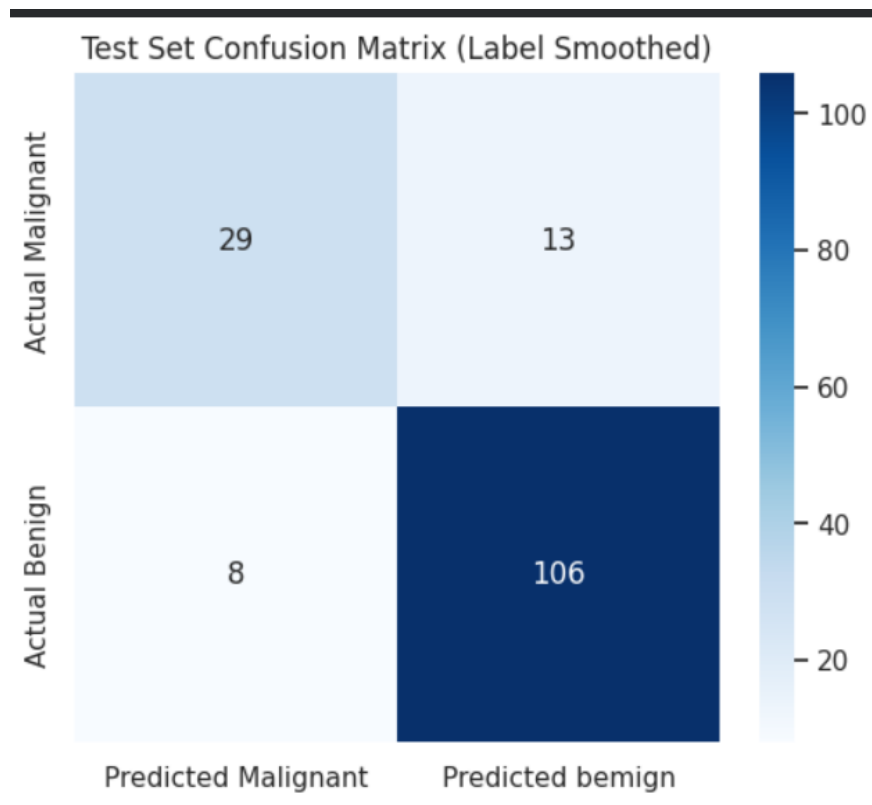


شکل ۵: نمودار آموزش و اعتبارسنجی

نمودار آموزش و اعتبارسنجی در epoch 7 مدل وارد بیش‌برازش شده است. برای حل مشکل از label smoothing = 0.1 در تابع CrossEntropy استفاده می‌شود. در این روش برچسب‌های 0 و 1 به 0.05 و 0.95 تبدیل می‌شوند.



شکل ۶: نتایج مدل جدید



شکل ۷: ماتریس آشفتگی مدل جدید

نتایج مدل جدید مدل جدید عملکرد پایدارتر و خطای کمتری در validation دارد، اما هنوز مشکل کلاس بدخیم به طور کامل حل نشده است و در ادامه از GAN برای بهبود استفاده می‌شود.

۲.۱ شبکه Conditional DCGAN

۱.۲.۱ بررسی ساختار شبکه اولیه

در این بخش، معماری و تنظیمات پایه در پیاده‌سازی اولیه شبکه متخاصم مولد شرطی (cDCGAN) به صورت دقیق بررسی می‌شود. ساختار کلی این مدل بر پایه ادغام بردارهای نویز و اطلاعات شرطی (برچسب کلاس) بنا شده است. جدول ۲ ابرپارامترهای کلیدی استفاده شده در این شبکه را نشان می‌دهد.

۱. ساختار شبکه مولد (Generator) ورودی این شبکه شامل یک بردار نویز $z \in \mathbb{R}^{100}$ و برچسب کلاس y است. برچسب کلاس ابتدا از یک لایه جایگذاری (Embedding) عبور کرده و به یک بردار ۱۰۰ بعدی تبدیل می‌شود. با اتصال (Concatenation) این دو بردار، یک ورودی با ۲۰۰ کانال به شکل فضایی $200 \times 1 \times 1$ شکل می‌گیرد که وارد لایه‌های کانولوشن ترانهاد (ConvTranspose2d) می‌شود:

- لایه اول: ابعاد از 1×1 به 4×4 با ۵۱۲ کانال بسط می‌یابد (هسته ۴، گام ۱). فعال‌ساز: BatchNorm و ReLU.

- لایه دوم: ابعاد به 8×8 با ۲۵۶ کانال افزایش می‌یابد (هسته ۴، گام ۲، حاشیه ۱). فعال‌ساز: BatchNorm و ReLU.

جدول ۲: ابرپارامترهای شبکه cDCGAN اولیه

پارامتر (Parameter)	مقدار
تعداد کلاس‌ها (Number of Classes)	۲
ابعاد فضای نهان (z - Latent Dimension)	۱۰۰
ابعاد تصویر خروجی (Image Size)	32×32
کانال‌های تصویر (Image Channels)	۳ (RGB)
اندازه نقشه ویژگی (Feature Map Size)	۶۴
تعداد اپوک‌ها (Epochs)	۵۰
اندازه دسته (Batch Size)	۳۲
الگوریتم بهینه‌ساز (Optimizer)	Adam
تابع زیان (Loss Function)	BCE Loss
مومنتم بهینه‌ساز (β_1)	۰.۵

- لایه سوم: ابعاد به 16×16 با ۱۲۸ کانال می‌رسد (هسته ۴، گام ۲، حاشیه ۱). فعال‌ساز: BatchNorm و ReLU.
- لایه چهارم (خروجی): تبدیل نهایی به ابعاد تصویر هدف یعنی 32×32 با ۳ کانال رنگی. تابع فعال‌ساز نهایی در این لایه Tanh است که خروجی را در بازه $[-1, 1]$ تثبیت می‌کند.
- ۲. ساختار شبکه متمایزگر (Discriminator) شبکه متمایزگر باید تصویر (واقعی یا تولید شده) را به همراه برچسب آن ارزیابی کند. برای این منظور، برچسب کلاس ابتدا از یک لایه جایگذاری عبور کرده و به یک بردار 1024 بعدی تبدیل می‌شود. سپس این بردار به یک نقشه فضایی (Label Map) با ابعاد $1 \times 32 \times 32$ تغییر شکل (Reshape) یافته و به عنوان کانال چهارم به تصویر اصلی ($3 \times 32 \times 32$) متصل می‌گردد. ورودی نهایی با ابعاد $4 \times 32 \times 32$ وارد لایه‌های کانولوشن (Conv2d) می‌شود:
- لایه اول: کاهش ابعاد به 16×16 با ۶۴ کانال (هسته ۴، گام ۲، حاشیه ۱). فعال‌ساز: LeakyReLU(0.2). (بدون نرمال‌سازی دسته‌ای).
- لایه دوم: کاهش ابعاد به 8×8 با ۱۲۸ کانال. فعال‌ساز: BatchNorm و LeakyReLU(0.2).
- لایه سوم: کاهش ابعاد به 4×4 با ۲۵۶ کانال. فعال‌ساز: BatchNorm و LeakyReLU(0.2).
- لایه چهارم (خروجی): فشرده‌سازی به ابعاد $1 \times 1 \times 1$ (هسته ۴، گام ۱). خروجی این لایه از یک تابع Sigmoid عبور می‌کند تا احتمال واقعی بودن تصویر را در بازه $[0, 1]$ نشان دهد.

۳. توابع زیان و بهینه‌سازی در این معماری از تابع زیان آنتروپی متقاطع باینری (BCE Loss) استفاده شده است که فرمول پایه آن به صورت $L = -[y \log(p) + (1 - y) \log(1 - p)]$ تعریف می‌شود. توابع زیان برای هر دو شبکه به شکل زیر صورت‌بندی شده‌اند:

$$L_D = \frac{1}{2} [BCELoss(D(x, y), 1) + BCELoss(D(G(z, y), y), 0)] \quad (۱)$$



$$L_G = BCELoss(D(G(z, y), y), 1) \quad (2)$$

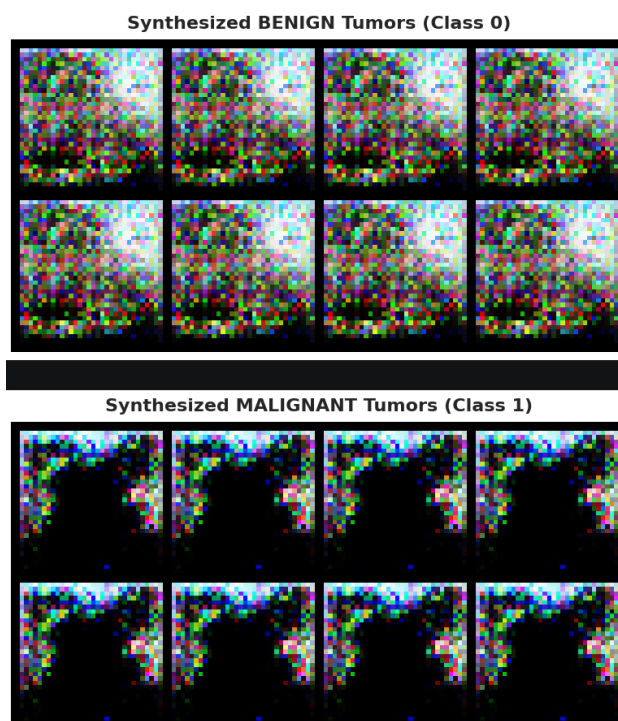
برای به‌روزرسانی وزن‌ها از الگوریتم Adam با پارامترهای $\beta_1 = 0.5$ و $\beta_2 = 0.999$ استفاده شده است. یک نکته فنی و حیاتی در این پیاده‌سازی، استفاده از نرخ یادگیری نامتقارن است؛ در ابتدا نرخ یادگیری هر دو برابر با $(lr = 0.0002)$ بود اما در ابتدای آموزش مقدار لاس مولد خیلی بالا بود و بالاتر میرفت؛ برای حل این مشکل رویکرد رو تغییر دادم، به طوری که نرخ یادگیری مولد $(lr = 0.0002)$ دقیقاً چهار برابر نرخ یادگیری متمایزگر $(lr = 0.00005)$ تنظیم شده است. این رویکرد تضمین می‌کند که متمایزگر در مراحل اولیه آموزش بیش از حد قدرتمند نشود و فرصت یادگیری را از مولد سلب نکند.

Epoch [1/50]	D Loss: 2.5491	G Loss: 0.2190
-> Generator improved! Model saved.		
Epoch [2/50]	D Loss: 2.5715	G Loss: 0.2884
Epoch [3/50]	D Loss: 2.0720	G Loss: 0.4494
Epoch [4/50]	D Loss: 1.5576	G Loss: 0.6800
Epoch [5/50]	D Loss: 1.2014	G Loss: 0.9542
Epoch [6/50]	D Loss: 0.9303	G Loss: 0.9635
Epoch [7/50]	D Loss: 0.8721	G Loss: 1.0857
Epoch [8/50]	D Loss: 0.7306	G Loss: 1.2451
Epoch [9/50]	D Loss: 0.6178	G Loss: 1.6607
Epoch [10/50]	D Loss: 0.5411	G Loss: 1.5713
Epoch [11/50]	D Loss: 0.5939	G Loss: 1.5076
Epoch [12/50]	D Loss: 0.6397	G Loss: 1.7392
Epoch [13/50]	D Loss: 0.5566	G Loss: 1.6557
Epoch [14/50]	D Loss: 0.7156	G Loss: 1.5578
Epoch [15/50]	D Loss: 0.5937	G Loss: 1.4546
Epoch [16/50]	D Loss: 0.6089	G Loss: 2.0523
Epoch [17/50]	D Loss: 0.5555	G Loss: 1.7328
Epoch [18/50]	D Loss: 0.4191	G Loss: 1.8796
Epoch [19/50]	D Loss: 0.4492	G Loss: 1.9711
Epoch [20/50]	D Loss: 0.4491	G Loss: 2.2127
Epoch [21/50]	D Loss: 0.3617	G Loss: 2.1726
Epoch [22/50]	D Loss: 0.3382	G Loss: 2.5234
Epoch [23/50]	D Loss: 0.4091	G Loss: 2.7254
Epoch [24/50]	D Loss: 0.4150	G Loss: 1.8643
Epoch [25/50]	D Loss: 0.6121	G Loss: 2.1951
Epoch [26/50]	D Loss: 0.3483	G Loss: 1.9863
Epoch [27/50]	D Loss: 0.4253	G Loss: 2.7736
Epoch [28/50]	D Loss: 0.3814	G Loss: 2.5684

شکل ۸: روند تغییرات تابع زیان مولد و متمایزگر در ۵۰ اپوک اولیه

بررسی نتایج و تحلیل تصاویر تولید شده توسط شبکه اولیه با بررسی لاگ‌های آموزش (تصویر ۸) و خروجی‌های بصری شبکه (تصویر ۹)، به وضوح مشخص است که پیاده‌سازی اولیه با شکست مواجه شده است. خروجی‌های شبکه به جای بازنمایی بافت‌های طبیعی سونوگرافی، به الگوهای نویزی، پیکسلی و کاملاً تکراری تبدیل شده‌اند. این مشاهدات ریشه در چندین خطای بنیادین در طراحی معماری و استراتژی آموزش دارد که در ادامه به صورت علمی تحلیل می‌شوند:

- بروز پدیده فروپاشی حالت (Mode Collapse): همان‌طور که در خروجی‌های تصویر ۹ مشهود است، شبکه برای هر بردار نویز ورودی، دقیقاً یک خروجی یکسان و نویزی را در هر کلاس کپی کرده است. این وضعیت بحرانی که در ادبیات شبکه‌های متخاصم به عنوان «فروپاشی حالت» یا Mode Collapse شناخته می‌شود، زمانی رخ می‌دهد که شبکه مولد از ایجاد تنوع عاجز مانده و روی یک الگوی واحد که موقتاً متمایزگر را فریب داده است، قفل می‌شود.
- عدم تعادل مخرب در مکانیزم به‌روزرسانی و نرخ یادگیری: در این معماری، نرخ یادگیری مولد چهار برابر متمایزگر تنظیم شد و همزمان، مولد در هر گام دو بار آموزش دید (Double Update). اگرچه این برتری محاسباتی باعث شد در اپوک‌های ابتدایی خطای مولد بسیار پایین باشد (مثلاً $G Loss: 0.2190$ در اپوک اول) و متمایزگر موقتاً مغلوب شود، اما این سرعت بالا منجر به پرش‌های ناگهانی و خشن گرادیان شد. شبکه نتوانست به تعادل نش (Nash Equilibrium) دست یابد و با گذر از اپوک ۲۰، متمایزگر توانست الگوهای تکراری و نویزی مولد را به راحتی شناسایی کند که نتیجه آن افت شدید خطای D و انفجار خطای G به اعداد بالای ۵.۲ بود.



شکل ۹: نمونه تصاویر تولید شده توسط مدل اولیه (بروز پدیده فروپاشی حالت و نویزهای شدید رنگی)

- گلوگاه فضایی و شروع از نقطه کور (1×1): یکی از بزرگترین چالش‌های معماری مولد در این مدل، تلاش برای استخراج مستقیم ویژگی‌های پیچیده تصویر از یک تنسور با ابعاد فضایی 1×1 بود. این رویکرد به مثابه خلق یک ساختار هندسی پیچیده از هیچ است. در واقع، شبکه فاقد یک بوم فضایی اولیه و منطقی بود تا بتواند الگوهای بافتی تومور را به درستی روی آن بسط (Upsampling) دهد و توسعه بافت را پایه‌ریزی کند.
- پیچیدگی مازاد و اضافه‌بار پارامترها (Over-parameterization): استفاده از ۵۱۲ کانال در لایه‌های اولیه برای تولید یک تصویر بسیار کوچک 32×32 ، یک معماری به شدت غول‌پیکر و غیربهبوده ایجاد کرد. این حجم عظیم و غیرضروری از پارامترها به جای کمک به تعمیم‌پذیری مدل (Generalization)، باعث شد شبکه به سرعت ظرفیت یادگیری خود را صرف حفظ کردن (Memorization) کند و به دلیل عدم توانایی در مدیریت این فضای پارامتری وسیع، دچار ناپایداری شده و در دریایی از نویزهای رنگی (RGB Artifacts) واگرا شود.
- خطای استراتژیک در مقیاس‌دهی مصنوعی تصاویر (Artificial Upscaling Fallacy): تصاویر اصلی در مجموعه داده BreastMNIST دارای ابعاد ذاتی 28×28 پیکسل هستند. در این معماری اولیه، با این پیش‌فرض که افزایش ابعاد به 32×32 ممکن است به استخراج ویژگی‌های بهتر (Feature Extraction) کمک کند، تصاویر مقیاس‌دهی (Upsample) شدند. از منظر پردازش سیگنال، بزرگ‌نمایی یک تصویر فراتر از رزولوشن ذاتی آن، هیچ اطلاعات یا فرکانس فضایی جدیدی به آن اضافه نمی‌کند، بلکه تنها مصنوعات درون‌یابی (Interpolation Artifacts) ایجاد می‌کند. در شبکه‌های GAN، متمایزگر به شدت به این مصنوعات مصنوعی حساس است. در نتیجه، شبکه به جای تمرکز بر یادگیری بافت بیولوژیکی و مورفولوژی تومور، درگیر یادگیری و بازتولید نویزهای ناشی از درون‌یابی شد. این عدم تطابق بین فرکانس طبیعی داده‌ها و ابعاد تحمیلی، یکی از دلایل اصلی افت کیفیت بافت در خروجی‌ها است.

در نهایت، ترکیب این معماری بیش از حد سنگین، ایجاد گلوگاه در بستر فضایی اولیه و اعمال فشار نامتعادل در فرآیند آموزش، به



تولید مدلی منجر شد که از نظر تنوری و عملی قادر به حفظ ثبات متخاصم و سنتز تصاویر معنادار پزشکی نبود.



۲.۲.۱ بررسی ساختار شبکه بهبود یافته (رویکرد میانی)

بر مبنای تحلیل‌های آسیب‌شناختی صورت گرفته روی معماری اولیه، مشخص گردید که ریشه بسیاری از ناپایداری‌ها و واگرایی‌های شبکه در اضافه‌بار پارامتری، فقدان بستر فضایی اولیه و عدم تقارن در پویایی یادگیری نهفته است. از این رو، در فاز دوم توسعه، یک بازنگری بنیادین در ساختار شبکه‌ها و استراتژی آموزش اعمال شد. هدف این مرحله، دستیابی به یک معماری چابک‌تر (Lightweight) و بهره‌گیری از تکنیک‌های پایدارسازی تئوریک بود.

۱. بازطراحی معماری شبکه: ایجاد بوم فضایی و بازگشت به ابعاد ذاتی نخستین و مهم‌ترین اصلاح ساختاری، تغییر رویکرد در نقطه شروع شبکه مولد بود. به جای تلاش بی‌ثمر برای استخراج ویژگی از یک نقطه فضایی کور (1×1) ، یک بستر فضایی اولیه (Spatial Canvas) در شبکه تعریف شد.

در این ساختار جدید، بردار نهان ترکیب شده ابتدا از طریق یک لایه تمام‌متصل به یک نقشه ویژگی با ابعاد $7 \times 7 \times 128$ نگاشت می‌شود. این ابعاد اولیه به شبکه اجازه می‌دهد تا الگوهای پایه و فرکانس پایین بافت تومور را پیش از عملیات مقیاس‌دهی (Upsampling) پایه‌ریزی کند. فرآیند بسط فضایی در شبکه مولد بهبود یافته، مسیر زیر را طی می‌کند:

$$(128 \times 7 \times 7) \xrightarrow{Upsample+Conv2d} (64 \times 14 \times 14) \xrightarrow{Conv2d} (32 \times 14 \times 14) \xrightarrow{Upsample+Conv2d} (1 \times 28 \times 28) \quad (۳)$$

این تغییرات سه دستاورد مهم به همراه داشت:

- لایه پردازش میانی $(14 \times 14 \times 32)$: افزودن یک لایه میانی بدون تغییر ابعاد فضایی، ظرفیت پردازشی (Capacity) مولد را برای یادگیری بافت‌های پیچیده‌تر افزایش داد.

- تناسب پارامتری (Right-sizing): کاهش شدید تعداد کانال‌ها (از ۵۱۲ به ۱۲۸ در لایه اول) معماری را با حجم محدود داده‌های BreastMNIST متناسب ساخت و از پدیده بیش‌برازش ساختاری جلوگیری کرد.

- بازگشت به فضای تک‌کاناله (Grayscale): با تنظیم کانال خروجی روی ۱ و ابعاد روی 28×28 ، شبکه دیگر درگیر تولید نویزهای رنگی بی‌معنی نشده و تمام ظرفیت خود را معطوف به یادگیری توزیع طیف خاکستری سونوگرافی نمود.

شبکه متمایزگر نیز به طور کاملاً متقارن ساده‌سازی شد تا در برابر این مولد چابک، رفتاری متعادل‌تر از خود نشان دهد.

۲. بکارگیری تکنیک هموارسازی برچسب‌ها (Label Smoothing) یکی از چالش‌های بنیادین در آموزش شبکه‌های GAN، پدیده اعتماد به نفس کاذب (Overconfidence) در متمایزگر است. در طبقه‌بندی باینری استاندارد، برچسب داده‌های واقعی به صورت مطلق برابر با 1.0 و داده‌های جعلی برابر با 0.0 در نظر گرفته می‌شود. از منظر ریاضیاتی تابع فعال‌سازی Sigmoid، برای آنکه شبکه به خروجی دقیقاً 1.0 دست یابد، باید وزن‌های لایه نهایی خود را به سمت بی‌نهایت میل دهد. این امر موجب می‌شود متمایزگر به جای یادگیری ویژگی‌های تعمیم‌پذیر، کوچکترین نویزهای داده‌های آموزش را حفظ کرده و به شدت مغرور شود.

زمانی که متمایزگر به اطمینان ۱۰۰ درصدی می‌رسد، تابع زیان متخاصم به اشباع رسیده و مشتق آن (گرادیان) به صفر میل می‌کند. در این حالت، پدیده مرگ گرادیان (Vanishing Gradient) رخ داده و شبکه مولد هیچ سیگنال مفیدی برای اصلاح خطاهای خود دریافت نمی‌کند.

برای رفع این بحران، تکنیک Label Smoothing به شبکه تزریق شد. در این روش، به جای استفاده از اهداف مطلق (سخت)، از اهداف نرم استفاده می‌شود:

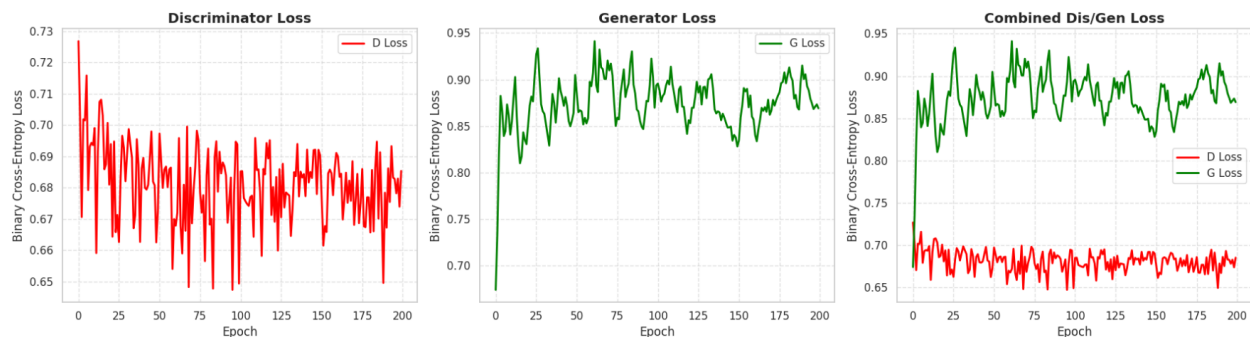
$$y_{real} = 0.9, \quad y_{fake} = 0.0$$

با تنظیم آستانه برچسب‌های واقعی روی 0.9، ما عملاً یک قانون اجباری برای متمایزگر وضع کردیم: «هیچ‌گاه حق نداری با قطعیت ۱۰۰ درصد تصمیم‌گیری کنی؛ همواره حداقل ۱۰ درصد احتمال خطا (Uncertainty) برای خود در نظر بگیر.» این مداخله ریاضی تأثیرات شگرفی در پویایی مدل داشت:

۱. جلوگیری از اشباع تابع زیان: تابع Cross-Entropy با هدف 0.9 هرگز به صفر مطلق نمی‌رسد، بنابراین جریان گرادیان همواره زنده مانده و به طور پیوسته به سمت مولد روانه می‌شود.

۲. کاهش جریمه‌های خشن: زمانی که متمایزگر فروتن باشد، در صورت فریب خوردن از سوی مولد، جریمه‌های ناگهانی و پرش‌های شدید در نمودار Loss ایجاد نمی‌کند، که این امر ثبات آموزش را به طرز چشمگیری افزایش می‌دهد.

جمع‌بندی دستاوردهای معماری میانی بکارگیری همزمان معماری چابک‌سازی‌شده مبتنی بر بوم فضایی 7×7 و تکنیک ریاضی Label Smoothing، توانست تا حد قابل قبولی جلوی واگرایی‌های خشن مدل اولیه را بگیرد و کیفیت بصری بافت‌ها را بهبود بخشد. با این وجود، این ساختار تنها یک بهبود نسبی بود. اگرچه پایداری نسبی حاصل شد، اما مشکل جدیدتری داریم، و اون هم این هست که مدل داده‌های تکراری تولید میکند.

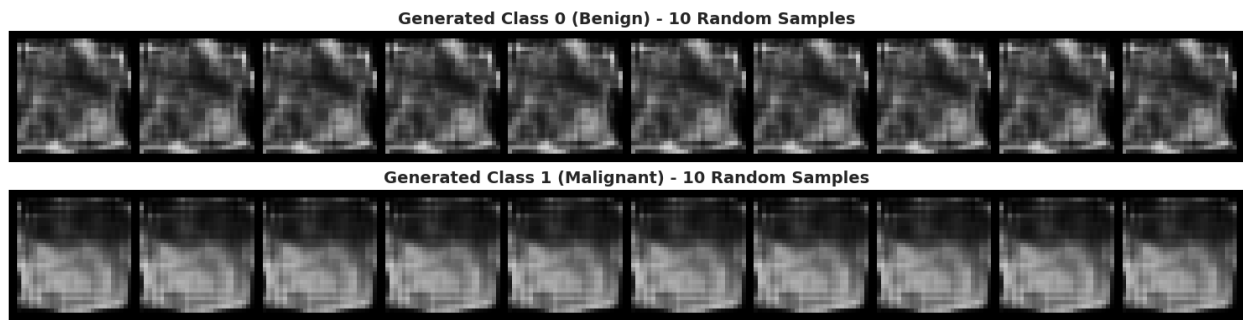


شکل ۱۰: منحنی تغییرات تابع زیان در معماری میانی؛ تثبیت گرادیان‌ها و رسیدن به مجاورت تعادل نش.

بررسی نتایج و تحلیل خروجی‌های معماری میانی در گام نخست ارزیابی، بررسی پویایی توابع زیان (تصویر ۱۰) نشان‌دهنده یک پیشرفت چشمگیر و موفقیت‌آمیز در پایدارسازی ریاضی شبکه است. برخلاف معماری اولیه که در آن شاهد پرش‌های خشن، نوسانات انفجاری و مرگ گرادیان بودیم، در این ساختار جدید، دینامیک یادگیری به یک ثبات بی‌نظیر دست یافته است:

- مهار متمایزگر: به لطف استفاده از تکنیک هموارسازی برچسب‌ها (Label Smoothing = 0.9)، خطای متمایزگر (D Loss) در یک بازه بسیار باریک و ایده‌آل بین 0.65 تا 0.72 نوسان می‌کند. از نظر تئوری ریاضی شبکه‌های GAN، نقطه تعادل نش (Nash Equilibrium) زمانی رخ می‌دهد که متمایزگر کاملاً گیج شده و احتمال 0.5 را خروجی دهد. در تابع BCE Loss، مقدار $-\ln(0.5) \approx 0.69$ است. نوسان خطای متمایزگر دقیقاً حول این عدد، اثبات می‌کند که شبکه از نظر ریاضی در ایده‌آل‌ترین حالت تتوریک خود قرار دارد.

- تثبیت خطای مولد: خطای مولد (G Loss) نیز از رفتارهای واگرایانه فاصله گرفته و با ثباتی منطقی در بازه 0.80 تا 0.95 حرکت می‌کند. این امر نشان می‌دهد که مولد به طور پیوسته سیگنال‌های گرادیان مفیدی دریافت کرده و در حال یادگیری است.



شکل ۱۱: نمونه خروجی‌های معماری میانی؛ بروز مصنوعات شطرنجی و پدیده فروپاشی حالت نسبی.

با این وجود، در دنیای شبکه‌های متخاصم، زیبایی منحنی‌های زیان لزوماً به معنای تولید تصاویر باکیفیت نیست. ارزیابی خروجی‌های بصری شبکه پرده از چالش‌های جدیدی برمی‌دارد. همان‌طور که در تصویر ۱۱ مشاهده می‌شود، با وجود حل مشکل واگرایی و نویزهای رنگی (به دلیل بازگشت به فضای تک‌کاناله)، دو پدیده مخرب بافتی در خروجی‌ها به وضوح قابل تشخیص است:

۱. بروز مجدد فروپاشی حالت (Mode Collapse): تصاویر تولید شده در هر کلاس (و حتی بین دو کلاس) شباهت بسیار زیادی به یکدیگر دارند و تنوع ساختاری (Diversity) در آن‌ها به شدت پایین است. دلیل این امر، یافتن یک «نقطه امن محلی» (Safe Local Minimum) توسط مولد است. از آنجا که متمایزگر به دلیل Label Smoothing تا حدی ملایم و فروتن شده است، مولد متوجه شده که با تولید یک الگوی بافتی مبهم اما ثابت، می‌تواند متمایزگر را فریب دهد. در نتیجه، مولد بردار نویز ورودی (z) را عملاً نادیده گرفته (Latent Code Ignoring) و از ایجاد تنوع بافتی که نیازمند ریسک‌پذیری در فضای بهینه‌سازی است، اجتناب می‌کند.

۲. ظهور مصنوعات شطرنجی (Checkerboard Artifacts): بافت تصاویر تولید شده صاف و طبیعی نیست، بلکه دارای یک شبکه توری‌شکل و پیکسلی است که در ادبیات پردازش تصویر به آن مصنوعات شطرنجی گفته می‌شود. این پدیده یکی از مشکلات کلاسیک در شبکه‌های مولد است و ریشه در مکانیزم مقیاس‌دهی فضایی (Upsampling) دارد. زمانی که ابعاد نقشه ویژگی از 7×7 به 14×14 و سپس 28×28 گسترش می‌یابد، اگر همپوشانی فیلترهای کانولوشنی (به‌ویژه در ConvTranspose2d یا ترکیبات درون‌یابی) به صورت یکنواخت و متقارن توزیع نشود، برخی پیکسل‌ها بیش از حد فعال شده و فرکانس‌های بالای متناوب (شکل شطرنجی) را در تصویر حک می‌کنند. این مصنوعات برای شبکه‌های دسته‌بندی‌کننده نهایی (مانند ResNet) به شدت گمراه‌کننده هستند، زیرا شبکه ممکن است این الگوهای شطرنجی را به اشتباه به عنوان ویژگی‌های بافت تومور (Biomarkers) تفسیر کند.

در یک جمع‌بندی، می‌توان گفت معماری میانی توانست مشکل «پایداری متخاصم» را در سطح گرادینان‌ها حل کند، اما در سطح بازنمایی بصری (Visual Representation) گرفتار حداقل‌های محلی و مصنوعات ساختاری شد. رفع این مشکلات نیازمند تزریق تنوع به مولد (نظیر استفاده از Dropout) و بکارگیری تکنیک‌های پیشرفته‌تر نرمال‌سازی مانند نرمال‌سازی طیفی (Spectral Normalization) در متمایزگر است تا بدون نیاز به هموارسازی افراطی برجسب‌ها، پایداری و کیفیت تصویر به صورت توان‌آمان تضمین گردد.

۳.۲.۱ بررسی ساختار شبکه نهایی (Final cDCGAN Architecture)

پس از تحلیل دقیق محدودیت‌ها و واگرایی‌های مشاهده‌شده در معماری‌های پیشین، شبکه نهایی با هدف دستیابی به یک سنتز شرطی (Class-conditional Synthesis) کاملاً پایدار برای مجموعه داده BreastMNIST بازطراحی گردید. در این فاز، مجموعه‌ای از تکنیک‌های

پیشرفته پایدارسازی (Stabilization Techniques) به صورت مهندسی شده با یکدیگر ترکیب شدند تا رفتار دینامیکی توابع زیان کنترل شده و کیفیت بصری تصاویر به حداکثر برسد.

جدول ۳، پیکربندی نهایی و ابرپارامترهای این معماری بهینه‌شده را نشان می‌دهد.

جدول ۳: ابرپارامترها و مشخصات معماری نهایی شبکه cDCGAN

مقدار	پارامتر / تنظیمات فنی
2	تعداد کلاس‌ها (Number of Classes)
100	ابعاد فضای نهان (Latent Dimension)
28×28	ابعاد تصویر خروجی (Image Size)
1 (Grayscale)	تعداد کانال‌های تصویر (Image Channels)
32	اندازه دسته (Batch Size)
0.0002	نرخ یادگیری مولد (Generator LR)
0.0003	نرخ یادگیری متمایزگر (Discriminator LR)
Adam	الگوریتم بهینه‌سازی (Optimizer)
0.999, 0.5	مومنتم‌های بهینه‌ساز (β_1, β_2)
BCE	تابع خطای معیار (Loss Function)
500	تعداد اپوک‌های آموزش (Epochs)

پایدارسازی و ارتقای این شبکه مدیون شش اصلاح ساختاری و ریاضیاتی است که در ادامه به تفصیل مورد تحلیل قرار می‌گیرند:

۱. جایگزینی **BatchNorm** با نرمال‌سازی طیفی (**Spectral Normalization**) در شبکه‌های GAN، استفاده از نرمال‌سازی دسته‌ای (Batch Normalization) در متمایزگر باعث می‌شود خروجی شبکه برای یک تصویر، به سایر تصاویر موجود در آن دسته (Mini-batch) وابسته شود. این وابستگی، تخمین دقیق توزیع داده‌ها را مختل می‌کند. برای حل این مشکل و همچنین جلوگیری از انفجار گرادیان، در شبکه نهایی تمامی لایه‌های BatchNorm متمایزگر با نرمال‌سازی طیفی (Spectral Normalization) جایگزین شدند. از منظر ریاضی، نرمال‌سازی طیفی ماتریس وزن‌های هر لایه را بر بزرگترین مقدار منفرد آن (شعاع طیفی) تقسیم می‌کند و بدین ترتیب، شبکه را مجبور می‌سازد تا شرط پیوستگی لپشیتز (با کران ثابت $K = 1$) را رعایت کند:

$$\|f(x) - f(y)\| \leq 1 \cdot \|x - y\|$$

این محدودیت سخت‌گیرانه ریاضی باعث می‌شود متمایزگر تغییرات ناگهانی و خشن نداشته باشد و گرادیان‌هایی بسیار نرم، پیوسته و قابل اعتماد به سمت مولد ارسال کند.

۲. تداخل تنظیمی و حذف هموارسازی برچسب‌ها (**Label Smoothing Removal**) در معماری میانی، از تکنیک Label Smoothing (تنظیم تارگت روی 0.9) برای جلوگیری از اعتماد به نفس افراطی متمایزگر استفاده شده بود. اما در معماری نهایی، این تکنیک به طور کامل حذف گردید. دلیل این امر یک تداخل ساختاری است: نرمال‌سازی طیفی (Spectral Norm) به خودی خود با محدود کردن کران لپشیتز، قدرت و اعتماد به نفس متمایزگر را کنترل و مهار می‌کند. اعمال همزمان Label Smoothing در کنار Spectral Norm، متمایزگر

را بیش از حد تضعیف کرده و باعث می‌شود مولد به راحتی بر آن غلبه کند که نتیجه‌ی آن واگرایی مدل است. بنابراین، با اتکا به قدرت نرمال‌سازی طیفی، اهداف متمایزگر به همان حالت خالص (Hard Labels یعنی 1.0 و 0.0) بازگردانده شدند.

۳. کنترل واگرایی با قاعده به‌روزرسانی دو مقیاسی (TTUR) با وجود اصلاحات فوق، در فازهای طولانی‌تر آموزش مشاهده شد که توابع زیان مولد و متمایزگر پس از مدتی از یکدیگر فاصله گرفته و دچار واگرایی می‌شوند. برای حل این چالش، از تکنیک قاعده به‌روزرسانی با دو مقیاس زمانی (Two Time-Scale Update Rule - TTUR) بهره گرفته شد. در این تکنیک، به جای استفاده از نرخ یادگیری یکسان، نرخ یادگیری متمایزگر (0.0003) مقداری بیشتر از مولد (0.0002) تنظیم شد. این رویکرد نامتقارن، تضمین می‌کند که متمایزگر همواره یک قدم از مولد جلوتر بوده و در حالت بهینه محلی خود باقی بماند تا بتواند سیگنال‌های خطای دقیق‌تری برای راهنمایی مولد صادر کند.

۴. اعمال دراپ‌اوت نامتقارن (Asymmetric Dropout) برای جلوگیری از حفظ کردن داده‌های آموزش توسط متمایزگر (بیش‌برازش)، یک لایه Dropout با نرخ 0.3 به لایه‌های پیمشی آن اضافه شد. اما این تکنیک به صورت عمدی در شبکه مولد استفاده نشد. دلیل این تصمیم‌گیری آن است که شبکه مولد برای تولید تصاویری با بافت طبیعی (به ویژه در تصاویر پزشکی که جزئیات بافت‌شناسی اهمیت حیاتی دارند)، نیازمند حفظ پیوستگی فضایی (Spatial Coherence) است. خاموش کردن تصادفی نورون‌ها در مولد باعث ایجاد اختلال در جریان تولید تصویر شده و خروجی‌هایی تار، نویزی و فاقد انسجام هندسی تولید می‌کند.

۵. مقداردهی اولیه استاندارد و کنترل‌شده (Custom Weight Initialization) چارچوب PyTorch به صورت پیش‌فرض از روش‌هایی نظیر کایمینگ (Kaiming/He) برای مقداردهی اولیه وزن‌ها استفاده می‌کند که برای شبکه‌های دسته‌بندی عمیق بهینه شده‌اند، اما شبکه‌های GAN به مقیاس وزن‌های اولیه به شدت حساس هستند. اگر به درستی وزن دهی انجام نشد ممکنه در همون ابتدای شبکه یکی از دیگری قوی‌تر باشه؛ برای رفع این مشکل، یک متد اختصاصی (_weights_init) پیاده‌سازی شد:

- لایه‌های پیمشی (Conv Layers): وزن‌ها از یک توزیع نرمال با میانگین صفر و انحراف معیار 0.02 استخراج می‌شوند: $W \sim \mathcal{N}(0, 0.02^2)$. این مقیاس کوچک از انفجار سیگنال در اولین پاس‌های پیش‌رو (Forward Pass) جلوگیری می‌کند.
- لایه‌های نرمال‌سازی (BatchNorm): برای جلوگیری از صفر شدن و ویژگی‌ها، وزن‌های این لایه (پارامتر گاما) روی توزیع $\mathcal{N}(1, 0.02^2)$ و مقادیر بایاس (پارامتر بتا) دقیقاً روی عدد صفر (0) تنظیم شدند تا توزیع داده‌ها در ابتدا کاملاً دست‌نخورده باقی بماند.

۶. تزریق شرط و تقویت داده‌ها (Conditional Embedding & Data Augmentation) در نهایت، برای تضمین پایداری و تعمیم‌پذیری بیشتر:

- تقویت داده‌ها: در طول فرآیند آموزش، از تکنیک قرینه‌سازی افقی تصادفی (Random Horizontal Flipping) استفاده شد تا تنوع بافتی تصاویری که به متمایزگر می‌رسند افزایش یابد.

۴.۲.۱ بررسی دقیق ساختار شبکه‌های نهایی

در این بخش، با استناد به خروجی‌های تحلیل‌گر ساختاری (torchinfo)، معماری نهایی شبکه‌های مولد و متمایزگر کالبدشکافی می‌شود. این بررسی نشان می‌دهد که چگونه تغییرات اعمال شده توانسته‌اند با بهینه‌سازی جریان تنسورها و کاهش چشمگیر اضافه‌بار پارامتری، مدلی چابک و قدرتمند برای پردازش تصاویر پزشکی خلق کنند.



Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Trainable
ConditionalGenerator	[32, 100]	[32, 1, 28, 28]	--	True
└Embedding: 1-1	[32]	[32, 100]	200	True
└Sequential: 1-2	[32, 200]	[32, 6272]	--	True
└Linear: 2-1	[32, 200]	[32, 6272]	1,260,672	True
└BatchNorm1d: 2-2	[32, 6272]	[32, 6272]	12,544	True
└ReLU: 2-3	[32, 6272]	[32, 6272]	--	--
└Sequential: 1-3	[32, 128, 7, 7]	[32, 1, 28, 28]	--	True
└Upsample: 2-4	[32, 128, 7, 7]	[32, 128, 14, 14]	--	--
└Conv2d: 2-5	[32, 128, 14, 14]	[32, 64, 14, 14]	73,728	True
└BatchNorm2d: 2-6	[32, 64, 14, 14]	[32, 64, 14, 14]	128	True
└ReLU: 2-7	[32, 64, 14, 14]	[32, 64, 14, 14]	--	--
└Upsample: 2-8	[32, 64, 14, 14]	[32, 64, 28, 28]	--	--
└Conv2d: 2-9	[32, 64, 28, 28]	[32, 1, 28, 28]	576	True
└Tanh: 2-10	[32, 1, 28, 28]	[32, 1, 28, 28]	--	--
Total params: 1,347,848				
Trainable params: 1,347,848				
Non-trainable params: 0				
Total mult-adds (Units.MEGABYTES): 517.63				
Input size (MB): 0.01				
Forward/backward pass size (MB): 9.86				
Params size (MB): 5.39				
Estimated Total Size (MB): 15.26				

شکل ۱۲: گزارش ساختاری و ابعاد تنسورها در لایه‌های شبکه مولد نهایی.

۱. شبکه مولد (Generator) همان‌طور که در تصویر ۱۲ مشاهده می‌شود، معماری مولد با حدود ۱.۳۴ میلیون پارامتر آموزش‌پذیر (Trainable Params)، به گونه‌ای طراحی شده است که تعادلی بی‌نظیر میان ظرفیت یادگیری و جلوگیری از بیش‌برازش برقرار کند. جریان اطلاعات در این شبکه طی سه بلوک اصلی پردازش می‌شود:

- بلوک جایگذاری و ترکیب (Embedding & Concatenation): شبکه در ابتدا یک دسته (Batch) شامل ۳۲ بردار نویز ۱۰۰ بعدی (32×100) و ۳۲ برجسب کلاس را دریافت می‌کند. برجسب‌ها با عبور از یک لایه جایگذاری به بردارهای ۱۰۰ بعدی تبدیل می‌شوند (با ۲۰۰ پارامتر تنظیم‌پذیر). با ترکیب این دو، ورودی بلوک بعدی به یک تنسور با ابعاد 32×200 ارتقا می‌یابد.
- بلوک نگاشت فضایی مبنا (Dense Block): تنسور 32×200 وارد یک لایه تمام‌متصل خطی (Linear) می‌شود که آن را به یک فضای ۶۲۷۲ بعدی بسط می‌دهد. این لایه با بیش از ۱.۲ میلیون پارامتر، سنگین‌ترین بخش مولد است و وظیفه دارد ارتباطات غیرخطی اولیه را شکل دهد. پس از عبور از لایه نرمال‌سازی دسته‌ای یک‌بعدی (BatchNorm1d) و تابع ReLU، این بردار مسطح به یک بوم فضایی سه‌بعدی با ابعاد $128 \times 7 \times 7$ تغییر شکل (Reshape) می‌دهد.
- بلوک بسط فضایی و سنتز (Convolutional Upsampling): یکی از مهم‌ترین اصلاحات این معماری، جایگزینی لایه‌های کانولوشن ترانهاد با ترکیب Upsample + Conv2d است. این تغییر استراتژیک به طور کامل از بروز مصنوعات شطرنجی (Checkerboard Artifacts) جلوگیری می‌کند. در گام اول، ابعاد فضایی توسط لایه Upsample دو برابر شده (14×14) و سپس یک لایه کانولوشن استاندارد، عمق کانال‌ها را از ۱۲۸ به ۶۴ کاهش می‌دهد (همراه با BatchNorm2d و ReLU). در گام دوم، ابعاد فضایی مجدداً دو برابر شده (28×28) و آخرین لایه کانولوشن، نقشه ویژگی را به یک تصویر تک‌کاناله نهایی با ابعاد $28 \times 28 \times 1$ تبدیل می‌کند. در نهایت، تابع Tanh توزیع نوری پیکسل‌ها را در بازه استاندارد تنظیم می‌نماید.

۲. شبکه متمایزگر (Discriminator) بر اساس تصویر ۱۳، شبکه متمایزگر به صورت یک مدل بسیار سبک و کارآمد با تنها ۱۴۰,۹۶۱ پارامتر آموزش‌پذیر طراحی شده است. این تفاوت حجم پارامتر (۱۴۰ هزار در برابر ۱.۳۴ میلیون در مولد) یک رویکرد هوشمندانه برای



Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Trainable
ConditionalDiscriminator	[32, 1, 28, 28]	[32, 1]	--	True
└Embedding: 1-1	[32]	[32, 784]	1,568	True
└Sequential: 1-2	[32, 2, 28, 28]	[32, 128, 7, 7]	--	True
└Conv2d: 2-1	[32, 2, 28, 28]	[32, 64, 14, 14]	2,048	True
└LeakyReLU: 2-2	[32, 64, 14, 14]	[32, 64, 14, 14]	--	--
└Dropout: 2-3	[32, 64, 14, 14]	[32, 64, 14, 14]	--	--
└Conv2d: 2-4	[32, 64, 14, 14]	[32, 128, 7, 7]	131,072	True
└LeakyReLU: 2-5	[32, 128, 7, 7]	[32, 128, 7, 7]	--	--
└Dropout: 2-6	[32, 128, 7, 7]	[32, 128, 7, 7]	--	--
└Sequential: 1-3	[32, 128, 7, 7]	[32, 1]	--	True
└Flatten: 2-7	[32, 128, 7, 7]	[32, 6272]	--	--
└Linear: 2-8	[32, 6272]	[32, 1]	6,273	True
└Sigmoid: 2-9	[32, 1]	[32, 1]	--	--
Total params: 140,961				
Trainable params: 140,961				
Non-trainable params: 0				
Total mult-adds (Units.MEGABYTES): 541.32				
Input size (MB): 0.10				
Forward/backward pass size (MB): 5.02				
Params size (MB): 0.56				
Estimated Total Size (MB): 5.68				

شکل ۱۳: گزارش ساختاری و ابعاد تنسورها در لایه‌های شبکه متمایزگر نهایی.

جلوگیری از غلبه زود هنگام متمایزگر است. جریان ارزیابی در این شبکه به شرح زیر است:

- بلوک هم‌آمیزی شرطی (**Conditional Fusion**): برچسب کلاس ورودی از طریق لایه جایگذاری به یک بردار 784×1 می‌شود. از آنجا که ابعاد تصویر ورودی 28×28 (معادل 784 پیکسل) است، این بردار شرطی به یک نقشه فضایی $28 \times 28 \times 1$ تغییر شکل یافته و در کنار تصویر ورودی (تولیدی یا واقعی) قرار می‌گیرد. در نتیجه، تنسور ورودی متمایزگر ساختاری $2 \times 28 \times 28$ با ابعاد $2 \times 28 \times 28$ پیدا می‌کند.

- بلوک استخراج ویژگی (**Feature Extraction**): این بلوک شامل دو لایه کانولوشن استاندارد با گام حرکت (Stride) ۲ است که وظیفه کاهش ابعاد فضایی و استخراج الگوهای بافتی را بر عهده دارند. لایه اول، ابعاد را به $14 \times 14 \times 64$ تقلیل داده و لایه دوم آن را به $7 \times 7 \times 128$ می‌رساند. پس از هر لایه کانولوشن، از تابع فعال‌سازی LeakyReLU برای حفظ گرادین‌های منفی و از لایه Dropout (برای کنترل بیش‌برازش و تضعیف عمدی متمایزگر جهت همگامی با مولد) استفاده شده است.

- بلوک تصمیم‌گیری (**Classification Block**): نقشه ویژگی نهایی ($7 \times 7 \times 128$) توسط لایه Flatten مسطح شده و به یک بردار 6272×1 بعدی تبدیل می‌گردد. در نهایت، یک لایه خطی (Linear) با 6273 پارامتر، این بردار را به یک اسکالر خروجی متراکم می‌کند که با عبور از تابع Sigmoid، احتمال واقعی یا جعلی بودن تصویر را تعیین می‌نماید.

در یک جمع‌بندی ساختاری، انتقال از معماری‌های غول‌پیکر مبتنی بر کانولوشن‌های ترانهاد به این معماری چابک و مبتنی بر Upsampling، نه تنها هزینه‌های محاسباتی (نظیر Forward/Backward pass size که در مجموع زیر ۱۵ مگابایت است) را به شدت کاهش داده، بلکه با ایجاد یک قیف اطلاعاتی منطقی، شبکه را در استخراج و بازسازی ویژگی‌های حیاتی سونوگرافی به حداکثر بهره‌وری رسانده است.

۵.۲.۱ بررسی فرایند آموزش و استراتژی ارزیابی شبکه

فرایند آموزش در شبکه‌های متخاصم مولد (GAN) به دلیل ماهیت بازی کمینه‌بیشینه (Minimax Game) همواره چالش برانگیز است. در این معماری، علاوه بر تنظیم دقیق دینامیک یادگیری بین مولد و متمایزگر، پروتکل‌های پیشرفته‌ای برای آماده‌سازی داده‌ها، ارزیابی پایداری و ذخیره‌سازی هوشمند وزن‌های شبکه پیاده‌سازی شده است که در ادامه به تفصیل مورد بررسی قرار می‌گیرند.

۱. پروتکل آماده‌سازی و تقویت داده‌ها (Data Augmentation & Transformation) یکی از اصول حیاتی در ارزیابی مدل‌های یادگیری عمیق، تفکیک دقیق سیاست‌های پیش‌پردازش بین فاز آموزش (Train) و فاز ارزیابی/آزمون (Validation/Test) است. همان‌طور که در بلوک `data augment` و `transform` مشخص است، این تفکیک به شکل زیر اعمال شده است:

- فاز آموزش (Training): داده‌ها ابتدا به فرمت تک‌کاناله (Grayscale) و ابعاد 28×28 تبدیل شده، سپس یک تبدیل تصادفی قرینه‌سازی افقی (RandomHorizontalFlip) با احتمال ۵۰ درصد روی آن‌ها اعمال می‌گردد. در نهایت، تصاویر به تنسور تبدیل شده و در بازه $[-1, 1]$ نرمال‌سازی می‌شوند (با میانگین و انحراف معیار ۰.۵). این تقویت داده‌ها (Data Augmentation) باعث افزایش تنوع الگوهای بافتی شده و از بیش‌برازش متمایزگر جلوگیری می‌کند.
- فاز ارزیابی و آزمون (Validation/Test): در این فاز، تبدیل قرینه‌سازی تصادفی به صورت عمدی حذف شده است. دلیل این امر، الزام به حفظ ثبات در معیارهای ارزیابی است. اگر بر روی مجموعه اعتبارسنجی نیز تغییرات تصادفی اعمال می‌شد، خروجی مدل در هر اپوک با یک تصویر متفاوت ارزیابی می‌گردید که این مسئله نویز شدیدی به معیارهای سنجش وارد کرده و مقایسه عملکرد مدل در طول زمان را غیرممکن می‌ساخت.

علاوه بر این، در تنظیمات بارگذار داده آموزشی (`'train'`)، پارامتر `'drop last=True'` فعال شده است. از آنجا که در شبکه مولد از لایه‌های نرمال‌سازی دسته‌ای (Batch Normalization) استفاده شده است، این پارامتر تضمین می‌کند که اگر در انتهای مجموعه داده، تعداد تصاویر باقیمانده کمتر از حجم دسته (مثلاً ۳۲) بود، آن دسته کوچک دور ریخته شود. دسته‌های کوچک باعث ایجاد ناپایداری شدید در محاسبات میانگین و واریانس لایه‌های BatchNorm شده و می‌توانند کل فرایند آموزش را دچار فروپاشی کنند.

۲. ساختار حلقه آموزش متخاصم در هر اپوک، شبکه بر اساس پروتکل استاندارد شبکه‌های متخاصم، در دو مرحله متوالی آموزش می‌بیند:

۱. به‌روزرسانی متمایزگر (Train D): ابتدا متمایزگر با یک دسته از تصاویر واقعی و یک دسته از تصاویر تولید شده توسط مولد تغذیه می‌شود. هدف متمایزگر بهینه‌سازی تابع زیان زیر است تا توانایی تفکیک خود را به حداکثر برساند:

$$L_D = \frac{1}{2} [L_{real} + L_{fake}]$$

۲. به‌روزرسانی مولد (Train G): در گام بعدی، مولد با استفاده از یک بردار نویز جدید تصویر تولید می‌کند. هدف مولد فریب دادن متمایزگر است؛ بنابراین تابع زیان آن به گونه‌ای محاسبه می‌شود که خروجی متمایزگر $D(G(z, y))$ به سمت عدد ۱ (واقعی بودن) هدایت شود.

پس از اتمام آموزش روی تمامی دسته‌ها، مدل وارد فاز اعتبارسنجی (Validation) شده و معیار تعادل برای آن محاسبه می‌گردد.

۳. طراحی معیار ارزیابی اختصاصی: امتیاز تعادل (Balance Score) در شبکه‌های عصبی استاندارد (مانند طبقه‌بندها)، همواره مدلی که کمترین خطای اعتبارسنجی (Validation Loss) را داشته باشد به عنوان بهترین مدل انتخاب می‌شود. اما در شبکه‌های GAN، توابع زیان همواره در حال رقابت و نوسان هستند و کاهش خطای مولد لزوماً به معنای تولید تصاویر بهتر نیست. برای حل این چالش، یک تابع ارزیابی اختصاصی تحت عنوان 'balance' gan 'evaluate' توسعه یافت.

این تابع عملکرد شبکه را در برابر مفهوم تئوریک تعادل نش (Nash Equilibrium) می‌سنجد. در این تابع، متمایزگر تصاویری را از مجموعه اعتبارسنجی (Real) و تصاویری که به تازگی توسط مولد سنتز شده‌اند (Fake) دریافت کرده و میانگین احتمالات پیش‌بینی شده را محاسبه می‌کند:

$$P_{real} = \frac{1}{N} \sum D(x)$$

$$P_{fake} = \frac{1}{N} \sum D(G(z, y))$$

در یک شبکه GAN که به ایده‌آل‌ترین حالت خود (تعادل نش) رسیده است، متمایزگر دیگر قادر به تفکیک داده‌های واقعی از جعلی نیست و برای هر دو گروه مقدار احتمال 0.5 را باز می‌گرداند. بر همین اساس، فرمول Balance Score به شکل زیر تعریف شد:

$$BalanceScore = |P_{real} - 0.5| + |P_{fake} - 0.5| \quad (4)$$

تفسیر ریاضی معیار:

- هرچه این امتیاز به صفر نزدیک‌تر باشد، شبکه در وضعیت متعادل‌تری قرار دارد.
- اگر شبکه دچار عدم تعادل شود (مثلاً متمایزگر بسیار قوی شده و $P_{real} = 0.95$ و $P_{fake} = 0.05$ شود)، امتیاز تعادل به شدت افزایش می‌یابد ($Score = 0.45 + 0.45 = 0.90$).

۴. استراتژی ذخیره‌سازی وضعیت (Checkpointing Strategy) بر خلاف رویکردهای سنتی، در این پیاده‌سازی، شرط ذخیره کردن «بهترین مدل» بر روی کمینه‌سازی خطای مولد استوار نیست، بلکه الگوریتم به دنبال یافتن نقطه‌ای است که در آن کمترین میزان امتیاز تعادل ثبت شود (Minimum Balance Score).

در تابع traincDCGAN، در انتهای هر اپوک یک دیکشنری جامع شامل شماره اپوک، ماتریس وزن‌های هر دو شبکه (State Dicts)، میانگین توابع زیان و امتیاز تعادل تشکیل شده و تحت سه فرمت استراتژیک در دیسک ذخیره می‌شود:

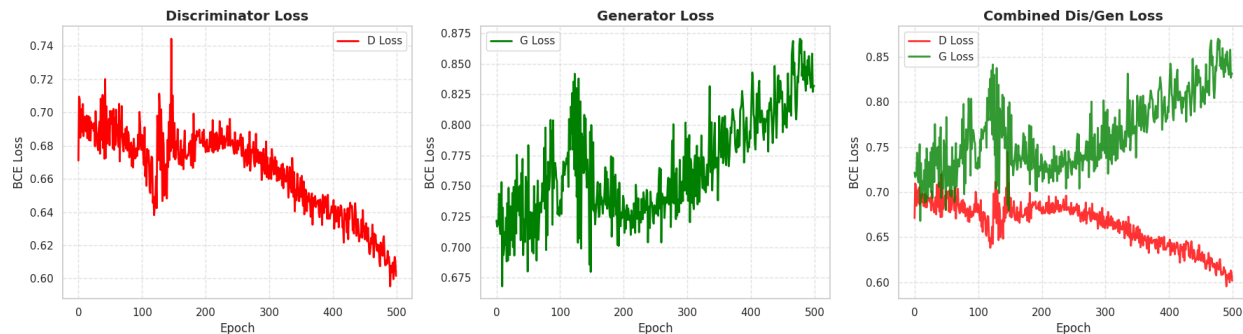
۱. `checkpoint_epoch_X.pth`: ذخیره وضعیت دقیق هر اپوک، جهت فراهم آوردن امکان بازگشت (Rollback) به هر نقطه دلخواه از تاریخچه آموزش.

۲. `last_model.pth`: یک فایل بازنویسی‌شونده (Overwriting) که همواره وضعیت آخرین اپوک اجرا شده را در خود نگه می‌دارد و برای ادامه آموزش (Resume Training) در صورت قطع فرآیند کاربرد دارد.

۳. `best_model.pth`: این فایل تنها زمانی بروزرسانی می‌شود که مدل فعلی توانسته باشد یک رکورد جدید در نزدیک شدن به تعادل نش (کاهش Balance Score) ثبت کند. این دقیقاً همان مدلی است که در فاز نهایی برای استخراج ویژگی و تولید تصویر مورد استفاده قرار خواهد گرفت.

۶.۲.۱ بررسی نتایج نهایی مدل

پس از اعمال تغییرات بنیادین در معماری، بهینه‌سازی جریان تنسورها و تزریق تکنیک‌های پایدارسازی ریاضی، مدل نهایی به مدت ۵۰۰ اپوک تحت آموزش قرار گرفت. در این بخش، رفتار دینامیکی توابع زیان و کیفیت بصری تصاویر سنتز شده به عنوان خروجی‌های نهایی این چارچوب مورد تحلیل و ارزیابی قرار می‌گیرند.



شکل ۱۴: منحنی تغییرات توابع زیان متمایزگر و مولد در طول ۵۰۰ اپوک در معماری نهایی.

۱. تحلیل پویایی توابع زیان (Loss Dynamics Analysis) نمودار ۱۴ نمایانگر یک رفتار بسیار پایدار، کنترل‌شده و کلاسیک از یک شبکه GAN موفق است. با واکاوی این نمودار، تأثیر مستقیم تصمیمات معماری بر رفتار شبکه به وضوح قابل مشاهده است:

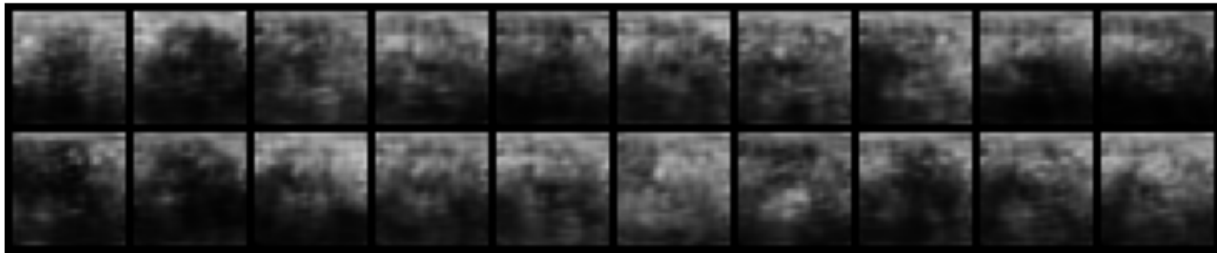
- تثبیت در مجاورت تعادل نش (Nash Equilibrium): در ۱۰۰ اپوک ابتدایی، خطای هر دو شبکه حول مقدار ۰.۶۹ تا ۰.۷۲ نوسان می‌کند. در تابع زیان BCE، زمانی که متمایزگر نتواند داده واقعی را از جعلی تشخیص دهد (احتمال ۰.۵)، مقدار خطا برابر با $-\ln(0.5) \approx 0.693$ خواهد بود. این نوسانات اولیه نشان می‌دهد که شبکه از همان ابتدا در یک تعادل تئوریک ایده‌آل کار خود را آغاز کرده است.

- تأثیر شگرف نرمال‌سازی طیفی (Spectral Normalization): برخلاف مدل اولیه که شاهد پرش‌های انفجاری و مرگ گرادیان (خطای بالای ۵.۲) بودیم، در اینجا نمودار فاقد هرگونه پرش خشن است. محدودیت لپشیتز که توسط نرمال‌سازی طیفی بر متمایزگر تحمیل شده، باعث شده است تا متمایزگر نتواند با قطعیت ۱۰۰ درصد تصمیم‌گیری کند. این امر یک جریان گرادیان (Gradient Flow) پیوسته، نرم و بسیار غنی را در تمام طول ۵۰۰ اپوک برای مولد تضمین کرده است.

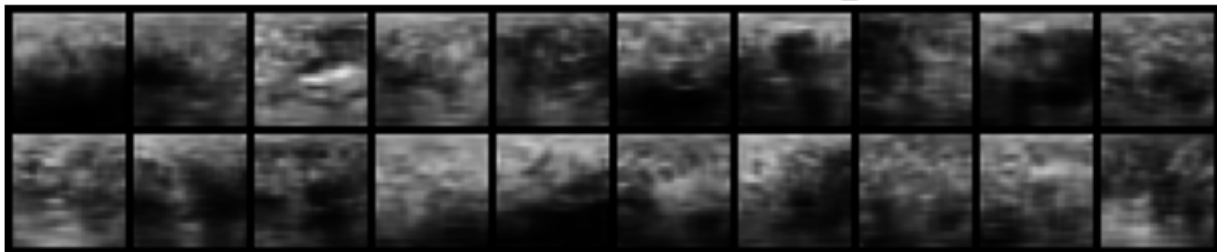
- جدایی کنترل‌شده به واسطه قاعده TTUR: از اپوک ۲۰۰ به بعد، مشاهده می‌شود که خطای متمایزگر به آرامی در حال کاهش (میل به سمت ۰.۶۰) و خطای مولد به تدریج در حال افزایش (میل به سمت ۰.۸۵) است. این پدیده دقیقاً خروجی مطلوب تکنیک TTUR (نرخ یادگیری متمایزگر ۰.۰۰۰۳ در برابر مولد ۰.۰۰۰۲) است. متمایزگر به دلیل نرخ یادگیری کمی بالاتر، همواره یک قدم از مولد جلوتر است و به مرور زمان برتری خفیفی پیدا می‌کند. این «برتری کنترل‌شده» بسیار حیاتی است؛ زیرا اگر متمایزگر ضعیف شود، نمی‌تواند ایرادات ریز تصاویر تولیدی را تشخیص دهد. در اینجا، متمایزگر به صورت آهسته سخت‌گیرتر می‌شود و مولد را مجبور می‌کند تا در اپوک‌های پایانی به جزئیات بسیار ظریف‌تری توجه کند، بدون آنکه منجر به فروپاشی شبکه شود.

۲. ارزیابی کیفی خروجی‌های بصری (Visual Quality Assessment) همان‌طور که در تصویر ۱۵ مشهود است، خروجی‌های مدل نهایی از نظر کیفی و ساختاری به یک بلوغ کامل رسیده‌اند. با مقایسه این خروجی‌ها با مدل‌های پیشین، دستاوردهای زیر مسجل می‌گردد:

Generated Class 0 (Benign) - last_model.pth



Generated Class 1 (Malignant) - last_model.pth



شکل ۱۵: تصاویر 28×28 سنتز شده توسط شبکه نهایی تفکیک شده بر اساس کلاس های خوش خیم و بدخیم.

۱. درمان قطعی فروپاشی حالت (Mode Collapse Resolution): در این تصاویر، هیچ گونه شباهت مطلق یا کپی برداری در بین نمونه های تولید شده دیده نمی شود. هر یک از ۲۰ تصویر نمایش داده شده در هر کلاس، دارای مورفولوژی، توزیع نوری و شکل هندسی کاملاً منحصر به فردی است. این تنوع ساختاری (Structural Diversity) اثبات می کند که شبکه مولد توانسته است به درستی کل فضای نهان ($z \in \mathbb{R}^{100}$) را به توزیع داده های واقعی نگاشت کند و در حداقل های محلی گرفتار نشده است.

۲. حذف مصنوعات شطرنجی و رنگی (Artifact Elimination): جایگزینی لایه های کانولوشن ترانهاد با معماری مبتنی بر Upsample، به طور کامل شبکه های توری شکل و مصنوعات شطرنجی را از بین برده است. بافت تصاویر، همانند سونوگرافی های واقعی، دارای نویزهای نقطه ای طبیعی (Speckle Noise) و گرادیان های نوری پیوسته است. همچنین، بازگشت به فضای تک کاناله (Grayscale) باعث تمرکز کامل ظرفیت شبکه بر روی اکوژنیسیته (Echogenicity) تومورها شده است.

در نتیجه، معماری نهایی با مهار ریاضیاتی گرادیان ها و بهینه سازی جریان اطلاعات فضایی، توانسته است تصاویری تولید کند که نه تنها از نظر تنوع و کیفیت بصری در سطح بسیار بالایی قرار دارند، بلکه از نظر ویژگی های مورفولوژیک پزشکی نیز کاملاً با کلاس شرطی داده شده منطبق هستند.

۷.۲.۱ تولید ۲۰۰۰ داده های مصنوعی و توازن بخشی به مجموعه داده آموزش

یکی از چالش های بنیادین در مجموعه داده های پزشکی نظیر BreastMNIST، عدم توازن شدید کلاس ها (Class Imbalance) است. در مجموعه داده آموزش اولیه، تعداد نمونه های بدخیم (۳۹۹ تصویر) به مراتب بیشتر از نمونه های خوش خیم (۱۴۷ تصویر) بود. این اختلاف فاحش باعث می شود مدل های دسته بند (نظیر ResNet) دچار سوگیری (Bias) به سمت کلاس اکثریت شوند. برای حل این بحران، از شبکه cDCGAN نهایی به عنوان یک موتور تولید داده و از یک استراتژی توازن بخشی اولویت محور (Priority-based Balancing) استفاده گردید.

۱. استراتژی تولید استخر داده‌ها (تولید ۲۰۰۰ نمونه) ابتدا با استفاده از مدل، یک «استخر داده‌های مصنوعی» (Synthetic Data Pool) ایجاد شد. به این منظور، شبکه مولد در حالت ارزیابی (eval) قرار گرفت و با تزریق بردارهای نویز تصادفی از توزیع نرمال، تعداد ۲۰۰۰ تصویر مجزا برای کلاس خوش خیم و ۲۰۰۰ تصویر برای کلاس بدخیم تولید گردید.

۲. استراتژی توازن بخشی اولویت محور (Target-Driven Balancing) پس از تولید داده‌ها، استراتژی ساخت مجموعه داده نهایی بر مبنای حفظ حداکثری اصالت داده‌ها بنا شد. در این روش، یک سقف هدف ($T = 600$) برای هر کلاس در نظر گرفته شد. الگوریتم توازن بخشی به این صورت عمل می‌کند که ابتدا تمامی داده‌های واقعی موجود در آن کلاس را وارد مجموعه نهایی می‌کند و تنها برای پر کردن خلأ موجود تا رسیدن به سقف هدف، از داده‌های مصنوعی استفاده می‌نماید. رابطه ریاضی این استراتژی به شکل زیر است:

$$N_{fake} = \max(0, T - N_{real})$$

بر اساس لاگ‌های اجرایی، توزیع داده‌ها به شکل زیر مهندسی شد:

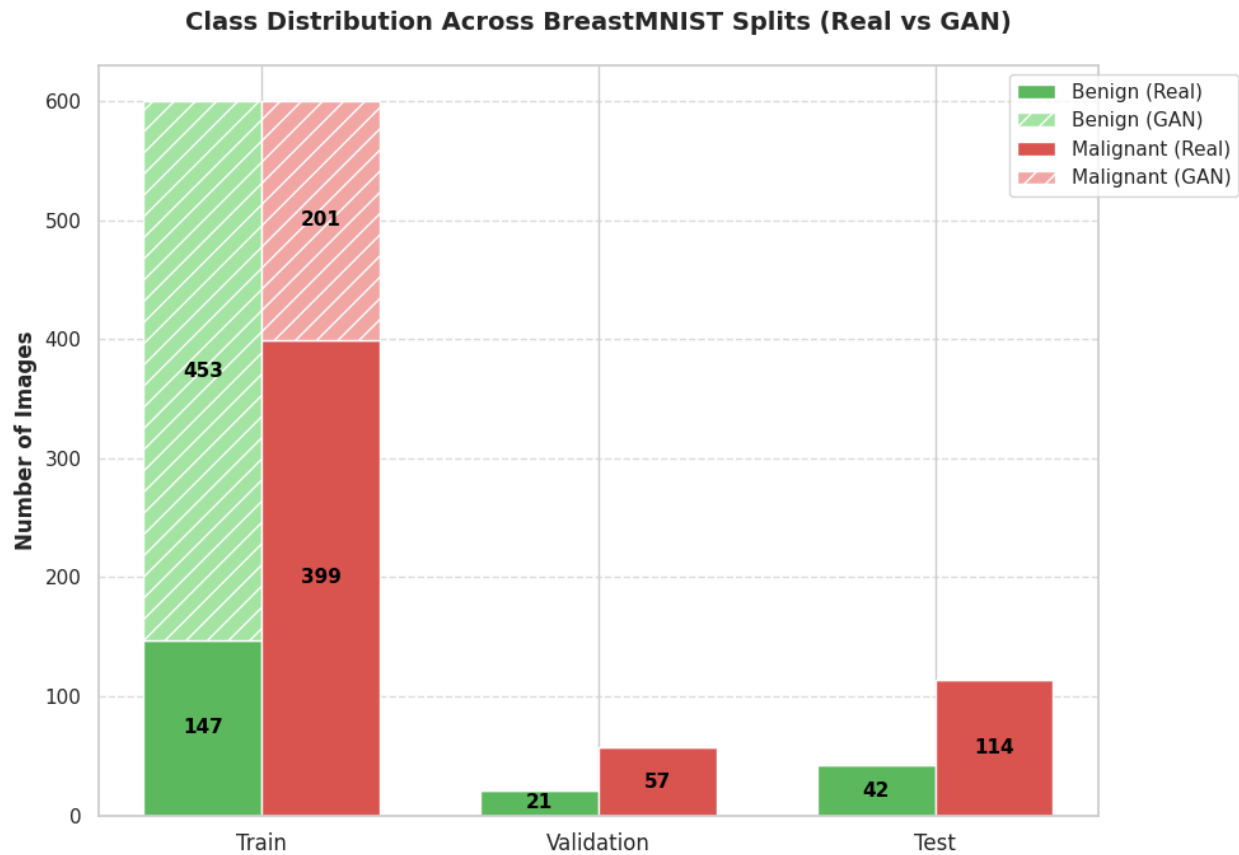
- کلاس ۰ (خوش خیم): از آنجا که تنها ۱۴۷ تصویر واقعی وجود داشت، دقیقاً ۴۵۳ تصویر مصنوعی به صورت تصادفی از استخر داده‌ها انتخاب و به این کلاس تزریق شد تا به عدد ۶۰۰ برسد.
- کلاس ۱ (بدخیم): با وجود ۳۹۹ تصویر واقعی، تنها به ۲۰۱ تصویر مصنوعی نیاز بود تا این کلاس نیز دقیقاً به سقف ۶۰۰ نمونه برسد.

در نهایت، یک مجموعه داده آموزشی کاملاً متوازن شامل ۱۲۰۰ نمونه (۶۰۰ خوش خیم و ۶۰۰ بدخیم) شکل گرفت.

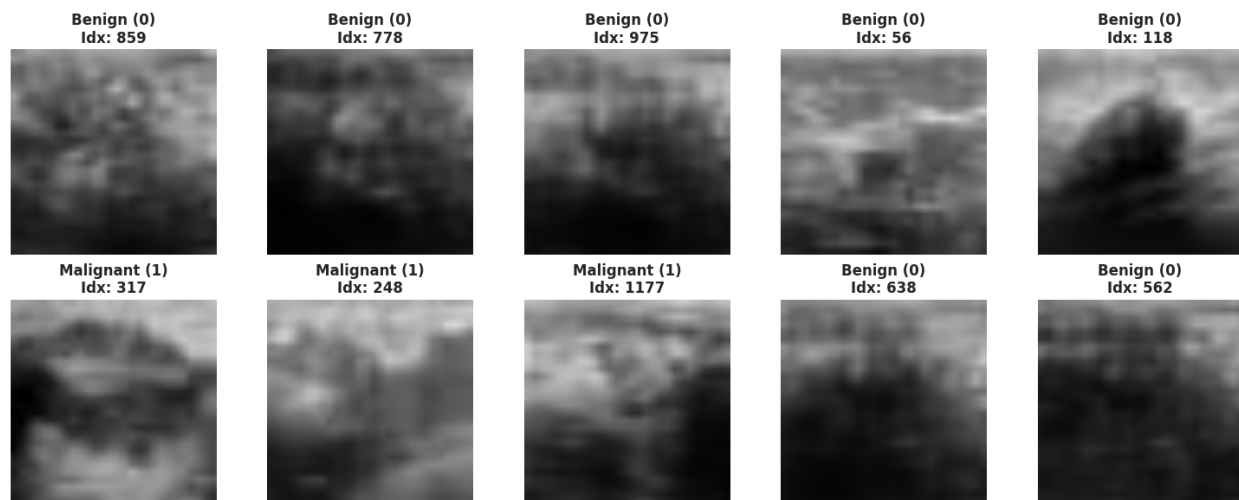
۳. منطق محدودسازی تقویت داده‌ها صرفاً به فاز آموزش یکی از اصول تخطی‌ناپذیر در متدولوژی یادگیری ماشین بالینی، عدم دستکاری مجموعه داده‌های اعتبارسنجی (Validation) و آزمون (Test) است. تزریق داده‌های GAN یا هرگونه توازن بخشی تنها و تنها روی مجموعه آموزش (Train) اعمال شد. دلیل این امر آن است که مدل باید در یک محیط غنی و متوازن «آموزش» ببیند تا مرزهای تصمیم‌گیری (Decision Boundaries) را بدون سوگیری یاد بگیرد؛ اما مرحله «ارزیابی» باید دقیقاً بازتاب‌دهنده‌ی واقعیت نامتوازن دنیای کلینیکال باشد. اگر داده‌های آزمون نیز با GAN بالانس می‌شدند، معیارهای ارزیابی نهایی (Precision, Recall) مخدوش شده و عملکرد مدل در دنیای واقعی به صورت کاذب، ایده‌آل به نظر می‌رسید (Data Leakage).

۴. تحلیل توزیع کلاس‌ها و بررسی کیفی داده‌های ترکیبی همان‌طور که در نمودار میله‌ای انباشته در تصویر ۱۶ مشاهده می‌شود، معماری داده‌ها به شفاف‌ترین شکل ممکن مصور شده است. در ستون مربوط به آموزش (Train)، هر دو کلاس دقیقاً در مرز ۶۰۰ نمونه با یکدیگر برابر شده‌اند. بخش‌های توپر (سبز و قرمز تیره) نمایانگر داده‌های اصیل کلینیکی، و بخش‌های هاشورخورده (سبز و قرمز روشن) نشان‌دهنده سهم داده‌های مصنوعی هستند. این نمودار به وضوح نشان می‌دهد که کلاس خوش خیم اتکای بیشتری به داده‌های GAN داشته است، در حالی که مجموعه‌های اعتبارسنجی و آزمون با همان توزیع نامتوازن و طبیعی اولیه خود دست‌نخورده باقی مانده‌اند.

تصویر ۱۷ نتیجه نهایی این ادغام را از منظر بصری به نمایش می‌گذارد. در این ماتریس تصویری، داده‌های واقعی و داده‌های تولید شده توسط cDCGAN در کنار یکدیگر و در قالب یک مجموعه داده واحد (DataLoader) قرار گرفته‌اند. پیوستگی بافتی (Textural Consistency)، حفظ کنتراست استاندارد سونوگرافی و عدم وجود مصنوعات مصنوعی باعث شده است تا تفکیک چشمی تصاویر واقعی از تصاویر تولیدی GAN تقریباً غیرممکن باشد. این یکپارچگی بصری، بالاترین تضمین برای کیفیت داده‌های سنتر شده است و بستر ایده‌آلی را برای آموزش شبکه‌های دسته‌بند عمیق نظیر ResNet فراهم می‌آورد.



شکل ۱۶: توزیع کلاس‌ها در بخش‌های مختلف مجموعه داده BreastMNIST پس از اعمال توازن‌بخشی در فاز آموزش. بخش‌های هاشورخورده نمایانگر سهم تصاویر تولید شده توسط GAN هستند.



شکل ۱۷: نمونه‌هایی تصادفی از مجموعه داده متوازن شده نهایی؛ ترکیبی یکپارچه از تصاویر واقعی و مصنوعی.

۸.۲.۱ ارزیابی نهایی: تحلیل عملکرد طبقه‌بند ResNet (داده‌های اصیل در برابر داده‌های افزوده با GAN)

پس از موفقیت در تولید تصاویر مصنوعی و ساخت مجموعه داده متوازن، مرحله نهایی ارزیابی به بررسی تأثیر این توازن بخشی بر روی عملکرد شبکه‌های تشخیص کلینیکی اختصاص یافت. برای این منظور، یک معماری ResNet یک بار بر روی مجموعه داده نامتوازن اولیه (Baseline) و بار دیگر بر روی مجموعه داده متوازن‌شده با cDCGAN آموزش دید و نتایج آن بر روی مجموعه آزمون (Test Set) دست‌نخورده مقایسه گردید.

جدول ۴ خلاصه‌ای از معیارهای ارزیابی آماری شامل دقت پیش‌بینی (Precision)، حساسیت (Recall) و امتیاز ترکیبی (F1-Score) را برای هر دو رویکرد نشان می‌دهد.

جدول ۴: مقایسه عملکرد ResNet روی مجموعه داده نامتوازن (اولیه) و متوازن (با GAN)

مدل متوازن‌شده (با GAN)			مدل پایه (داده‌های نامتوازن)			معیار ارزیابی
F1-score	Recall	Precision	F1-score	Recall	Precision	
0.67	0.57	0.80	0.73	0.69	0.78	کلاس ۰ (خوش‌خیم)
0.90	0.95	0.86	0.91	0.93	0.89	کلاس ۱ (بدخیم)
	0.85			0.87		دقت کل (Accuracy)
	0.78			0.82		میانگین کلان (Macro Avg F1)

با کالبدشکافی دقیق این نتایج، در نگاه اول به نظر می‌رسد که دقت کل (Accuracy) در مدل متوازن‌شده با GAN حدود ۲ درصد افت داشته است (از 0.87 به 0.85). با این حال، در حوزه تصویربرداری پزشکی و تشخیص سرطان، تحلیل خام دقت کل گمراه‌کننده است و باید پویایی جابجایی خطاها بین دو کلاس به صورت علمی تفسیر شود:

۱. دستاورد بالینی: افزایش حساسیت در تشخیص سرطان (کلاس بدخیم) مهم‌ترین و حیاتی‌ترین شاخص در غربالگری تومورها، شاخص Recall (یا حساسیت - Sensitivity) در کلاس بدخیم است. این شاخص نشان می‌دهد که مدل چند درصد از بیماران سرطانی واقعی را به درستی تشخیص داده است. همان‌طور که در جدول مشخص است، تزریق داده‌های GAN باعث ارتقای Recall کلاس بدخیم از 0.93 به 0.95 شده است. از منظر پزشکی، این یک موفقیت بسیار بزرگ محسوب می‌شود؛ زیرا خطای نوع دوم یا منفی کاذب (False Negative - یعنی تشخیص تومور سرطانی به عنوان بافت سالم) هزینه‌ای معادل با به خطر افتادن جان بیمار دارد. داده‌های مصنوعی تولید شده توسط cDCGAN با ایجاد تنوع بیشتر در مورفولوژی تومورهای بدخیم، به ResNet کمک کردند تا الگوهای پنهان سرطان را بهتر بشناسد و بیماران سرطانی کمتری را از دست بدهد.

۲. مصالحه آماری (Trade-off): افت Recall در کلاس خوش‌خیم افزایش حساسیت به تومورهای بدخیم همواره با یک مصالحه (Trade-off) همراه است. در مدل متوازن‌شده، شاخص Recall برای کلاس خوش‌خیم از 0.69 به 0.57 افت کرده است. این بدان معناست که مدل در برابر توده‌های خوش‌خیم به شدت محتاط و بدبین شده است و تمایل دارد موارد مشکوک را «بدخیم» اعلام کند. ریشه این رفتار در توزیع داده‌های مصنوعی نهفته است. از آنجا که برای رسیدن به مرز ۶۰۰ نمونه، ۴۵۳ نمونه مصنوعی به کلاس خوش‌خیم (نسبت به تنها ۲۰۱ نمونه بدخیم مصنوعی) افزوده شد، حجم عظیمی از ویژگی‌های سنتز شده وارد فضای این کلاس گردید. به نظر می‌رسد همپوشانی لبه‌های تصمیم‌گیری (Decision Boundary Overlap) در داده‌های مصنوعی خوش‌خیم با داده‌های بدخیم، باعث شده تا طبقه‌بند نهایی (ResNet) آستانه تصمیم‌گیری خود را به نفع کلاس بدخیم تغییر دهد.



۳. ارتقای دقت پیش‌بینی کلاس خوش‌خیم نکته جالب توجه دیگر، افزایش Precision کلاس خوش‌خیم از 0.78 به 0.80 در مدل متوازن است. این یعنی زمانی که مدل جدید ما توده‌ای را «خوش‌خیم» اعلام می‌کند، با اطمینان و قطعیت بسیار بیشتری می‌توان به این تشخیص اعتماد کرد، زیرا هشدار کاذب کمتری در این جهت صادر می‌کند.

جمع‌بندی در ارزیابی نهایی، اگرچه تزریق داده‌های مصنوعی تولید شده توسط cDCGAN باعث کاهش ۲ درصدی در دقت کلی (Accuracy) مدل گردید، اما این رویکرد به صورت هدفمند، سوگیری (Bias) مدل را به نفع «تشخیص ایمن‌تر» تغییر داد. در کاربردهای کلینیکال، مدلی که Recall سرطان در آن ۹۵ درصد باشد اما هشدارهای کاذب (False Positives) بیشتری تولید کند (مدل مبتنی بر GAN)، به مراتب ارزشمندتر و قابل‌اعتمادتر از مدلی است که دقت کلی بالاتری دارد اما بیماران سرطانی بیشتری را از دست می‌دهد (مدل نامتوازن اولیه). بنابراین، این آزمایش پتانسیل بالای شبکه‌های متخصص را در بازطراحی مرزهای تصمیم‌گیری به نفع پروتکل‌های ایمنی پزشکی به اثبات می‌رساند.



۳.۱ کد کامل سوال اول

```
1 # -*- coding: utf-8 -*-
2 """Final_CDCGAN.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1PY295jk7bthZU323yya6vUlrOPnh5mSy
8
9 Install Libraries
10 """
11
12 !pip install -q medmnist torch torchvision matplotlib seaborn scikit-learn torchinfo
13
14 """Import Libraries and set Device"""
15
16 import torch
17 import torch.nn as nn
18 import torch.optim as optim
19 from torch.utils.data import DataLoader
20 from torchvision import models, transforms, datasets
21 from medmnist import BreastMNIST
22 import matplotlib.pyplot as plt
23 import seaborn as sns
24 from sklearn.metrics import confusion_matrix
25 import numpy as np
26 from torchinfo import summary
27 import os
28 import random
29 import pandas as pd
30 from sklearn.metrics import classification_report, confusion_matrix
31 import torch
32 from typing import Dict
33 import torch.nn.utils as utils
34
35
36
37 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
38 print(f"Using device: {device}")
```

```
۳۹
۴۰ """Set seed to stop variation in results"""
۴۱
۴۲ def set_seed(seed=42):
۴۳     """Sets the seed for reproducibility across all libraries."""
۴۴
۴۵     random.seed(seed)
۴۶     np.random.seed(seed)
۴۷     torch.manual_seed(seed)
۴۸     torch.cuda.manual_seed(seed)
۴۹     torch.cuda.manual_seed_all(seed)
۵۰     torch.backends.cudnn.deterministic = True
۵۱     torch.backends.cudnn.benchmark = False
۵۲     os.environ['PYTHONHASHSEED'] = str(seed)
۵۳
۵۴     print(f"Random seed set to: {seed}")
۵۵
۵۶ set_seed(42)
۵۷
۵۸ """Load and organize the dataset into specified subfolders"""
۵۹
۶۰ base_dir = "breast_mnist_images"
۶۱
۶۲ def extract_and_save(dataset, split_name):
۶۳     print(f"Creating folders and saving {split_name} data...")
۶۴     for i in range(len(dataset)):
۶۵         # The raw medmnist dataset returns a PIL Image and an array containing the label
۶۶         img, label = dataset[i]
۶۷         class_id = label[0]
۶۸
۶۹         # Create the directory structure: base_dir / split / class_id
۷۰         folder_path = os.path.join(base_dir, split_name, str(class_id))
۷۱         os.makedirs(folder_path, exist_ok=True)
۷۲
۷۳         # Save the image into its specific folder
۷۴         img_path = os.path.join(folder_path, f"{split_name}_img_{i}.jpg")
۷۵         img.save(img_path)
۷۶
۷۷ #Load the RAW datasets
۷۸ raw_train = BreastMNIST(split="train", download=True)
```



```
۷۹ raw_val = BreastMNIST(split="val", download=True)
۸۰ raw_test = BreastMNIST(split="test", download=True)
۸۱
۸۲ extract_and_save(raw_train, "train")
۸۳ extract_and_save(raw_val, "val")
۸۴ extract_and_save(raw_test, "test")
۸۵
۸۶ """create train, val, test datasets and dataloaders"""
۸۷
۸۸ data_transform = transforms.Compose([
۸۹     transforms.Grayscale(num_output_channels=3),
۹۰     transforms.Resize((224, 224)),
۹۱     transforms.ToTensor(),
۹۲     transforms.Normalize(mean=[485.0, 456.0, 406.0], std=[229.0, 224.0, 225.0])
۹۳ ])
۹۴
۹۵ base_dir = "breast_mnist_images"
۹۶
۹۷ # Load datasets directly from the newly created folders
۹۸ train_dataset = datasets.ImageFolder(os.path.join(base_dir, 'train'), transform=data_transform)
۹۹ val_dataset = datasets.ImageFolder(os.path.join(base_dir, 'val'), transform=data_transform)
۱۰۰ test_dataset = datasets.ImageFolder(os.path.join(base_dir, 'test'), transform=data_transform)
۱۰۱
۱۰۲ # Package into DataLoaders
۱۰۳ batch_size = 32
۱۰۴ train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
۱۰۵ val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
۱۰۶ test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
۱۰۷
۱۰۸ """visualize some samples from dataset to see what kind of data we have"""
۱۰۹
۱۱۰ def visualize_dataset_samples(train_set, val_set, test_set):
۱۱۱     # 1. Combine datasets into a single list with a tracking tag
۱۱۲     # Each item in the pool is a tuple: (image_tensor, label, split_tag)
۱۱۳     pool = []
۱۱۴     pool.extend([(img, lbl, "Train") for img, lbl in train_set])
۱۱۵     pool.extend([(img, lbl, "Val") for img, lbl in val_set])
۱۱۶     pool.extend([(img, lbl, "Test") for img, lbl in test_set])
۱۱۷
۱۱۸     # 2. Randomly select 25 unique samples for our 5x5 grid
```



```
119     if len(pool) < 25:
120         raise ValueError("Not enough images in the dataset to create a 5x5 grid.")
121     samples = random.sample(pool, 25)
122
123     # 3. Class names for BreastMNIST: 0 is 'benign' (normal/benign), 1 is 'malignant'
124     # For BreastMNIST, 0 = benign, 1 = malignant
125     class_names = {0: "Benign/Normal", 1: "Malignant"}
126
127     # 4. Initialize the 5x5 matplotlib figure
128     fig, axes = plt.subplots(5, 5, figsize=(15, 15))
129     axes = axes.flatten() # Flatten 2D array of axes to 1D for easy iteration
130
131     for i, (img_tensor, label, split) in enumerate(samples):
132         ax = axes[i]
133
134         # PyTorch tensors are (Channels, Height, Width)
135         # Matplotlib expects (Height, Width, Channels) for plotting
136         # We also need to un-normalize the image if we want to view it realistically,
137         # but since we just want to inspect the structure, a basic permutation or clip works:
138         img_array = img_tensor.permute(1, 2, 0).numpy()
139         # Clip values to [0, 1] range to avoid matplotlib warnings if values went slightly out of
140         # ↳ bounds due to normalization
141         img_array = (img_array - img_array.min()) / (img_array.max() - img_array.min() + 1e-8)
142
143
144         ax.imshow(img_array)
145         ax.axis('off')
146         class_name = class_names.get(label, f"Class {label}")
147         ax.set_title(f"[{split}]\n{class_name}", fontsize=10, pad=8)
148
149     plt.tight_layout()
150     plt.show()
151
152 visualize_dataset_samples(train_dataset, val_dataset, test_dataset)
153
154 """checking the class imbalance issue; most of the medical datasets have this problem"""
155
156 def get_label_counts(dataset, split_name):
157     # Iterate through the dataset and extract the label
```

```

۱۵۸     labels = []
۱۵۹     for _, label in dataset:
۱۶۰         # Handle both raw MedMNIST arrays and standard PyTorch integer labels
۱۶۱         if isinstance(label, list) or hasattr(label, '__iter__'):
۱۶۲             labels.append(label[0])
۱۶۳         else:
۱۶۴             labels.append(label)
۱۶۵
۱۶۶     # Count occurrences of each class
۱۶۷     return pd.Series(labels).value_counts().rename(split_name)
۱۶۸
۱۶۹
۱۷۰ train_counts = get_label_counts(train_dataset, 'Train')
۱۷۱ val_counts = get_label_counts(val_dataset, 'Validation')
۱۷۲ test_counts = get_label_counts(test_dataset, 'Test')
۱۷۳
۱۷۴ # Combine the counts into a single DataFrame
۱۷۵ df = pd.concat([train_counts, val_counts, test_counts], axis=1).fillna(0).astype(int)
۱۷۶
۱۷۷ # Map the numerical indices to their clinical names
۱۷۸ df.index = ['Malignant (Class 1)' if i == 1 else 'Benign (Class 0)' for i in df.index]
۱۷۹
۱۸۰ # Reshape the data for Seaborn
۱۸۱ df_melted = df.reset_index().melt(id_vars='index', var_name='Split', value_name='Number of
    ↳ Images')
۱۸۲ df_melted.rename(columns={'index': 'Tumor Type'}, inplace=True)
۱۸۳
۱۸۴ plt.figure(figsize=(10, 6))
۱۸۵ sns.set_theme(style="whitegrid")
۱۸۶ ax = sns.barplot(data=df_melted, x='Split', y='Number of Images', hue='Tumor Type',
    ↳ palette='Set2')
۱۸۷
۱۸۸ plt.title('Class Distribution Across BreastMNIST Splits', fontsize=14, pad=15)
۱۸۹ plt.ylabel('Count', fontsize=12)
۱۹۰ plt.xlabel('Dataset Split', fontsize=12)
۱۹۱
۱۹۲ # Add the exact number on top of each bar for clarity
۱۹۳ for p in ax.patches:
۱۹۴     height = p.get_height()
۱۹۵     if height > 0:
    
```



```
196         ax.annotate(f"{int(height)}",
197                     (p.get_x() + p.get_width() / 2., height),
198                     ha='center', va='center',
199                     xytext=(0, 8), textcoords='offset points',
200                     fontsize=10, fontweight='bold')
201
202 plt.tight_layout()
203 plt.show()
204
205 """check dataset info to realize what the size of our images tensors are and make sure our
    ↳ transformation is applied."""
206
207 def print_dataset_info(dataset, split_name):
208     print(f"--- {split_name.upper()} DATASET INFO ---")
209
210     # 1. Total number of samples
211     num_samples = len(dataset)
212     print(f"Total Images: {num_samples}")
213
214     # 2. Inspect the first image and label
215     first_image, first_label = dataset[0]
216
217     # 3. Image Shape / Dimensions
218     # PyTorch tensors are in [Channels, Height, Width] format
219     print(f"Image Tensor Shape: {first_image.shape}")
220
221     # 4. Data Type
222     print(f"Data Type: {first_image.dtype}")
223
224     # 5. Class Information
225     if hasattr(dataset, 'class_to_idx'):
226         print(f"Classes Found: {dataset.classes}")
227     else:
228         # Fallback for raw MedMNIST dataset
229         print(f"Sample Label Format: {first_label}")
230
231     print("-" * 30 + "\n")
232
233 print_dataset_info(train_dataset, "Train")
234 print_dataset_info(val_dataset, "Validation")
```



```
۲۳۵ print_dataset_info(test_dataset, "Test")
۲۳۶
۲۳۷ total_images = len(train_dataset) + len(val_dataset) + len(test_dataset)
۲۳۸ print(f"Total Images Across All Splits: {total_images}")
۲۳۹
۲۴۰ """Load resnet18 with pretrained weights and change the head to be matched with our dataset"""
۲۴۱
۲۴۲ model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
۲۴۳ # Change the last layer of the model to recognize 2 classes of our data (by default it recognizes
→ 1000 classes)
۲۴۴ num_fts = model.fc.in_features
۲۴۵ num_classes = len(train_dataset.classes)
۲۴۶ print(f"Number of classes recognized: {num_classes}")
۲۴۷ model.fc = nn.Linear(num_fts, num_classes)
۲۴۸
۲۴۹ model = model.to(device)
۲۵۰
۲۵۱ criterion = nn.CrossEntropyLoss()
۲۵۲ optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
۲۵۳
۲۵۴ """Exporting a summary of our model Structure"""
۲۵۵
۲۵۶ # pass an input size matching our DataLoader: (batch_size, channels, height, width)
۲۵۷ summary(model, input_size=(32, 3, 224, 224),
۲۵۸         col_names=["input_size", "output_size", "num_params", "trainable"],
۲۵۹         col_width=20,
۲۶۰         row_settings=["var_names"])
۲۶۱
۲۶۲ """Training, Validating, and saving best model"""
۲۶۳
۲۶۴ epochs = 10
۲۶۵ train_acc_history = []
۲۶۶ val_acc_history = []
۲۶۷
۲۶۸ train_loss_history = []
۲۶۹ val_loss_history = []
۲۷۰ best_val_loss = float('inf')
۲۷۱ for epoch in range(epochs):
۲۷۲     model.train()
۲۷۳     running_corrects = 0
```

```
۲۷۴     running_loss = 0.0
۲۷۵     total_train = 0
۲۷۶
۲۷۷
۲۷۸     for inputs, targets in train_loader:
۲۷۹
۲۸۰         inputs, targets = inputs.to(device), targets.to(device)
۲۸۱
۲۸۲         optimizer.zero_grad()
۲۸۳         outputs = model(inputs)
۲۸۴         loss = criterion(outputs, targets)
۲۸۵         loss.backward()
۲۸۶         optimizer.step()
۲۸۷
۲۸۸         _, preds = torch.max(outputs, 1)
۲۸۹         running_corrects += torch.sum(preds == targets)
۲۹۰         running_loss += loss.item() * inputs.size(0)
۲۹۱         total_train += targets.size(0)
۲۹۲
۲۹۳     train_acc = running_corrects.double() / total_train
۲۹۴     train_loss = running_loss / total_train
۲۹۵
۲۹۶     train_acc_history.append(train_acc.item())
۲۹۷     train_loss_history.append(train_loss)
۲۹۸
۲۹۹     model.eval()
۳۰۰     val_corrects = 0
۳۰۱     val_loss = 0.0
۳۰۲     total_val = 0
۳۰۳
۳۰۴     with torch.no_grad():
۳۰۵         for inputs, targets in val_loader:
۳۰۶             inputs, targets = inputs.to(device), targets.to(device)
۳۰۷
۳۰۸             outputs = model(inputs)
۳۰۹             loss = criterion(outputs, targets)
۳۱۰
۳۱۱             _, preds = torch.max(outputs, 1)
۳۱۲             val_corrects += torch.sum(preds == targets)
۳۱۳             val_loss += loss.item() * inputs.size(0)
```



```
314         total_val += targets.size(0)
315
316     val_acc = val_corrects.double() / total_val
317     epoch_val_loss = val_loss / total_val
318
319     val_acc_history.append(val_acc.item())
320     val_loss_history.append(epoch_val_loss)
321
322     print(f'Epoch [{epoch+1}/{epochs}] | Train Loss: {train_loss:.4f} | Train Acc:
    ↳ {train_acc:.4f} | Val Loss: {epoch_val_loss:.4f} | Val Acc: {val_acc:.4f}')
323     if epoch_val_loss < best_val_loss:
324         best_val_loss = epoch_val_loss
325         torch.save(model.state_dict(), 'best_breast_mnist_model_resnet.pth')
326         print(" -> Best model saved!")
327
328     """Load best model and have a validation on test set"""
329
330     model.load_state_dict(torch.load('best_breast_mnist_model_resnet.pth'))
331
332     model.eval()
333     test_corrects = 0
334     total_test = 0
335
336     all_preds = []
337     all_targets = []
338
339     with torch.no_grad():
340         for inputs, targets in test_loader:
341             inputs, targets = inputs.to(device), targets.to(device)
342
343             outputs = model(inputs)
344             _, preds = torch.max(outputs, 1)
345
346             test_corrects += torch.sum(preds == targets)
347             total_test += targets.size(0)
348
349             all_preds.extend(preds.cpu().numpy())
350             all_targets.extend(targets.cpu().numpy())
351
352     test_acc = test_corrects.double() / total_test
```



```
۳۵۳ print(f'Test Accuracy (Best Model): {test_acc:4f.}')
۳۵۴
۳۵۵ """Classification report and visualization"""
۳۵۶
۳۵۷ print("\n--- Classification Report ---")
۳۵۸ print(classification_report(all_targets, all_preds, target_names=['Benign (0)', 'Malignant
→ (1)']))
۳۵۹ cm = confusion_matrix(all_targets, all_preds)
۳۶۰
۳۶۱ plt.figure(figsize=(6, 5))
۳۶۲ sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
۳۶۳             xticklabels=['Predicted Malignant', 'Predicted Benign'],
۳۶۴             yticklabels=['Actual Malignant', 'Actual benign'])
۳۶۵ plt.title('Test Set Confusion Matrix')
۳۶۶ plt.show()
۳۶۷
۳۶۸ """Visualize and Validation"""
۳۶۹
۳۷۰ sns.set_theme(style="whitegrid", palette="deep")
۳۷۱ fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6), dpi=100)
۳۷۲ epochs_range = range(1, epochs + 1)
۳۷۳
۳۷۴ ax1.plot(epochs_range, train_acc_history, label='Train Accuracy', marker='o', linewidth=2.5,
→ markersize=7)
۳۷۵ ax1.plot(epochs_range, val_acc_history, label='Validation Accuracy', marker='s', linewidth=2.5,
→ markersize=7)
۳۷۶
۳۷۷ best_acc_epoch = np.argmax(val_acc_history) + 1
۳۷۸ best_acc = max(val_acc_history)
۳۷۹ ax1.scatter(best_acc_epoch, best_acc, s=200, facecolors='none', edgecolors='green',
۳۸۰             linewidth=2, zorder=5, label=f'Best Accuracy ({best_acc:2f.})')
۳۸۱
۳۸۲ ax1.set_title('ResNet18 Accuracy', fontsize=16, fontweight='bold', pad=15)
۳۸۳ ax1.set_xlabel('Epochs', fontsize=13)
۳۸۴ ax1.set_ylabel('Accuracy', fontsize=13)
۳۸۵ ax1.set_xticks(epochs_range)
۳۸۶ ax1.legend(loc='lower right', fontsize=11, frameon=True, shadow=True)
۳۸۷
۳۸۸
```



```
۳۸۹ ax2.plot(epochs_range, train_loss_history, label='Train Loss', marker='o', linewidth=2.5,
    ↪ markersize=7)
۳۹۰ ax2.plot(epochs_range, val_loss_history, label='Validation Loss', marker='s', linewidth=2.5,
    ↪ markersize=7)
۳۹۱
۳۹۲ best_loss_epoch = np.argmin(val_loss_history) + 1
۳۹۳ best_loss = min(val_loss_history)
۳۹۴ ax2.scatter(best_loss_epoch, best_loss, s=200, facecolors='none', edgecolors='red',
۳۹۵             linewidth=2, zorder=5, label=f'Best Model Saved ({best_loss:2f.})')
۳۹۶
۳۹۷ ax2.set_title('ResNet18 Cross-Entropy Loss', fontsize=16, fontweight='bold', pad=15)
۳۹۸ ax2.set_xlabel('Epochs', fontsize=13)
۳۹۹ ax2.set_ylabel('Loss', fontsize=13)
۴۰۰ ax2.set_xticks(epochs_range)
۴۰۱ ax2.legend(loc='upper right', fontsize=11, frameon=True, shadow=True)
۴۰۲
۴۰۳
۴۰۴ fig.suptitle('BreastMNIST Training Performance Dashboard', fontsize=20, fontweight='black',
    ↪ y=1.05)
۴۰۵
۴۰۶ plt.tight_layout()
۴۰۷ plt.show()
۴۰۸
۴۰۹ """smooth"""
۴۱۰
۴۱۱ model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
۴۱۲
۴۱۳ num_ftrs = model.fc.in_features
۴۱۴ num_classes = len(train_dataset.classes)
۴۱۵ print(f"Number of classes recognized: {num_classes}")
۴۱۶ model.fc = nn.Linear(num_ftrs, num_classes)
۴۱۷
۴۱۸ model = model.to(device)
۴۱۹
۴۲۰ criterion = nn.CrossEntropyLoss(label_smoothing=1.0)
۴۲۱ optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
۴۲۲
۴۲۳ epochs = 10
۴۲۴ train_acc_history = []
۴۲۵ val_acc_history = []
```




```
۴۲۶ train_loss_history = []
۴۲۷ val_loss_history = []
۴۲۸ best_val_loss = float('inf')
۴۲۹
۴۳۰ for epoch in range(epochs):
۴۳۱     model.train()
۴۳۲     running_corrects = 0
۴۳۳     running_loss = 0.0
۴۳۴     total_train = 0
۴۳۵
۴۳۶     for inputs, targets in train_loader:
۴۳۷         inputs, targets = inputs.to(device), targets.to(device)
۴۳۸
۴۳۹         optimizer.zero_grad()
۴۴۰         outputs = model(inputs)
۴۴۱         loss = criterion(outputs, targets)
۴۴۲         loss.backward()
۴۴۳         optimizer.step()
۴۴۴
۴۴۵         _, preds = torch.max(outputs, 1)
۴۴۶         running_corrects += torch.sum(preds == targets)
۴۴۷         running_loss += loss.item() * inputs.size(0)
۴۴۸         total_train += targets.size(0)
۴۴۹
۴۵۰     train_acc = running_corrects.double() / total_train
۴۵۱     train_loss = running_loss / total_train
۴۵۲
۴۵۳     train_acc_history.append(train_acc.item())
۴۵۴     train_loss_history.append(train_loss)
۴۵۵
۴۵۶     model.eval()
۴۵۷     val_corrects = 0
۴۵۸     val_loss = 0.0
۴۵۹     total_val = 0
۴۶۰
۴۶۱     with torch.no_grad():
۴۶۲         for inputs, targets in val_loader:
۴۶۳             inputs, targets = inputs.to(device), targets.to(device)
۴۶۴
۴۶۵             outputs = model(inputs)
```



```
۴۶۶         loss = criterion(outputs, targets)
۴۶۷
۴۶۸         _, preds = torch.max(outputs, 1)
۴۶۹         val_corrects += torch.sum(preds == targets)
۴۷۰         val_loss += loss.item() * inputs.size(0)
۴۷۱         total_val += targets.size(0)
۴۷۲
۴۷۳     val_acc = val_corrects.double() / total_val
۴۷۴     epoch_val_loss = val_loss / total_val
۴۷۵
۴۷۶     val_acc_history.append(val_acc.item())
۴۷۷     val_loss_history.append(epoch_val_loss)
۴۷۸
۴۷۹     print(f'Epoch [{epoch+1}/{epochs}] | Train Loss: {train_loss:.4f} | Train Acc:
→ {train_acc:.4f} | Val Loss: {epoch_val_loss:.4f} | Val Acc: {val_acc:.4f}')
۴۸۰
۴۸۱     if epoch_val_loss < best_val_loss:
۴۸۲         best_val_loss = epoch_val_loss
۴۸۳
۴۸۴         torch.save(model.state_dict(), 'best_breast_mnist_model_resnet_smoothed.pth')
۴۸۵         print("Best smoothed model saved!")
۴۸۶
۴۸۷     model.load_state_dict(torch.load('best_breast_mnist_model_resnet_smoothed.pth'))
۴۸۸
۴۸۹     model.eval()
۴۹۰     test_corrects = 0
۴۹۱     total_test = 0
۴۹۲
۴۹۳     all_preds = []
۴۹۴     all_targets = []
۴۹۵
۴۹۶     with torch.no_grad():
۴۹۷         for inputs, targets in test_loader:
۴۹۸             inputs, targets = inputs.to(device), targets.to(device)
۴۹۹
۵۰۰             outputs = model(inputs)
۵۰۱             _, preds = torch.max(outputs, 1)
۵۰۲
۵۰۳             test_corrects += torch.sum(preds == targets)
۵۰۴             total_test += targets.size(0)
```



```
۵۰۵
۵۰۶     all_preds.extend(preds.cpu().numpy())
۵۰۷     all_targets.extend(targets.cpu().numpy())
۵۰۸
۵۰۹ test_acc = test_corrects.double() / total_test
۵۱۰ print(f'Test Accuracy (Smoothed Model): {test_acc:4f.}')
۵۱۱
۵۱۲ test_acc = test_corrects.double() / total_test
۵۱۳ print(f'Test Accuracy (Smoothed Model): {test_acc:4f.}')
۵۱۴
۵۱۵ print("\n--- Classification Report ---")
۵۱۶ print(classification_report(all_targets, all_preds, target_names=['Benign (0)', 'Malignant
→ (1)']))
۵۱۷ cm = confusion_matrix(all_targets, all_preds)
۵۱۸
۵۱۹ plt.figure(figsize=(6, 5))
۵۲۰ sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
۵۲۱             xticklabels=['Predicted Malignant', 'Predicted benign'],
۵۲۲             yticklabels=['Actual Malignant', 'Actual Benign'])
۵۲۳ plt.title('Test Set Confusion Matrix (Label Smoothed)')
۵۲۴ plt.show()
۵۲۵
۵۲۶ """C DCGANS"""
۵۲۷
۵۲۸ class ConditionalGenerator(nn.Module):
۵۲۹     def __init__(self, num_classes: int, latent_dim: int):
۵۳۰         super().__init__()
۵۳۱         self.latent_dim = latent_dim
۵۳۲
۵۳۳         self.label_embedding = nn.Embedding(num_classes, latent_dim)
۵۳۴
۵۳۵         self.fc_block = nn.Sequential(
۵۳۶             nn.Linear(latent_dim * 2, 6272),
۵۳۷             nn.BatchNorm1d(6272),
۵۳۸             nn.ReLU(inplace=True)
۵۳۹         )
۵۴۰
۵۴۱         self.conv_blocks = nn.Sequential(
۵۴۲
۵۴۳             nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False),
```



```
۵۴۴         nn.Conv2d(128, 64, kernel_size=3, stride=1, padding=1, bias=False),
۵۴۵         nn.BatchNorm2d(64),
۵۴۶         nn.ReLU(inplace=True),
۵۴۷
۵۴۸
۵۴۹         nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False),
۵۵۰         nn.Conv2d(64, 1, kernel_size=3, stride=1, padding=1, bias=False),
۵۵۱         nn.Tanh()
۵۵۲     )
۵۵۳
۵۵۴     def forward(self, noise: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
۵۵۵         embedded_labels = self.label_embedding(labels)
۵۵۶         x = torch.cat([noise, embedded_labels], dim=1)
۵۵۷         x = self.fc_block(x)
۵۵۸         x = x.view(-1, 128, 7, 7)
۵۵۹         return self.conv_blocks(x)
۵۶۰
۵۶۱     import torch
۵۶۲     import torch.nn as nn
۵۶۳     from torch.nn.utils import spectral_norm
۵۶۴
۵۶۵     class ConditionalDiscriminator(nn.Module):
۵۶۶         def __init__(self, num_classes: int):
۵۶۷             super().__init__()
۵۶۸
۵۶۹             self.label_embedding = nn.Embedding(num_classes, 784)
۵۷۰
۵۷۱             self.conv_blocks = nn.Sequential(
۵۷۲
۵۷۳                 spectral_norm(nn.Conv2d(2, 64, kernel_size=4, stride=2, padding=1, bias=False)),
۵۷۴                 nn.LeakyReLU(2.0, inplace=True),
۵۷۵                 nn.Dropout(3.0),
۵۷۶
۵۷۷                 spectral_norm(nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1, bias=False)),
۵۷۸                 nn.LeakyReLU(2.0, inplace=True),
۵۷۹                 nn.Dropout(3.0)
۵۸۰             )
۵۸۱
۵۸۲             self.fc_block = nn.Sequential(
۵۸۳                 nn.Flatten(),
```



```
۵۸۴         nn.Linear(6272, 1),
۵۸۵         nn.Sigmoid()
۵۸۶     )
۵۸۷
۵۸۸     def forward(self, img: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
۵۸۹         embedded_labels = self.label_embedding(labels).view(-1, 1, 28, 28)
۵۹۰         x = torch.cat([img, embedded_labels], dim=1)
۵۹۱         x = self.conv_blocks(x)
۵۹۲         return self.fc_block(x)
۵۹۳
۵۹۴ import torch
۵۹۵ import torch.nn as nn
۵۹۶ import torch.optim as optim
۵۹۷ from typing import Dict
۵۹۸
۵۹۹ class CDCModel:
۶۰۰     """
۶۰۱     Wrapper class managing the initialization, hyperparameter configurations,
۶۰۲     and adversarial training loop steps for the cDCGAN framework.
۶۰۳     """
۶۰۴
۶۰۵     def __init__(self, num_classes: int, latent_dim: int, lr_g: float = 0002.0,
۶۰۶                 lr_d: float = 0003.0, beta1: float = 5.0, device: str = "cuda"):
۶۰۷
۶۰۸         self.device = torch.device(device if torch.cuda.is_available() else "cpu")
۶۰۹         self.latent_dim = latent_dim
۶۱۰
۶۱۱         # Instantiate networks based on our customized classes
۶۱۲         self.generator = ConditionalGenerator(
۶۱۳             num_classes=num_classes,
۶۱۴             latent_dim=latent_dim
۶۱۵         ).to(self.device)
۶۱۶
۶۱۷         self.discriminator = ConditionalDiscriminator(
۶۱۸             num_classes=num_classes
۶۱۹         ).to(self.device)
۶۲۰
۶۲۱         # Apply formal weights initialization pass
۶۲۲         self.generator.apply(self._weights_init)
۶۲۳         self.discriminator.apply(self._weights_init)
```



```
۶۲۴
۶۲۵     # Loss and Optimizers
۶۲۶     self.criterion = nn.BCELoss()
۶۲۷
۶۲۸     self.opt_G = optim.Adam(self.generator.parameters(), lr=lr_g, betas=(beta1, 999.0))
۶۲۹     self.opt_D = optim.Adam(self.discriminator.parameters(), lr=lr_d, betas=(beta1, 999.0))
۶۳۰
۶۳۱     @staticmethod
۶۳۲     def _weights_init(m: nn.Module) -> None:
۶۳۳         """Applies formal normal weight initializations to convolutional blocks."""
۶۳۴         classname = m.__class__.__name__
۶۳۵         if classname.find('Conv') != -1:
۶۳۶             nn.init.normal_(m.weight.data, 0.0, 0.2)
۶۳۷         elif classname.find('BatchNorm') != -1:
۶۳۸             nn.init.normal_(m.weight.data, 0.1, 0.2)
۶۳۹             nn.init.constant_(m.bias.data, 0)
۶۴۰
۶۴۱     def train_step(self, real_imgs: torch.Tensor, labels: torch.Tensor) -> Dict[str, float]:
۶۴۲         """
۶۴۳         Executes a highly balanced adversarial training step.
۶۴۴         """
۶۴۵         batch_size = real_imgs.size(0)
۶۴۶
۶۴۷
۶۴۸         valid = torch.full((batch_size, 1), 0.1, device=self.device)
۶۴۹         fake = torch.zeros((batch_size, 1), device=self.device)
۶۵۰
۶۵۱         real_imgs = real_imgs.to(self.device)
۶۵۲         labels = labels.to(self.device)
۶۵۳
۶۵۴
۶۵۵     # Train Discriminator
۶۵۶     self.opt_D.zero_grad()
۶۵۷
۶۵۸     validity_real = self.discriminator(real_imgs, labels)
۶۵۹     d_real_loss = self.criterion(validity_real, valid)
۶۶۰
۶۶۱     z = torch.randn(batch_size, self.latent_dim, device=self.device)
۶۶۲     gen_imgs = self.generator(z, labels)
۶۶۳     validity_fake = self.discriminator(gen_imgs.detach(), labels)
```



```
۶۶۴         d_fake_loss = self.criterion(Validity_fake, fake)
۶۶۵
۶۶۶         d_loss = (d_real_loss + d_fake_loss) / 2
۶۶۷
۶۶۸         d_loss.backward()
۶۶۹         self.opt_D.step()
۶۷۰
۶۷۱
۶۷۲         # Train Generator
۶۷۳         self.opt_G.zero_grad()
۶۷۴
۶۷۵         valid_g = torch.ones((batch_size, 1), device=self.device)
۶۷۶
۶۷۷         validity_gen = self.discriminator(gen_imgs, labels)
۶۷۸         g_loss = self.criterion(validity_gen, valid_g)
۶۷۹
۶۸۰         g_loss.backward()
۶۸۱         self.opt_G.step()
۶۸۲
۶۸۳         return {"d_loss": d_loss.item(), "g_loss": g_loss.item()}
۶۸۴
۶۸۵     def generate(self, num_samples: int, class_label: int) -> torch.Tensor:
۶۸۶         """Generates synthetic evaluations for testing or sampling checks."""
۶۸۷         self.generator.eval()
۶۸۸         with torch.no_grad():
۶۸۹             z = torch.randn(num_samples, self.latent_dim, device=self.device)
۶۹۰             labels = torch.full((num_samples,), class_label, dtype=torch.long,
۶۹۱             ↪ device=self.device)
۶۹۱             generated_imgs = self.generator(z, labels)
۶۹۲             self.generator.train()
۶۹۳             return generated_imgs
۶۹۴
۶۹۵ from torchinfo import summary
۶۹۶
۶۹۷ gan = CDCModel(
۶۹۸     num_classes=2,
۶۹۹     latent_dim=100,
۷۰۰     device="cuda"
۷۰۱ )
۷۰۲
```



```

۷۰۳ print("GENERATOR ARCHITECTURE:\n")
۷۰۴
۷۰۵ summary(gan.generator, input_data=[torch.randn(32, 100).to(gan.device), torch.randint(0, 2,
→ (32,)).to(gan.device)],
۷۰۶         col_names=["input_size", "output_size", "num_params", "trainable"])
۷۰۷
۷۰۸ print("DISCRIMINATOR ARCHITECTURE:\n")
۷۰۹ summary(gan.discriminator, input_data=[torch.randn(32, 1, 28, 28).to(gan.device),
→ torch.randint(0, 2, (32,)).to(gan.device)],
۷۱۰         col_names=["input_size", "output_size", "num_params", "trainable"])
۷۱۱
۷۱۲ data_transform = transforms.Compose([
۷۱۳     transforms.Grayscale(num_output_channels=1),
۷۱۴     transforms.Resize((28, 28)),
۷۱۵     transforms.ToTensor(),
۷۱۶     transforms.Normalize(mean=[5.0], std=[5.0])
۷۱۷ ])
۷۱۸
۷۱۹ data_transform_augment = transforms.Compose([
۷۲۰     transforms.Grayscale(num_output_channels=1),
۷۲۱     transforms.Resize((28, 28)),
۷۲۲     transforms.RandomHorizontalFlip(p=5.0),
۷۲۳     transforms.ToTensor(),
۷۲۴     transforms.Normalize(mean=[5.0], std=[5.0])
۷۲۵ ])
۷۲۶ base_dir = "breast_mnist_images"
۷۲۷
۷۲۸ train_dataset = datasets.ImageFolder(os.path.join(base_dir, 'train'),
→ transform=data_transform_augment)
۷۲۹ val_dataset = datasets.ImageFolder(os.path.join(base_dir, 'val'), transform=data_transform)
۷۳۰ test_dataset = datasets.ImageFolder(os.path.join(base_dir, 'test'), transform=data_transform)
۷۳۱
۷۳۲ batch_size = 32
۷۳۳ # Drop_last=True is highly recommended in GANs to avoid batch normalization issues with small
→ final batches
۷۳۴ train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, drop_last=True)
۷۳۵ val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
۷۳۶ test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
۷۳۷
۷۳۸ import os

```




```
۷۳۹ import torch
۷۴۰
۷۴۱ def evaluate_gan_balance(gan, val_loader, device):
۷۴۲     """
۷۴۳     Validation step: Calculates how close the GAN is to the ideal Nash Equilibrium.
۷۴۴     """
۷۴۵     gan.generator.eval()
۷۴۶     gan.discriminator.eval()
۷۴۷
۷۴۸     total_real_prob = 0.0
۷۴۹     total_fake_prob = 0.0
۷۵۰     num_batches = 0
۷۵۱
۷۵۲     with torch.no_grad():
۷۵۳         for real_imgs, labels in val_loader:
۷۵۴             real_imgs = real_imgs.to(device)
۷۵۵             labels = labels.to(device)
۷۵۶             batch_size = real_imgs.size(0)
۷۵۷
۷۵۸             # Discriminator's average prediction on REAL images
۷۵۹             real_preds = gan.discriminator(real_imgs, labels)
۷۶۰             total_real_prob += real_preds.mean().item()
۷۶۱
۷۶۲             # Discriminator's average prediction on FAKE images
۷۶۳             z = torch.randn(batch_size, gan.latent_dim, device=device)
۷۶۴             fake_imgs = gan.generator(z, labels)
۷۶۵             fake_preds = gan.discriminator(fake_imgs, labels)
۷۶۶             total_fake_prob += fake_preds.mean().item()
۷۶۷
۷۶۸             num_batches += 1
۷۶۹
۷۷۰     avg_real_prob = total_real_prob / num_batches
۷۷۱     avg_fake_prob = total_fake_prob / num_batches
۷۷۲
۷۷۳     # Ideal equilibrium is 5.0 for both. Lower score means closer to ideal (Best Model).
۷۷۴     balance_score = abs(avg_real_prob - 5.0) + abs(avg_fake_prob - 5.0)
۷۷۵
۷۷۶     gan.generator.train()
۷۷۷     gan.discriminator.train()
۷۷۸
```



```
779     return avg_real_prob, avg_fake_prob, balance_score
780
781
782 def train_cDCGAN(gan, train_loader, val_loader, epochs: int, device: str):
783
784     save_dir = "checkpoints"
785     os.makedirs(save_dir, exist_ok=True)
786     best_balance_score = float('inf')
787
788     print(f"Starting Training for {epochs} Epochs on {device}...\n")
789
790     for epoch in range(epochs):
791
792         epoch_d_loss = 0.0
793         epoch_g_loss = 0.0
794
795         for real_imgs, labels in train_loader:
796
797             losses = gan.train_step(real_imgs, labels)
798
799             epoch_d_loss += losses['d_loss']
800             epoch_g_loss += losses['g_loss']
801
802         avg_d_loss = epoch_d_loss / len(train_loader)
803         avg_g_loss = epoch_g_loss / len(train_loader)
804
805         real_prob, fake_prob, balance_score = evaluate_gan_balance(gan, val_loader, device)
806
807         print(f"Epoch [{epoch+1}/{epochs}] | D Loss: {avg_d_loss:.4f} | G Loss:
808         ↳ {avg_g_loss:.4f}")
809
810         print(f"  -> Val D(Real): {real_prob:.4f} | Val D(Fake): {fake_prob:.4f} | Balance Score:
811         ↳ {balance_score:.4f}")
812
813         checkpoint_data = {
814             'epoch': epoch + 1,
815             'generator_state_dict': gan.generator.state_dict(),
816             'discriminator_state_dict': gan.discriminator.state_dict(),
817             'd_loss': avg_d_loss,
818             'g_loss': avg_g_loss,
```



```
۸۱۷         'balance_score': balance_score
۸۱۸     }
۸۱۹
۸۲۰     # Save EVERY epoch
۸۲۱     epoch_path = os.path.join(save_dir, f"checkpoint_epoch_{epoch+1}.pth")
۸۲۲     torch.save(checkpoint_data, epoch_path)
۸۲۳
۸۲۴     # Save LAST model (overwrites previous)
۸۲۵     last_path = os.path.join(save_dir, "last_model.pth")
۸۲۶     torch.save(checkpoint_data, last_path)
۸۲۷
۸۲۸     # Save BEST model
۸۲۹     if balance_score < best_balance_score:
۸۳۰         best_balance_score = balance_score
۸۳۱         best_path = os.path.join(save_dir, "best_model.pth")
۸۳۲         torch.save(checkpoint_data, best_path)
۸۳۳         print(f"    New Best Model Saved! (Score: {best_balance_score:4f.})\n")
۸۳۴     else:
۸۳۵         print("")
۸۳۶
۸۳۷
۸۳۸     epochs = 500
۸۳۹     device = "cuda" if torch.cuda.is_available() else "cpu"
۸۴۰     train_cDCGAN(gan, train_loader, val_loader, epochs, device)
۸۴۱
۸۴۲     import os
۸۴۳     import torch
۸۴۴     import torchvision.utils as vutils
۸۴۵     import matplotlib.pyplot as plt
۸۴۶     import numpy as np
۸۴۷
۸۴۸     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
۸۴۹
۸۵۰     eval_generator = ConditionalGenerator(
۸۵۱         num_classes=2,
۸۵۲         latent_dim=100
۸۵۳     ).to(device)
۸۵۴
۸۵۵     checkpoint_path = 'checkpoints/last_model.pth'
۸۵۶     checkpoint = torch.load(checkpoint_path, map_location=device)
```



```
۸۵۷
۸۵۸ eval_generator.load_state_dict(checkpoint['generator_state_dict'])
۸۵۹ eval_generator.eval()
۸۶۰
۸۶۱ print(f"Successfully loaded weights from '{checkpoint_path}' (Epoch {checkpoint.get('epoch',
→ 'N/A')})!")
۸۶۲
۸۶۳
۸۶۴
۸۶۵ NUM_SAMPLES_PER_CLASS = 2000
۸۶۶
۸۶۷ def generate_class_samples(generator, num_samples, class_label, latent_dim, device):
۸۶۸     """Generates synthetic samples for a specific class label."""
۸۶۹     with torch.no_grad():
۸۷۰
۸۷۱         z = torch.randn(num_samples, latent_dim, dtype=torch.float32, device=device)
۸۷۲         labels = torch.full((num_samples,), class_label, dtype=torch.long, device=device)
۸۷۳         generated_imgs = generator(z, labels)
۸۷۴     return generated_imgs
۸۷۵
۸۷۶ print(f"\nGenerating {NUM_SAMPLES_PER_CLASS} samples for Class 0 (Benign)...")
۸۷۷ fake_class_0 = generate_class_samples(eval_generator, NUM_SAMPLES_PER_CLASS, 0, 100, device)
۸۷۸
۸۷۹ print(f"Generating {NUM_SAMPLES_PER_CLASS} samples for Class 1 (Malignant)...")
۸۸۰ fake_class_1 = generate_class_samples(eval_generator, NUM_SAMPLES_PER_CLASS, 1, 100, device)
۸۸۱
۸۸۲
۸۸۳ def plot_random_images(fake_0, fake_1, model_name):
۸۸۴
۸۸۵     idx_0 = torch.randperm(fake_0.size(0))[:20]
۸۸۶     idx_1 = torch.randperm(fake_1.size(0))[:20]
۸۸۷
۸۸۸     sample_0 = fake_0[idx_0].cpu()
۸۸۹     sample_1 = fake_1[idx_1].cpu()
۸۹۰
۸۹۱     # Un-normalize from [-1, 1] to [0, 1] for matplotlib
۸۹۲     sample_0 = (sample_0 + 1) / 0.2
۸۹۳     sample_1 = (sample_1 + 1) / 0.2
۸۹۴
۸۹۵
```



```
۸۹۶ grid_0 = vutils.make_grid(sample_0, nrow=10, padding=2, normalize=False)
۸۹۷ grid_1 = vutils.make_grid(sample_1, nrow=10, padding=2, normalize=False)
۸۹۸
۸۹۹ fig, axes = plt.subplots(2, 1, figsize=(16, 4))
۹۰۰
۹۰۱ axes[0].imshow(np.transpose(grid_0.numpy(), (1, 2, 0)), cmap='gray')
۹۰۲ axes[0].set_title(f"Generated Class 0 (Benign) - {model_name}", fontsize=14,
↳ fontweight='bold')
۹۰۳ axes[0].axis('off')
۹۰۴
۹۰۵ axes[1].imshow(np.transpose(grid_1.numpy(), (1, 2, 0)), cmap='gray')
۹۰۶ axes[1].set_title(f"Generated Class 1 (Malignant) - {model_name}", fontsize=14,
↳ fontweight='bold')
۹۰۷ axes[1].axis('off')
۹۰۸
۹۰۹ plt.tight_layout()
۹۱۰ plt.show()
۹۱۱
۹۱۲ plot_random_images(fake_class_0, fake_class_1, os.path.basename(checkpoint_path))
۹۱۳
۹۱۴
۹۱۵ def extract_and_plot_losses(checkpoints_dir="checkpoints", num_epochs=300):
۹۱۶     """
۹۱۷     Automatically reads the saved epoch checkpoints to extract and plot loss history.
۹۱۸     """
۹۱۹     d_losses = []
۹۲۰     g_losses = []
۹۲۱
۹۲۲     print("\nExtracting loss history from checkpoints...")
۹۲۳     for epoch in range(1, num_epochs + 1):
۹۲۴         epoch_path = os.path.join(checkpoints_dir, f"checkpoint_epoch_{epoch}.pth")
۹۲۵         if os.path.exists(epoch_path):
۹۲۶             chkpt = torch.load(epoch_path, map_location='cpu')
۹۲۷             d_losses.append(chkpt['d_loss'])
۹۲۸             g_losses.append(chkpt['g_loss'])
۹۲۹         else:
۹۳۰             break # Stop if a checkpoint is missing
۹۳۱
۹۳۲     if not d_losses:
۹۳۳         print("No checkpoint files found to plot losses.")
```

```
934         return
935
936     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
937
938     axes[0].plot(d_losses, color='red', linewidth=2, label='D Loss')
939     axes[0].set_title("Discriminator Loss", fontsize=14, fontweight='bold')
940     axes[0].set_xlabel("Epoch")
941     axes[0].set_ylabel("BCE Loss")
942     axes[0].grid(True, linestyle='--', alpha=6.0)
943     axes[0].legend()
944
945     axes[1].plot(g_losses, color='green', linewidth=2, label='G Loss')
946     axes[1].set_title("Generator Loss", fontsize=14, fontweight='bold')
947     axes[1].set_xlabel("Epoch")
948     axes[1].set_ylabel("BCE Loss")
949     axes[1].grid(True, linestyle='--', alpha=6.0)
950     axes[1].legend()
951
952     axes[2].plot(d_losses, color='red', linewidth=2, label='D Loss', alpha=8.0)
953     axes[2].plot(g_losses, color='green', linewidth=2, label='G Loss', alpha=8.0)
954     axes[2].set_title("Combined Dis/Gen Loss", fontsize=14, fontweight='bold')
955     axes[2].set_xlabel("Epoch")
956     axes[2].set_ylabel("BCE Loss")
957     axes[2].grid(True, linestyle='--', alpha=6.0)
958     axes[2].legend()
959
960     plt.tight_layout()
961     plt.show()
962
963     extract_and_plot_losses("checkpoints", epochs)
964
965     import os
966     import torch
967     import torch.nn.functional as F
968     import torchvision.transforms as transforms
969     import torchvision.datasets as datasets
970     from torch.utils.data import ConcatDataset, Subset, DataLoader
971     import numpy as np
972
973     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```



```
۹۷۴
۹۷۵ NUM_FAKE_TO_GENERATE = 2000
۹۷۶ with torch.no_grad():
۹۷۷     z_0 = torch.randn(NUM_FAKE_TO_GENERATE, 100, dtype=torch.float32, device=device)
۹۷۸     labels_0 = torch.full((NUM_FAKE_TO_GENERATE,), 0, dtype=torch.long, device=device)
۹۷۹     fake_class_0 = eval_generator(z_0, labels_0)
۹۸۰
۹۸۱     z_1 = torch.randn(NUM_FAKE_TO_GENERATE, 100, dtype=torch.float32, device=device)
۹۸۲     labels_1 = torch.full((NUM_FAKE_TO_GENERATE,), 1, dtype=torch.long, device=device)
۹۸۳     fake_class_1 = eval_generator(z_1, labels_1)
۹۸۴
۹۸۵ resnet_transform = transforms.Compose([
۹۸۶     transforms.Grayscale(num_output_channels=3),
۹۸۷     transforms.Resize((224, 224)),
۹۸۸     transforms.ToTensor(),
۹۸۹     transforms.Normalize(mean=[485.0, 456.0, 406.0], std=[229.0, 224.0, 225.0])
۹۹۰ ])
۹۹۱
۹۹۲ base_dir = "breast_mnist_images"
۹۹۳ train_dataset = datasets.ImageFolder(os.path.join(base_dir, 'train'), transform=resnet_transform)
۹۹۴
۹۹۵ def prepare_gan_data_for_resnet(fake_tensor, label_value):
۹۹۶
۹۹۷     tensor_01 = (fake_tensor.detach().cpu() + 0.1) / 0.2
۹۹۸     tensor_resized = F.interpolate(tensor_01, size=(224, 224), mode='bilinear',
۹۹۹     ↪ align_corners=False)
۱۰۰۰     tensor_rgb = tensor_resized.repeat(1, 3, 1, 1)
۱۰۰۱
۱۰۰۲     mean = torch.tensor([485.0, 456.0, 406.0]).view(1, 3, 1, 1)
۱۰۰۳     std = torch.tensor([229.0, 224.0, 225.0]).view(1, 3, 1, 1)
۱۰۰۴     tensor_normalized = (tensor_rgb - mean) / std
۱۰۰۵
۱۰۰۶     dataset_list = [(tensor_normalized[i], int(label_value)) for i in
۱۰۰۷     ↪ range(len(tensor_normalized))]
۱۰۰۸     return dataset_list
۱۰۰۹
۱۰۱۰ fake_dataset_0 = prepare_gan_data_for_resnet(fake_class_0, 0)
fake_dataset_1 = prepare_gan_data_for_resnet(fake_class_1, 1)
```



```

1011 def create_prioritized_balanced_dataset(real_dataset, fake_ds_0, fake_ds_1,
1012 ↪ target_per_class=500):
1013
1014     real_idx_0, real_idx_1 = [], []
1015
1016     for i in range(len(real_dataset)):
1017         label = real_dataset[i][1]
1018         label_val = label.item() if isinstance(label, torch.Tensor) else (label[0] if
1019 ↪ isinstance(label, (np.ndarray, list)) else label)
1020
1021         if label_val == 0:
1022             real_idx_0.append(i)
1023         elif label_val == 1:
1024             real_idx_1.append(i)
1025
1026     real_idx_0 = real_idx_0[:target_per_class]
1027     real_idx_1 = real_idx_1[:target_per_class]
1028
1029     real_subset_0 = Subset(real_dataset, real_idx_0)
1030     real_subset_1 = Subset(real_dataset, real_idx_1)
1031
1032     needed_0 = max(0, target_per_class - len(real_subset_0))
1033     needed_1 = max(0, target_per_class - len(real_subset_1))
1034
1035     print("-" * 50)
1036     print(f"Class 0 (Benign): Found {len(real_subset_0)} Real images -> Adding {needed_0} Fake
1037 ↪ images.")
1038     print(f"Class 1 (Malignant): Found {len(real_subset_1)} Real images -> Adding {needed_1} Fake
1039 ↪ images.")
1040     print("-" * 50)
1041
1042     fake_idx_0 = list(range(len(fake_ds_0)))
1043     fake_idx_1 = list(range(len(fake_ds_1)))
1044
1045     np.random.shuffle(fake_idx_0)
1046     np.random.shuffle(fake_idx_1)
1047
1048     selected_fake_0 = Subset(fake_ds_0, fake_idx_0[:needed_0])
1049     selected_fake_1 = Subset(fake_ds_1, fake_idx_1[:needed_1])
1050

```




```
1047     final_dataset = ConcatDataset([real_subset_0, real_subset_1, selected_fake_0,
1048                                     ↳ selected_fake_1])
1049
1050     return final_dataset
1051
1052 balanced_train_dataset = create_prioritized_balanced_dataset(
1053     train_dataset,
1054     fake_dataset_0,
1055     fake_dataset_1,
1056     target_per_class=600
1057 )
1058
1059 BATCH_SIZE = 64
1060
1061 final_train_loader = DataLoader(balanced_train_dataset, batch_size=BATCH_SIZE, shuffle=True)
1062
1063 print(f"Train Loader Ready! Total balanced samples: {len(balanced_train_dataset)}")
1064
1065 import matplotlib.pyplot as plt
1066 import numpy as np
1067 import pandas as pd
1068 import torch
1069
1070 def get_label_counts(dataset):
1071     labels = []
1072     for _, label in dataset:
1073         if isinstance(label, torch.Tensor):
1074             labels.append(label.item())
1075         elif isinstance(label, list) or hasattr(label, '__iter__'):
1076             labels.append(label[0])
1077         else:
1078             labels.append(label)
1079     return pd.Series(labels).value_counts()
1080
1081 val_counts = get_label_counts(val_dataset)
1082 test_counts = get_label_counts(test_dataset)
1083
1084 train_real_benign = len(balanced_train_dataset.datasets[0])
1085 train_real_malignant = len(balanced_train_dataset.datasets[1])
1086 train_gan_benign = len(balanced_train_dataset.datasets[2])
1087 train_gan_malignant = len(balanced_train_dataset.datasets[3])
```

```

۱۰۸۶
۱۰۸۷ splits = ['Train', 'Validation', 'Test']
۱۰۸۸
۱۰۸۹
۱۰۹۰ benign_real = [train_real_benign, val_counts.get(0, 0), test_counts.get(0, 0)]
۱۰۹۱ benign_gan = [train_gan_benign, 0, 0]
۱۰۹۲
۱۰۹۳ malignant_real = [train_real_malignant, val_counts.get(1, 0), test_counts.get(1, 0)]
۱۰۹۴ malignant_gan = [train_gan_malignant, 0, 0]
۱۰۹۵
۱۰۹۶ x = np.arange(len(splits))
۱۰۹۷ width = 35.0
۱۰۹۸
۱۰۹۹ fig, ax = plt.subplots(figsize=(12, 7))
۱۱۰۰
۱۱۰۱ color_benign_real = '#5cb85c'
۱۱۰۲ color_benign_gan = '#a3e4a3'
۱۱۰۳ color_malignant_real = '#d9534f'
۱۱۰۴ color_malignant_gan = '#f2a6a4'
۱۱۰۵
۱۱۰۶ bar1 = ax.bar(x - width/2, benign_real, width, label='Benign (Real)', color=color_benign_real,
    ↳ edgecolor='white')
۱۱۰۷ bar2 = ax.bar(x - width/2, benign_gan, width, bottom=benign_real, label='Benign (GAN)',
    ↳ color=color_benign_gan, edgecolor='white', hatch='//')
۱۱۰۸
۱۱۰۹ bar3 = ax.bar(x + width/2, malignant_real, width, label='Malignant (Real)',
    ↳ color=color_malignant_real, edgecolor='white')
۱۱۱۰ bar4 = ax.bar(x + width/2, malignant_gan, width, bottom=malignant_real, label='Malignant (GAN)',
    ↳ color=color_malignant_gan, edgecolor='white', hatch='//')
۱۱۱۱
۱۱۱۲ ax.set_ylabel('Number of Images', fontsize=12, fontweight='bold')
۱۱۱۳ ax.set_title('Class Distribution Across BreastMNIST Splits (Real vs GAN)', fontsize=14,
    ↳ fontweight='bold', pad=20)
۱۱۱۴ ax.set_xticks(x)
۱۱۱۵ ax.set_xticklabels(splits, fontsize=12)
۱۱۱۶ ax.legend(fontsize=11, loc='upper right', bbox_to_anchor=(15.1, 1))
۱۱۱۷ ax.grid(axis='y', linestyle='--', alpha=7.0)
۱۱۱۸
۱۱۱۹ for container in [bar1, bar2, bar3, bar4]:
۱۱۲۰     for p in container:
    
```



```
1121     height = p.get_height()
1122     if height > 0:
1123
1124         ax.annotate(f"{int(height)}",
1125                     (p.get_x() + p.get_width() / 2., p.get_y() + height / 2.),
1126                     ha='center', va='center',
1127                     fontsize=11, fontweight='bold', color='black')
1128
1129 plt.tight_layout(rect=[0, 0, 85.0, 1])
1130 plt.show()
1131
1132 val_dataset = datasets.ImageFolder(os.path.join(base_dir, 'val'), transform=resnet_transform)
1133 test_dataset = datasets.ImageFolder(os.path.join(base_dir, 'test'), transform=resnet_transform)
1134 final_val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
1135 final_test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
1136
1137 import matplotlib.pyplot as plt
1138 import numpy as np
1139 import torch
1140
1141 def show_10_random_train_samples(dataset):
1142     random_indices = np.random.choice(len(dataset), 10, replace=False)
1143     fig, axes = plt.subplots(2, 5, figsize=(15, 6))
1144     axes = axes.flatten()
1145     mean = np.array([485.0, 456.0, 406.0])
1146     std = np.array([229.0, 224.0, 225.0])
1147
1148     for i, idx in enumerate(random_indices):
1149         image_tensor, label = dataset[idx]
1150
1151         if isinstance(label, torch.Tensor):
1152             label_val = label.item()
1153         elif isinstance(label, (np.ndarray, list)):
1154             label_val = label[0]
1155         else:
1156             label_val = label
1157
1158         image_np = image_tensor.numpy().transpose((1, 2, 0))
1159
1160         image_unnorm = (std * image_np) + mean
```



```
1161
1162     image_unnorm = np.clip(image_unnorm, 0, 1)
1163
1164     axes[i].imshow(image_unnorm)
1165
1166     class_name = "Malignant (1)" if label_val == 1 else "Benign (0)"
1167     axes[i].set_title(f"{class_name}\nIdx: {idx}", fontsize=12, fontweight='bold')
1168     axes[i].axis('off')
1169
1170     plt.tight_layout()
1171     plt.show()
1172
1173 show_10_random_train_samples(balanced_train_dataset)
1174
1175 import torch.nn as nn
1176 import torch.optim as optim
1177 from torchvision import models
1178
1179 model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
1180 num_fts = model.fc.in_features
1181 model.fc = nn.Linear(num_fts, 2)
1182 model = model.to(device)
1183
1184 criterion = nn.CrossEntropyLoss(label_smoothing=1.0)
1185 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
1186
1187 epochs = 10
1188 train_acc_history, val_acc_history = [], []
1189 train_loss_history, val_loss_history = [], []
1190 best_val_loss = float('inf')
1191
1192 print(f"\nStarting ResNet Training for {epochs} Epochs on {device}...")
1193
1194 for epoch in range(epochs):
1195
1196     model.train()
1197     running_corrects = 0
1198     running_loss = 0.0
1199     total_train = 0
1200
```



```
۱۲۰۱     for inputs, targets in final_train_loader:
۱۲۰۲         inputs, targets = inputs.to(device), targets.to(device)
۱۲۰۳
۱۲۰۴         optimizer.zero_grad()
۱۲۰۵         outputs = model(inputs)
۱۲۰۶         loss = criterion(outputs, targets)
۱۲۰۷         loss.backward()
۱۲۰۸         optimizer.step()
۱۲۰۹
۱۲۱۰         _, preds = torch.max(outputs, 1)
۱۲۱۱         running_corrects += torch.sum(preds == targets)
۱۲۱۲         running_loss += loss.item() * inputs.size(0)
۱۲۱۳         total_train += targets.size(0)
۱۲۱۴
۱۲۱۵     train_acc = running_corrects.double() / total_train
۱۲۱۶     train_loss = running_loss / total_train
۱۲۱۷
۱۲۱۸     train_acc_history.append(train_acc.item())
۱۲۱۹     train_loss_history.append(train_loss)
۱۲۲۰
۱۲۲۱     model.eval()
۱۲۲۲     val_corrects = 0
۱۲۲۳     val_loss = 0.0
۱۲۲۴     total_val = 0
۱۲۲۵
۱۲۲۶     with torch.no_grad():
۱۲۲۷         for inputs, targets in final_val_loader:
۱۲۲۸             inputs, targets = inputs.to(device), targets.to(device)
۱۲۲۹
۱۲۳۰             outputs = model(inputs)
۱۲۳۱             loss = criterion(outputs, targets)
۱۲۳۲
۱۲۳۳             _, preds = torch.max(outputs, 1)
۱۲۳۴             val_corrects += torch.sum(preds == targets)
۱۲۳۵             val_loss += loss.item() * inputs.size(0)
۱۲۳۶             total_val += targets.size(0)
۱۲۳۷
۱۲۳۸     val_acc = val_corrects.double() / total_val
۱۲۳۹     epoch_val_loss = val_loss / total_val
۱۲۴۰
```

```
۱۲۴۱ val_acc_history.append(val_acc.item())
۱۲۴۲ val_loss_history.append(epoch_val_loss)
۱۲۴۳
۱۲۴۴ print(f'Epoch [{epoch+1}/{epochs}] | Train Loss: {train_loss:.6f} | Train Acc:
    ↳ {train_acc:.6f} | Val Loss: {epoch_val_loss:.6f} | Val Acc: {val_acc:.6f}')
۱۲۴۵
۱۲۴۶ if epoch_val_loss < best_val_loss:
۱۲۴۷     best_val_loss = epoch_val_loss
۱۲۴۸     torch.save(model.state_dict(), 'best_breast_mnist_model_resnet.pth')
۱۲۴۹     print(" Best ResNet model saved!")
۱۲۵۰
۱۲۵۱ from sklearn.metrics import classification_report
۱۲۵۲ import torch
۱۲۵۳
۱۲۵۴ def get_clinical_report(model, loader, device):
۱۲۵۵     model.eval()
۱۲۵۶     all_preds = []
۱۲۵۷     all_targets = []
۱۲۵۸
۱۲۵۹     with torch.no_grad():
۱۲۶۰         for inputs, targets in loader:
۱۲۶۱             inputs = inputs.to(device)
۱۲۶۲             outputs = model(inputs)
۱۲۶۳             _, preds = torch.max(outputs, 1)
۱۲۶۴
۱۲۶۵             all_preds.extend(preds.cpu().numpy())
۱۲۶۶             all_targets.extend(targets.numpy())
۱۲۶۷
۱۲۶۸
۱۲۶۹     print(classification_report(all_targets, all_preds, target_names=['Benign (0)', 'Malignant
    ↳ (1)']))
۱۲۷۰
۱۲۷۱
۱۲۷۲ print("Clinical Evaluation Report:")
۱۲۷۳
۱۲۷۴ get_clinical_report(model, final_test_loader, device)
۱۲۷۵
```



۲ پرسش ۲: شبکه های رمزگذار - رمزگشا مولد

۱.۲ مجموعه دادگان

در این بخش با توجه به اینکه مجموع رقم آخر شماره دانشجویی اعضای گروه برابر با ۹ است، مجموعه دادگان Cifar10 را دانلود کردیم. در کد زیر این بخش نشان داده شده است:

```
1 from keras.datasets import cifar10
2 (x_train , y_train), (x_test , y_test) = cifar10.load_data()
3
```

۱.۱.۲ معرفی مجموعه داده CIFAR-۱۰

مجموعه داده CIFAR-۱۰ یکی از معروف ترین و پرکاربردترین مجموعه های داده در حوزه بینایی ماشین و یادگیری عمیق برای آموزش و ارزیابی الگوریتم های طبقه بندی تصویر است. این مجموعه داده شامل ۶۰,۰۰۰ تصویر رنگی با وضوح پایین است که در ۱۰ دسته بندی کاملاً مجزا قرار دارند.

ویژگی های کلیدی:

- ابعاد و حجم: تصاویر دارای ابعاد 32×32 پیکسل و در ۳ کانال رنگی (RGB) هستند.
- توزیع داده ها: شامل ۵۰,۰۰۰ تصویر برای آموزش (Train) و ۱۰,۰۰۰ تصویر برای آزمایش (Test) است.
- کلاس ها: ۱۰ کلاس موجود شامل هواپیما، خودرو، پرنده، گربه، گوزن، سگ، قورباغه، اسب، کشتی و کامیون می باشد. در هر کلاس دقیقاً ۶,۰۰۰ تصویر وجود دارد.

۲.۲ انجام PCA و ISOMAP

۱.۲.۲ الگوریتم PCA (Principal Component Analysis)

شیوه عملکرد PCA یک روش خطی برای کاهش ابعاد است که تلاش می کند داده ها را به فضایی با ابعاد کمتر منتقل کند به گونه ای که بیشترین میزان واریانس حفظ شود. مراحل اصلی عبارتند از:

۱. استانداردسازی داده ها ۲. محاسبه ماتریس کوواریانس ۳. محاسبه مقادیر ویژه و بردارهای ویژه ۴. انتخاب مؤلفه های اساسی و تصویربرداری داده ها روی آن ها

مزایا

- سادگی و سرعت بالا
- کاهش نویز
- عدم بروز بیش برآزش (Overfitting)

معایب

- محدودیت در نمایش ساختارهای غیرخطی
- حساسیت به نقاط پرت
- دشوار شدن تفسیر مؤلفه‌ها

۲.۲.۲ الگوریتم ISOMAP (Isometric Mapping)

شیوه عملکرد ISOMAP یک روش غیرخطی مبتنی بر خمینه‌ها است و به جای فاصله اقلیدسی، فاصله ژئودزیک را حفظ می‌کند.

مراحل اجرای الگوریتم ISOMAP

۱. تشکیل گراف همسایگی در این مرحله ساختار محلی داده‌ها مدل می‌شود. فرض اصلی ISOMAP این است که داده‌های با بعد بالا روی یک خمینه (Manifold) با بعد کمتر قرار دارند. برای تقریب این خمینه، یک گراف همسایگی ساخته می‌شود که در آن هر نقطه داده یک رأس (Node) در گراف است. برای تعیین همسایه‌ها معمولاً از دو روش استفاده می‌شود:

- K-Nearest Neighbors (KNN): برای هر نقطه، k تا از نزدیک‌ترین نقاط بر اساس فاصله اقلیدسی انتخاب می‌شوند و بین آن‌ها یال برقرار می‌شود.
- ϵ -Neighborhood: تمام نقاطی که فاصله اقلیدسی آن‌ها از یک نقطه کمتر از مقدار آستانه ϵ باشد، به عنوان همسایه در نظر گرفته می‌شوند.

وزن هر یال در گراف برابر با فاصله اقلیدسی بین دو نقطه است:

$$d_{ij} = \|x_i - x_j\|$$

در نتیجه یک گراف وزن‌دار ساخته می‌شود که تنها ارتباطات محلی داده‌ها را حفظ می‌کند.

۲. محاسبه فواصل ژئودزیک پس از ساخت گراف همسایگی، هدف تخمین فاصله واقعی بین نقاط روی خمینه است. در فضای غیرخطی، فاصله مستقیم اقلیدسی بین دو نقطه دور ممکن است مسیر واقعی روی سطح خمیده را نشان ندهد. به همین دلیل ISOMAP از فاصله ژئودزیک استفاده می‌کند. فاصله ژئودزیک بین دو نقطه به صورت کوتاه‌ترین مسیر روی گراف تعریف می‌شود. برای محاسبه این فاصله‌ها از الگوریتم‌های کوتاه‌ترین مسیر استفاده می‌شود:

- الگوریتم Dijkstra برای محاسبه کوتاه‌ترین مسیر از یک گره به تمام گره‌های دیگر
- الگوریتم Floyd-Warshall برای محاسبه کوتاه‌ترین مسیر بین تمام جفت گره‌ها

خروجی این مرحله یک ماتریس فاصله ژئودزیک است:

$$D_G = \begin{bmatrix} 0 & d_G(x_1, x_2) & \dots & d_G(x_1, x_n) \\ d_G(x_2, x_1) & 0 & \dots & d_G(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ d_G(x_n, x_1) & d_G(x_n, x_2) & \dots & 0 \end{bmatrix}$$

که در آن $d_G(x_i, x_j)$ فاصله ژئودزیک بین نقاط x_i و x_j است.

۳. اعمال الگوریتم **MDS** در مرحله نهایی، هدف پیدا کردن نمایش داده‌ها در بعد پایین است به طوری که فواصل ژئودزیک حفظ شوند. برای این کار از **Multidimensional Scaling (MDS)** کلاسیک استفاده می‌شود. ابتدا ماتریس فاصله ژئودزیک به یک ماتریس شباهت تبدیل می‌شود:

$$B = -\frac{1}{2}HD_G^2H$$

که در آن:

$$H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$$

ماتریس مرکزسازی (centering matrix) است. در رابطه بالا منظور از ۱ همان بردار $\mathbf{1}$ در n است که شامل اعداد یک است. نتیجه ضرب این بردار در ترانزپوز آن، یک ماتریس n در n می‌شود که تمام درایه‌های آن برابر با عدد یک است. سپس با تجزیه مقادیر ویژه:

$$B = V\Lambda V^T$$

مختصات داده‌ها در فضایی با بعد پایین d به صورت زیر به دست می‌آید:

$$Y = V_d\Lambda_d^{1/2}$$

در این رابطه V_d شامل d بردار ویژه اول و Λ_d شامل بزرگ‌ترین مقادیر ویژه است. ماتریس Y داده‌ها را در بعد پایین نشان می‌دهد.

مزایا

- توانایی کشف ساختارهای غیرخطی
- حفظ هندسه سراسری
- راه‌حل بهینه و بدون مینیمم محلی

معایب

- هزینه محاسباتی بسیار بالا ($O(N^3)$)
- حساسیت به نویز و ایجاد مسیرهای کوتاه اشتباه

۳.۲.۲ ارزیابی الگوریتم‌های کاهش ابعاد (ISOMAP و PCA) به همراه طبقه‌بند KNN

آماده‌سازی داده‌ها و انتخاب زیرمجموعه در این پیاده‌سازی، تصاویر مجموعه داده CIFAR-10 که دارای ابعاد $32 \times 32 \times 3$ هستند، ابتدا به بردارهای یک بعدی با اندازه 3072 تغییر شکل (Reshape) یافتند. به دلیل هزینه محاسباتی بالای الگوریتم‌هایی نظیر ISOMAP از یک زیرمجموعه کوچک‌تر شامل 2000 داده آموزشی و 500 داده آزمایشی استفاده شد. در نهایت داده‌ها با استفاده از StandardScaler استانداردسازی شدند تا میانگین صفر و واریانس یک داشته باشند.

روند آزمایش برای طبقه‌بندی، ابتدا داده‌های خام استاندارد شده توسط دو الگوریتم PCA (با 100 مؤلفه) و ISOMAP (با 50 مؤلفه و 15 همسایه) کاهش بعد داده شدند. سپس خروجی این دو الگوریتم به عنوان ورودی به طبقه‌بند K-Nearest Neighbors (KNN) با $K = 7$ داده شد.

دلیل استفاده از فاصله اقلیدسی در KNN در طبقه‌بند KNN از معیار فاصله اقلیدسی (Euclidean Distance) استفاده شده است. منطق استفاده از این فاصله در این مسئله آن است که باعث می‌شود تصاویری که پیکسل‌های مشابهی از نظر شدت روشنایی (Brightness) و رنگ در موقعیت‌های مکانی یکسان دارند، در فضای ویژگی فاصله کمتری با یکدیگر داشته باشند. کد زیر در این بخش زده شده است:

```

1  import numpy as np
2  from keras.datasets import cifar10
3  from sklearn.decomposition import PCA
4  from sklearn.manifold import Isomap
5  from sklearn.neighbors import KNeighborsClassifier
6  from sklearn.model_selection import RandomizedSearchCV
7  from sklearn.pipeline import Pipeline
8  from sklearn.preprocessing import StandardScaler
9  from sklearn.metrics import accuracy_score
10
11 print(f"the shape of the x_train is {x_train.shape}")
12 print(f"the shape of the x_test is {x_test.shape}")
13
14 x_train_R = x_train.reshape(x_train.shape[0], -1)
15 x_test_R = x_test.reshape(x_test.shape[0], -1)
16 print("-----")
17 print("After reshaping the data to Vector:")
18 print(f"the shape of the x_train is {x_train_R.shape}")
19 print(f"the shape of the x_test is {x_test_R.shape}")
20
21
    
```



```
22     scaler = StandardScaler()
23     x_train_scaled = scaler.fit_transform(x_train_R)
24     x_test_scaled = scaler.transform(x_test_R)
25
26
27     n_train_samples = 2000
28     n_test_samples = 500
29
30     x_train_sub = x_train_scaled[:n_train_samples]
31     y_train_sub = y_train[:n_train_samples].ravel()
32
33     x_test_sub = x_test_scaled[:n_test_samples]
34     y_test_sub = y_test[:n_test_samples].ravel()
35     print("-----")
36     print("Data ready")
37     class_names = [
38         'airplane',
39         'automobile',
40         'bird',
41         'cat',
42         'deer',
43         'dog',
44         'frog',
45         'horse',
46         'ship',
47         'truck'
48     ]
49     from sklearn.pipeline import Pipeline
50     from sklearn.decomposition import PCA
51     from sklearn.manifold import Isomap
52     from sklearn.neighbors import KNeighborsClassifier
53     from sklearn.model_selection import RandomizedSearchCV
54     from sklearn.metrics import accuracy_score
55     from sklearn.metrics import classification_report
56
57
58     print("--- Fixed Parameters (Subsets) ---")
59
60     # PCA + KNN
61     # Added random_state=42 to PCA for reproducible results
62     pca_fixed = PCA(n_components=100, random_state=42)
63     knn_pca_fixed = KNeighborsClassifier(n_neighbors=7, weights='uniform', metric='euclidean')
64
65     x_train_sub_pca = pca_fixed.fit_transform(x_train_sub)
66     x_test_sub_pca = pca_fixed.transform(x_test_sub)
```



```

67 knn_pca_fixed.fit(x_train_sub_pca, y_train_sub)
68 y_pred_pca = knn_pca_fixed.predict(x_test_sub_pca)
69 print("Fixed PCA Acc:", accuracy_score(y_test_sub, knn_pca_fixed.predict(x_test_sub_pca)))
70 print("PCA Classification Report:")
71 print(classification_report(y_test_sub, y_pred_pca, target_names=class_names))
72
73
74
75 # ISOMAP + KNN
76 iso_fixed = Isomap(n_components=50, n_neighbors=15)
77 knn_iso_fixed = KNeighborsClassifier(n_neighbors=7, weights='uniform')
78
79 x_train_sub_iso = iso_fixed.fit_transform(x_train_sub)
80 x_test_sub_iso = iso_fixed.transform(x_test_sub)
81
82 knn_iso_fixed.fit(x_train_sub_iso, y_train_sub)
83 y_pred_iso = knn_iso_fixed.predict(x_test_sub_iso)
84 print("Fixed ISOMAP Acc:", accuracy_score(y_test_sub, knn_iso_fixed.predict(x_test_sub_iso)))
85 print("ISOMAP Classification Report:")
86 print(classification_report(y_test_sub, y_pred_iso, target_names=class_names))
87

```

نتایج به دست آمده و گزارش دسته‌بندی پس از اجرای مدل‌ها بر روی 500 داده آزمایشی، دقت (Accuracy) مدل ترکیبی PCA برابر با 0.258 (حدود 26%) و مدل ISOMAP برابر با 0.22 (22%) به دست آمد. جزئیات گزارش دسته‌بندی برای هر دو روش در جداول زیر آورده شده است:

جدول ۵: گزارش دسته‌بندی مدل ترکیبی PCA + KNN (دقت: 0.26)

Class	Precision	Recall	F1-Score	Support
airplane	0.28	0.56	0.38	57
automobile	0.67	0.20	0.30	41
bird	0.21	0.45	0.29	51
cat	0.35	0.14	0.20	49
deer	0.09	0.20	0.12	40
dog	0.18	0.04	0.07	48
frog	0.24	0.31	0.27	54
horse	0.30	0.06	0.11	47
ship	0.42	0.42	0.42	57
truck	0.45	0.09	0.15	56
accuracy	0.26			500
macro avg	0.32	0.25	0.23	500
weighted avg	0.32	0.26	0.24	500



جدول ۶: گزارش دسته‌بندی مدل ترکیبی ISOMAP + KNN (دقت: 0.22)

Class	Precision	Recall	F1-Score	Support
airplane	0.25	0.42	0.31	57
automobile	0.38	0.27	0.31	41
bird	0.17	0.25	0.20	51
cat	0.17	0.14	0.16	49
deer	0.11	0.20	0.14	40
dog	0.27	0.06	0.10	48
frog	0.22	0.22	0.22	54
horse	0.17	0.09	0.11	47
ship	0.30	0.33	0.32	57
truck	0.28	0.16	0.20	56
accuracy	0.22			500
macro avg	0.32	0.25	0.23	500
weighted avg	0.32	0.26	0.24	500

تحلیل و مقایسه نتایج: دلیل دقت پایین مدل‌ها همان‌طور که در نتایج مشخص است، دقت مدل مبتنی بر PCA برابر با 26% و دقت ISOMAP برابر با 22% است. دلیل اصلی این دقت پایین آن است که ما در حال استفاده از 3072 ویژگی خام پیکسلی هستیم. الگوریتم‌هایی مانند KNN بر پایه محاسبه فاصله کار می‌کنند و زمانی که داده‌ها خام و بدون استخراج ویژگی‌های معنایی (Semantic) باشند، محاسبه فاصله اقلیدسی بین بردارهای پیکسلی کارایی ندارد.

چرا عملکرد ISOMAP ضعیف‌تر از PCA بود؟ با وجود اینکه ISOMAP یک الگوریتم قدرتمند غیرخطی است، در این آزمایش نتیجه ضعیف‌تری (22%) نسبت به روش خطی PCA (26%) ثبت کرده است. دلایل این امر عبارتند از:

- حساسیت به نویز تصاویر: داده‌های خام تصویری دارای نویز بسیار بالایی هستند. الگوریتم ISOMAP به شدت به نویز حساس است و وجود نویز باعث پدیده اتصالات کوتاه (Short-circuiting) در گراف همسایگی می‌شود که محاسبات فواصل ژئودزیک را دچار خطای شدید می‌کند.
- پیچیدگی منیفولد و کمبود داده: تصاویر طبیعی CIFAR-10 دارای ساختار منیفولد بسیار پیچیده‌ای هستند. الگوریتم ISOMAP با تعداد داده کم (2000 نمونه آموزشی) قادر به بازسازی هندسه سراسری این منیفولد نیست. در مقابل، PCA صرفاً به دنبال جهات با بیشترین واریانس است و در فضاهای نویزی با ویژگی‌های خام، رفتار باثبات‌تری از خود نشان می‌دهد.

۴.۲.۲ بهینه‌سازی مدل‌ها با استفاده از جستجوی تصادفی (Random Search) و Pipeline

روند پیاده‌سازی و استفاده از Pipeline برای یافتن بهترین پارامترهای مدل، از مفهوم Pipeline در کتابخانه Scikit-Learn استفاده شد. پایپ‌لاین به ما اجازه می‌دهد تا مراحل کاهش ابعاد (PCA یا ISOMAP) و طبقه‌بندی (KNN) را به صورت یک جریان واحد و پیوسته تعریف کنیم. این کار علاوه بر تمیزتر شدن کد، از نشت داده‌ها (Data Leakage) در فرآیند ارزیابی متقاطع جلوگیری می‌کند. سپس با استفاده از RandomizedSearchCV، فضای جستجویی از پارامترهای مختلف تعریف شد تا الگوریتم به صورت تصادفی ترکیبات مختلف را طی ارزیابی متقاطع ۳-لایه (3-Fold CV) بررسی کرده و بهترین پیکربندی را بیابد.



تحلیل بهترین پارامترهای یافت شده برای PCA بهترین پارامترها در جستجوی تصادفی برای مدل PCA به این صورت انتخاب شد: تعداد مؤلفه‌ها برابر با 50، تعداد همسایه‌ها $K = 15$ ، وزن‌دهی uniform و معیار فاصله euclidean.

- معیار فاصله اقلیدسی نسبت به منهن: انتخاب فاصله اقلیدسی (L_2) نسبت به منهن (L_1) نشان‌دهنده آن است که در فضای کاهش یافته PCA، فاصله مستقیم و هندسی بین مؤلفه‌ها اهمیت بیشتری در تفکیک کلاس‌ها دارد. بردارهای ویژگی مترکم، فاصله اقلیدسی معمولاً به دلیل حساسیت کمتر به چرخش محورها، رفتار پایدارتری در یافتن همسایه‌های واقعی از خود نشان می‌دهد.
- وزن‌دهی uniform: این پارامتر نشان می‌دهد که در فضای خطی PCA، رأی تمامی 15 همسایه نزدیک ارزش یکسانی دارد و نیازی به جریمه کردن همسایه‌های دورتر نبوده است.

تحلیل بهترین پارامترهای یافت شده برای ISOMAP برای مدل ISOMAP، جستجو به این پارامترها رسید: تعداد مؤلفه‌ها 50، تعداد همسایه‌های گراف ISOMAP برابر با 10، تعداد همسایه‌های KNN برابر با 10 ($K = 10$) و وزن‌دهی distance.

- وزن‌دهی distance: برعکس PCA، در اینجا مدل به وزن‌دهی مبتنی بر فاصله روی آورده است. دلیل این امر طبیعت غیرخطی خمینه‌های ISOMAP است. در فضاهای غیرخطی و پیچیده، همسایه‌هایی که از نظر هندسی به نمونه آزمون (تصویر ورودی) نزدیک‌ترند، قابلیت اعتماد بسیار بالاتری نسبت به همسایه‌های دورتر در همان گروه K -تایی دارند؛ لذا دادن وزن بیشتر به آن‌ها (رابطه معکوس با فاصله) دقت را بهبود بخشیده است.

جداول نتایج به‌دست آمده پس از بهینه‌سازی در ادامه، گزارش دسته‌بندی هر دو روش پس از اعمال بهترین پارامترهای یافت شده توسط جستجوی تصادفی آورده شده است:

جدول ۷: گزارش دسته‌بندی مدل ترکیبی PCA + KNN بهینه‌شده با Random Search (دقت: 0.27)

Class	Precision	Recall	F1-Score	Support
airplane	0.26	0.42	0.32	57
automobile	0.62	0.12	0.20	41
bird	0.22	0.37	0.27	51
cat	0.30	0.06	0.10	49
deer	0.13	0.28	0.18	40
dog	0.43	0.06	0.11	48
frog	0.24	0.48	0.32	54
horse	0.30	0.06	0.11	47
ship	0.41	0.53	0.46	57
truck	0.55	0.20	0.29	56
accuracy	0.27			500
macro avg	0.35	0.26	0.24	500
weighted avg	0.35	0.27	0.25	500

جدول ۸: گزارش دسته‌بندی مدل ترکیبی ISOMAP + KNN بهینه‌شده با Random Search (دقت: 0.274)

Class	Precision	Recall	F1-Score	Support
airplane	0.32	0.33	0.33	57
automobile	0.42	0.34	0.38	41
bird	0.18	0.31	0.23	51
cat	0.26	0.12	0.17	49
deer	0.14	0.17	0.15	40
dog	0.21	0.15	0.17	48
frog	0.27	0.31	0.29	54
horse	0.26	0.17	0.21	47
ship	0.36	0.47	0.41	57
truck	0.37	0.29	0.32	56
accuracy	0.274			500
macro avg	0.28	0.27	0.27	500
weighted avg	0.28	0.27	0.27	500

مقایسه نتایج قبل و بعد از بهینه‌سازی با مقایسه نتایج مدل‌های اولیه با مدل‌های بهینه‌شده توسط Random Search، شاهد افزایش دقت در هر دو روش هستیم:

- مدل PCA: دقت از 26% به 27% افزایش یافت. این تغییر جزئی نشان می‌دهد که PCA تا حد زیادی در همان مدل پایه نیز توانسته بود واریانس اصلی داده‌ها را استخراج کند.

- مدل ISOMAP: دقت این مدل پرش قابل توجهی داشته و از 22% به 27.4% رسیده است. این موضوع نشان می‌دهد که ISOMAP به شدت به انتخاب صحیح پارامترها (مانند تعداد همسایه‌های گراف برای ساخت منیفلد و وزن‌دهی مبتنی بر فاصله در KNN) حساس است. با یافتن این تنظیمات بهینه، ISOMAP توانست از یک روش خطی مانند PCA پیشی بگیرد و ساختار غیرخطی داده‌های CIFAR-10 را بهتر نمایان کند.

کد زیر برای این بخش زده شده است:

```

1
2 print("--- Random Search (Subsets) ---")
3
4 # --- PCA ---
5 pca_pipeline = Pipeline([
6     ('pca', PCA(random_state=42)),
7     ('knn', KNeighborsClassifier())
8 ])
9
10 pca_param_dist = {
11     'pca__n_components': [50, 100, 150, 200], # Removed very large values
12     'knn__n_neighbors': [5, 7, 10, 15],
13     'knn__weights': ['uniform', 'distance'], # Added distance weighting
14     'knn__metric': ['manhattan', 'euclidean'] # Used different distance metrics

```

```
15 }
16
17 pca_random_search = RandomizedSearchCV(
18     pca_pipeline, param_distributions=pca_param_dist,
19     n_iter=15, cv=3, n_jobs=-1, random_state=42
20 )
21
22 pca_random_search.fit(x_train_sub, y_train_sub)
23 y_pred_pca_random = pca_random_search.predict(x_test_sub)
24 print("Best PCA Params:", pca_random_search.best_params_)
25 print("PCA Test Acc:", accuracy_score(y_test_sub, pca_random_search.predict(x_test_sub)))
26 print("PCA Classification Report:")
27 print(classification_report(y_test_sub, y_pred_pca_random, target_names=class_names))
28
29 # --- ISOMAP ---
30 iso_pipeline = Pipeline([
31     ('isomap', Isomap()),
32     ('knn', KNeighborsClassifier())
33 ])
34
35 iso_param_dist = {
36     'isomap__n_components': [30, 50, 100],
37     'isomap__n_neighbors': [10, 15, 20], # Increased Isomap neighbors for better global
38     'knn__n_neighbors': [5, 7, 10, 15],
39     'knn__weights': ['distance', 'uniform']
40 }
41
42 iso_random_search = RandomizedSearchCV(
43     iso_pipeline, param_distributions=iso_param_dist,
44     n_iter=10, cv=3, n_jobs=-1, random_state=42
45 )
46
47 iso_random_search.fit(x_train_sub, y_train_sub)
48 y_pred_iso_R = iso_random_search.predict(x_test_sub)
49 print("\nBest ISOMAP Params:", iso_random_search.best_params_)
50 print("ISOMAP Test Acc:", accuracy_score(y_test_sub, iso_random_search.predict(x_test_sub)))
51 print("ISOMAP Classification Report:")
52 print(classification_report(y_test_sub, y_pred_iso_R, target_names=class_names))
```

۵.۲.۲ اعمال PCA بر روی کل مجموعه داده‌ها

پس از یافتن بهترین پارامترهای مدل‌ها از طریق جستجوی تصادفی (Random Search)، در این مرحله تصمیم گرفته شد تا عملکرد الگوریتم را روی مقیاس بزرگ‌تری از داده‌ها بسنجیم. به همین منظور، الگوریتم PCA با پارامترهای بهینه‌شده (تعداد ۵۰ مؤلفه اصلی

برای کاهش ابعاد، طبقه‌بند KNN با ۱۵ همسایه، وزن‌دهی یکنواخت یا Uniform و معیار فاصله اقلیدسی) بر روی تمام ۵۰,۰۰۰ نمونه‌ی آموزشی CIFAR-10 آموزش داده شد.

تحلیل نتایج PCA روی کل داده‌ها: همان‌طور که در جدول ۹ مشاهده می‌شود، با افزایش حجم داده‌های آموزشی از ۲۰۰۰ نمونه به ۵۰,۰۰۰ نمونه، دقت کلی (Accuracy) مدل جهش قابل توجهی داشته و از حدود ۲۷٪ در فاز قبل، به ۴۱.۰۴٪ رسیده است. این بهبود چشم‌گیر به خوبی نشان می‌دهد که الگوریتم KNN به شدت نیازمند فضای ویژگی متراکم است؛ به عبارت دیگر، وقتی تعداد داده‌های آموزشی افزایش می‌یابد، در فضای ویژگی‌های کاهش‌یافته (فضای ۵۰ بعدی PCA)، نمونه‌های آموزشی فضای محلی را بهتر پوشش می‌دهند و در نتیجه، احتمال این‌که همسایه‌های پیدا شده برای داده‌های آزمون مربوط به کلاس صحیح باشند، بسیار بالاتر می‌رود.

عدم اعمال این مرحله برای ISOMAP: باید توجه داشت که این ارزیابی جامع تنها برای الگوریتم PCA انجام شده است. دلیل عدم استفاده از کل مجموعه داده‌ها برای الگوریتم ISOMAP، محدودیت‌های شدید محاسباتی این روش است. ISOMAP به دلیل نیاز به محاسبه گراف همسایگی در کل فضای داده‌ها و یافتن کوتاه‌ترین مسیرهای ژئودزیک میان تمام جفت‌نقطه‌ها، دارای پیچیدگی زمانی از مرتبه $O(N^3)$ است. اعمال این الگوریتم بر روی ماتریسی با ۵۰,۰۰۰ نمونه، نیازمند حافظه رم (RAM) بسیار بالا و زمان پردازش طولانی است که انجام آن را در عمل غیرمنطقی و از نظر هزینه محاسباتی بسیار گران می‌کند.

جدول ۹: گزارش عملکرد مدل PCA بهینه‌شده آموزش‌دیده روی کل 50,000 نمونه آموزشی

Class	Precision	Recall	F1-Score	Support
airplane	0.48	0.58	0.52	1000
automobile	0.66	0.35	0.46	1000
bird	0.27	0.41	0.33	1000
cat	0.36	0.20	0.26	1000
deer	0.29	0.51	0.37	1000
dog	0.53	0.26	0.35	1000
frog	0.35	0.53	0.42	1000
horse	0.64	0.30	0.41	1000
ship	0.45	0.66	0.54	1000
truck	0.67	0.30	0.41	1000
Accuracy		0.4104		10000
Macro Avg	0.47	0.41	0.41	10000
Weighted Avg	0.47	0.41	0.41	10000

کد زیر در این بخش زده شده است:

```

1 print("--- FULL Training Data: PCA + KNN ---")
2
3 # Use PCA to keep 90% of the variance (or use a fixed number like 50)
4 pca_full = PCA(n_components=50, random_state=42)
5
6 print("Fitting PCA on 50,000 samples...")
7 # Fit on all standardized training data

```



```

8 x_train_full_pca = pca_full.fit_transform(x_train_scaled)
9 x_test_full_pca = pca_full.transform(x_test_scaled)
10
11 print(f"Reduced features from 3072 to {x_train_full_pca.shape[1]}")
12
13 # Train KNN on the full dataset
14 knn_full = KNeighborsClassifier(n_neighbors=15, weights='uniform', metric='euclidean', n_jobs
15                               =-1)
16
17 print("Training KNN on 50,000 samples...")
18 knn_full.fit(x_train_full_pca, y_train.ravel())
19
20 print("Predicting on Test Data...")
21 y_pred_full = knn_full.predict(x_test_full_pca)
22
23 print("Full Dataset Accuracy:", accuracy_score(y_test.ravel(), y_pred_full))
24 print("PCA Classification Report:")
25 print(classification_report(y_test.ravel(), y_pred_full, target_names=class_names))

```

۳.۲ رمزگذار-رمزگشا (Autoencoder)

۱.۳.۲ استخراج ویژگی با استفاده از شبکه‌های خودرمزگذار (Autoencoders)

برای بهبود استخراج ویژگی و غلبه بر محدودیت‌های روش‌های خطی مانند PCA، از شبکه‌های عصبی عمیق در قالب معماری‌های خودرمزگذار (Autoencoder) استفاده شد. این شبکه‌ها به صورت بدون ناظر (Unsupervised) و با هدف کمینه‌سازی خطای بازسازی تصویر (تابع زیان MSE) با بهینه‌ساز Adam آموزش دیدند. در این رویکرد، داده‌ها به یک فضای پنهان (Latent Space) با ابعاد کاهش یافته (۶۴ و ۱۰۰ بُعد) نگاشت شده و سپس ویژگی‌های استخراج شده برای طبقه‌بندی به الگوریتم KNN داده شدند.

خودرمزگذار متراکم (Dense Autoencoder) و از دست رفتن فواصل مکانی در معماری متراکم، تصاویر سه‌کاناله با اندازه $32 \times 32 \times 3$ ابتدا به یک بردار مسطح با طول 3072 تبدیل می‌شوند. سپس این بردار از چند لایه کاملاً متصل (Fully Connected) عبور داده می‌شود تا به یک نمایش فشرده در فضای پنهان برسد. در این تمرین، ساختار رمزگذار شامل سه لایه خطی با اندازه‌های 512، 256 و در نهایت بعد فضای پنهان (latent_dim) است. در سمت رمزگشا نیز ساختار مشابهی به صورت معکوس برای بازسازی تصویر استفاده شده است. فعال‌ساز ReLU در لایه‌های میانی و تابع Sigmoid در خروجی استفاده شده است تا مقادیر پیکسل‌ها در بازه مناسب باقی بمانند. ساختار پیاده‌سازی این مدل در چارچوب PyTorch در کد زیر نشان داده شده است:

```

1 class DenseAutoencoder(nn.Module):
2     def __init__(self, latent_dim):
3         super(DenseAutoencoder, self).__init__()
4         self.encoder = nn.Sequential(
5             nn.Linear(32 * 32 * 3, 512),
6             nn.ReLU(),

```

```

7     nn.Linear(512, 256),
8     nn.ReLU(),
9     nn.Linear(256, latent_dim)
10    )
11    self.decoder = nn.Sequential(
12        nn.Linear(latent_dim, 256),
13        nn.ReLU(),
14        nn.Linear(256, 512),
15        nn.ReLU(),
16        nn.Linear(512, 32 * 32 * 3),
17        nn.Sigmoid()
18    )
19
20    def encode(self, x):
21        if len(x.shape) > 2:
22            x = x.view(x.size(0), -1)
23        return self.encoder(x)
24
25    def forward(self, x):
26        z = self.encode(x)
27        out = self.decoder(z)
28        return out
29

```

به دلیل تبدیل تصویر به بردار مسطح، ساختار مکانی پیکسل‌ها در این مدل از بین می‌رود. با این حال، وجود توابع فعال‌سازی غیرخطی باعث می‌شود مدل بتواند الگوهای پیچیده‌تری نسبت به روش‌های خطی مانند PCA استخراج کند. نتایج تجربی نشان دادند که این مدل با فضای پنهان ۱۰۰ بعدی توانست به دقت حدود ۴۴٪ برسد که نسبت به PCA عملکرد بهتری نشان می‌دهد.

خودرمزگذار کانولوشنی (CNN Autoencoder) و کیفیت بازسازی برای حفظ ساختار مکانی تصاویر، یک معماری مبتنی بر شبکه‌های پیچشی (CNN) شامل لایه‌های Conv2d در رمزگذار و ConvTranspose2d در رمزگشا طراحی شد. این شبکه ویژگی‌های محلی، لبه‌ها و بافت‌ها را به خوبی درک می‌کند؛ در نتیجه، نمودار تابع زیان (Loss) آن سریع‌تر کاهش یافته و تصاویر بازسازی‌شده از کیفیت بصری بسیار بالاتری نسبت به مدل متراکم برخوردارند.

چرا عملکرد KNN روی ویژگی‌های CNN ضعیف‌تر شد؟ با وجود بازسازی بهتر تصاویر در مدل CNN، دقت طبقه‌بندی آن با KNN در بهترین حالت به ۴۰٪ (در فضای ۶۴ بعدی) کاهش یافت. دلایل این افت عملکرد عبارتند از:

- ماهیت بدون ناظر بازسازی: شبکه CNN تنها برای بازسازی دقیق تصویر آموزش دیده است، نه برای تفکیک کلاس‌ها. بنابراین، ویژگی‌های استخراج‌شده شامل جزئیاتی مانند رنگ پس‌زمینه یا نورپردازی هستند که برای بازسازی مهم‌اند، اما کلاس تصویر را مشخص نمی‌کنند.

- ضعف KNN در فضاهای پیچیده: الگوریتم KNN بر اساس فاصله اقلیدسی کار می‌کند. فضای ویژگی‌های استخراج‌شده توسط CNN بسیار انتزاعی است. ممکن است یک قورباغه روی چمن و یک گربه روی چمن از نظر ساختار تصویر به هم شبیه باشند و

در فضای پنهان CNN فاصله اقلیدسی کمی داشته باشند، که این امر KNN را به اشتباه می‌اندازد. در مدل متراکم، به دلیل ترکیب کلی پیکسل‌ها، فاصله اقلیدسی کمی بهتر عمل کرده است.

اگر به جای بازسازی بدون ناظر، از یادگیری با ناظر (دادن برچسب‌ها به مدل) یا خودرمزگذارهای شرطی (Conditional Autoencoders) استفاده می‌شد، فضای پنهان مجبور به فاصله‌گذاری معنایی میان کلاس‌ها می‌شد و دقت تفکیک بسیار بالاتر می‌رفت. پیاده‌سازی این معماری به صورت زیر است:

```
1 class ConvAutoencoder(nn.Module):
2
3     def __init__(self, latent_dim):
4         super(ConvAutoencoder, self).__init__()
5
6         self.encoder_cnn = nn.Sequential(
7             nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
8             nn.BatchNorm2d(32),
9             nn.ReLU(),
10
11            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
12            nn.BatchNorm2d(64),
13            nn.ReLU(),
14
15            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
16            nn.BatchNorm2d(128),
17            nn.ReLU(),
18        )
19
20        self.encoder_fc = nn.Linear(128 * 4 * 4, latent_dim)
21
22        self.decoder_fc = nn.Linear(latent_dim, 128 * 4 * 4)
23
24        self.decoder_cnn = nn.Sequential(
25            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
26            nn.BatchNorm2d(64),
27            nn.ReLU(),
28
29            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1),
30            nn.BatchNorm2d(32),
31            nn.ReLU(),
32
33            nn.ConvTranspose2d(32, 3, kernel_size=4, stride=2, padding=1),
34            nn.Sigmoid()
35        )
36
37    def encode(self, x):
38        x = self.encoder_cnn(x)
39        x = x.view(x.size(0), -1)
```

```

40     z = self.encoder_fc(x)
41     return z
42
43     def forward(self, x):
44         x = self.encoder_cnn(x)
45         x = x.view(x.size(0), -1)
46         z = self.encoder_fc(x)
47
48         z = self.decoder_fc(z)
49         z = z.view(z.size(0), 128, 4, 4)
50         out = self.decoder_cnn(z)
51         return out
52

```

جداول نتایج طبقه‌بندی در جداول زیر، نتایج بهترین ابعاد پنهان برای هر دو معماری گزارش شده است.

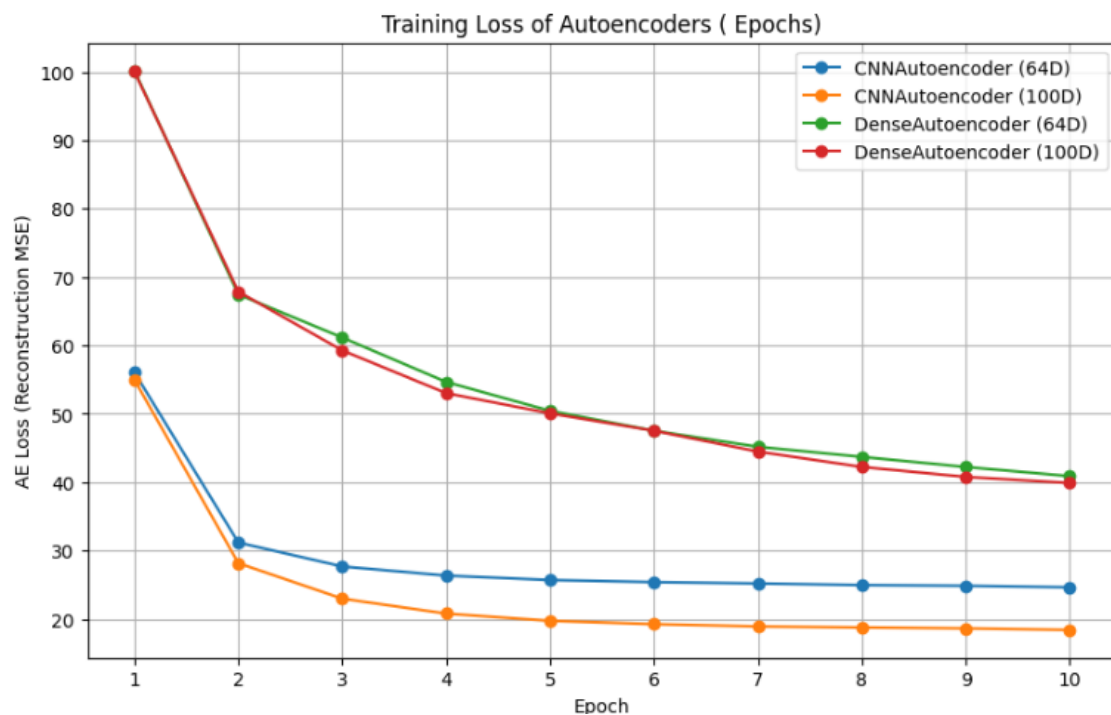
جدول ۱۰: گزارش عملکرد مدل KNN + Dense Autoencoder (فضای پنهان 100 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.44	0.64	0.52	1000
automobile	0.57	0.48	0.52	1000
bird	0.31	0.45	0.37	1000
cat	0.34	0.27	0.30	1000
deer	0.34	0.42	0.37	1000
dog	0.48	0.31	0.38	1000
frog	0.43	0.52	0.47	1000
horse	0.61	0.39	0.48	1000
ship	0.55	0.62	0.58	1000
truck	0.61	0.33	0.43	1000
Accuracy		0.44		10000
Macro Avg	0.47	0.44	0.44	10000
Weighted Avg	0.47	0.44	0.44	10000

جدول ۱۱: گزارش عملکرد مدل KNN + CNN Autoencoder (فضای پنهان 64 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.42	0.60	0.50	1000
automobile	0.70	0.33	0.45	1000
bird	0.25	0.45	0.32	1000
cat	0.33	0.18	0.24	1000
deer	0.28	0.53	0.36	1000
dog	0.50	0.24	0.32	1000
frog	0.37	0.44	0.40	1000
horse	0.66	0.30	0.42	1000
ship	0.47	0.67	0.55	1000
truck	0.73	0.26	0.38	1000
Accuracy		0.40		10000
Macro Avg	0.47	0.40	0.39	10000
Weighted Avg	0.47	0.40	0.39	10000

در زیر نیز نمودار لاس بر حسب اپوک برای هر دو اتوانکودر نشان داده شده است (برای هر اتوانکودر ۲ latent dims طراحی شده است):



شکل ۱۸: نمودارهای خطا بر حسب اپوک برای مدل های اتوانکودر

همانطور که از نمودارها مشخص است، مدل CNN با ارور کمتری توانسته است تصاویر را بازسازی کند. همچنین مشاهده می شود که Latent dims های بزرگتر بهتر توانسته اند فضای ویژگی را بازسازی بکنند. این نشان می دهد که بالاتر بودن ابعاد ویژگی در فضای latent به بازسازی تصاویر بهتر کمک کرده است. هرچند که در گام بعدی برای مدل KNN این موضوع برقرار نیست. KNN برای خروجی ویژگی از مدل CNN با latent dims برابر با ۶۴ نتیجه بهتری داشته است. همانطور که می بینیم ارزیابی آن روی داده تست بهتر است. نتایج ارزیابی KNN روی دو ساختار دیگر نیز به صورت زیر است:

جدول ۱۲: گزارش عملکرد مدل KNN + CNN Autoencoder (فضای پنهان 100 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.44	0.58	0.50	1000
automobile	0.73	0.26	0.38	1000
bird	0.25	0.47	0.32	1000
cat	0.32	0.18	0.23	1000
deer	0.26	0.57	0.35	1000
dog	0.48	0.20	0.29	1000
frog	0.35	0.39	0.37	1000
horse	0.66	0.27	0.39	1000
ship	0.47	0.66	0.55	1000
truck	0.76	0.21	0.32	1000
Accuracy		0.38		10000
Macro Avg	0.47	0.38	0.37	10000
Weighted Avg	0.47	0.38	0.37	10000

جدول ۱۳: گزارش عملکرد مدل KNN + Dense Autoencoder (فضای پنهان 64 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.42	0.63	0.51	1000
automobile	0.57	0.45	0.50	1000
bird	0.30	0.43	0.35	1000
cat	0.34	0.27	0.30	1000
deer	0.33	0.44	0.37	1000
dog	0.46	0.29	0.36	1000
frog	0.41	0.49	0.45	1000
horse	0.59	0.36	0.45	1000
ship	0.53	0.62	0.57	1000
truck	0.64	0.34	0.45	1000
Accuracy		0.43		10000
Macro Avg	0.46	0.43	0.43	10000
Weighted Avg	0.46	0.43	0.43	10000

۴.۲ خود رمزگذار متغیر (Variational Auto Encoder)

۱.۴.۲ مروری بر خود رمزگذارهای متغیر (Variational Autoencoders - VAEs)

خود رمزگذارهای متغیر (VAE) که برای اولین بار توسط Kingma و Welling در سال ۲۰۱۳ معرفی شدند، دسته‌ای از مدل‌های زبانی عمیق (Deep Generative Models) هستند. برخلاف خود رمزگذارهای سنتی که داده‌ها را به یک بردار ثابت و قطعی در فضای پنهان نگاشت می‌کنند، VAE داده‌ها را به یک توزیع احتمال نگاشت می‌کنند. این ویژگی باعث می‌شود فضای پنهان پیوسته و منظم باشد و بتوان از آن برای تولید داده‌های جدید و شبیه به داده‌های آموزشی استفاده کرد.

معماری و شیوه عملکرد یک VAE از دو بخش اصلی تشکیل شده است:

- رمزگذار (Encoder): داده ورودی x را دریافت کرده و به جای یک بردار ویژگی ثابت، پارامترهای یک توزیع احتمال (معمولاً میانگین μ و واریانس σ^2 از یک توزیع نرمال) را برای فضای پنهان تخمین می‌زند.
- رمزگشا (Decoder): یک نمونه تصادفی z را از توزیع به دست آمده در فضای پنهان می‌گیرد و سعی می‌کند داده اولیه را بازسازی کند (تولید $\hat{x}$).

ترفند بازپارامترسازی (Reparameterization Trick) برای اینکه شبکه عصبی بتواند از طریق الگوریتم پس انتشار خطا (Backpropagation) آموزش ببیند، باید بتوان از عملیات نمونه برداری مشتق گرفت. از آنجایی که نمونه برداری یک فرآیند تصادفی است و مشتق پذیر نیست، از

ترفند بازپارامترسازی استفاده می‌شود. در این روش، متغیر پنهان z به صورت زیر تعریف می‌شود:

$$z = \mu + \sigma \odot \epsilon$$

که در آن $\epsilon \sim \mathcal{N}(0, I)$ یک نویز تصادفی از توزیع نرمال استاندارد است و $\odot$ ضرب درایه به درایه را نشان می‌دهد. با این کار، تصادفی بودن به متغیر ϵ منتقل شده و گرادیان‌ها می‌توانند از μ و σ عبور کنند.

تابع زیان (Loss Function) تابع زیان در VAE ها از دو بخش اساسی تشکیل شده است:

$$\mathcal{L} = \text{Loss Reconstruction} + D_{KL}(q_{\phi}(z|x)||p(z))$$

۱. خطای بازسازی (Reconstruction Loss): میزان شباهت داده تولید شده $\hat{x}$ به داده ورودی x را می‌سنجد (معمولاً با استفاده از MSE یا Cross-Entropy).

۲. واگرایی کولبک-لایبِر (KL Divergence): این جمله به عنوان یک تنظیم‌کننده (Regularizer) عمل می‌کند و توزیع تخمین زده شده توسط رمزگذار $q_{\phi}(z|x)$ را مجبور می‌کند تا حد امکان به توزیع پیشین (Prior) که معمولاً یک توزیع نرمال استاندارد $p(z) = \mathcal{N}(0, I)$ است، نزدیک شود.

ترکیب این دو خطا باعث می‌شود تا فضای پنهان هم برای بازسازی داده‌ها مناسب باشد (حفظ اطلاعات) و هم ساختار پیوسته‌ای داشته باشد تا با درون‌یابی (Interpolation) بین نقاط آن، داده‌های معناداری تولید شود.

۲.۴.۲ پیاده‌سازی معماری‌های VAE

برای بررسی عملکرد فضای پنهان پیوسته، دو مدل DenseVAE و CNN_VAE پیاده‌سازی شدند.

معماری DenseVAE در این مدل، تصاویر به بردارهای یک‌بعدی تبدیل شده و از لایه‌های تمام‌متصل برای رمزگذاری و رمزگشایی استفاده می‌شود. خروجی رمزگذار شامل دو بردار μ و $\log(\sigma^2)$ است که با ترفند بازپارامترسازی به z تبدیل می‌شوند.

```

1 class DenseVAE(nn.Module):
2     def __init__(self, z_size):
3         super(DenseVAE, self).__init__()
4         self.z_size = z_size
5
6         self.encoder = nn.Sequential(
7             nn.Linear(32 * 32 * 3, 512),
8             nn.ReLU(),
9             nn.Linear(512, 256),
10            nn.ReLU()
11        )
12        self.fc_mu = nn.Linear(256, z_size)
13        self.fc_logvar = nn.Linear(256, z_size)
14
15        self.decoder = nn.Sequential(
16            nn.Linear(z_size, 256),
17            nn.ReLU(),
    
```

```

18     nn.Linear(256, 512),
19     nn.ReLU(),
20     nn.Linear(512, 32 * 32 * 3),
21     nn.Sigmoid()
22 )
23
24     def encode(self, x):
25
26         if len(x.shape) > 2:
27             x = x.view(x.size(0), -1)
28             h = self.encoder(x)
29             return self.fc_mu(h), self.fc_logvar(h)
30
31     def reparameterize(self, mu, logvar):
32         std = torch.exp(0.5 * logvar)
33         eps = torch.randn_like(std)
34         return mu + eps * std
35
36     def forward(self, x):
37         if len(x.shape) > 2:
38             x = x.view(x.size(0), -1)
39             mu, logvar = self.encode(x)
40             z = self.reparameterize(mu, logvar)
41             out = self.decoder(z)
42             return out, mu, logvar
43

```

معماری CNN_VAE این مدل با استفاده از لایه‌های کانولوشن (Conv2d) در رمزگذار و کانولوشن ترانهاد (ConvTranspose2d) در رمزگشا، ساختار مکانی تصاویر را حفظ می‌کند.

```

1     class VAE(nn.Module):
2     def __init__(self, z_size, image_size=32, channel_num=3, kernel_num=128):
3     super().__init__()
4     self.image_size = image_size
5     self.channel_num = channel_num
6     self.kernel_num = kernel_num
7     self.z_size = z_size
8
9
10    self.encoder = nn.Sequential(
11    self._conv(channel_num, kernel_num // 4),
12    self._conv(kernel_num // 4, kernel_num // 2),
13    self._conv(kernel_num // 2, kernel_num),
14    )
15
16    self.feature_size = image_size // 8
17    self.feature_volume = kernel_num * (self.feature_size ** 2)
18
19    self.q_mean = nn.Linear(self.feature_volume, z_size)
20    self.q_logvar = nn.Linear(self.feature_volume, z_size)
21    self.project = nn.Linear(z_size, self.feature_volume)
22

```

```

23
24     self.decoder = nn.Sequential(
25         self._deconv(kernel_num, kernel_num // 2),
26         self._deconv(kernel_num // 2, kernel_num // 4),
27         self._deconv(kernel_num // 4, channel_num),
28         nn.Sigmoid()
29     )
30
31     def forward(self, x):
32         mean, logvar, z = self.encode(x)
33         z_projected = self.project(z).view(
34             -1, self.kernel_num,
35             self.feature_size,
36             self.feature_size,
37         )
38         x_reconstructed = self.decoder(z_projected)
39         return (mean, logvar), x_reconstructed
40
41     def encode(self, x):
42         encoded = self.encoder(x)
43         mean, logvar = self.q(encoded)
44         z = self.z(mean, logvar)
45         return mean, logvar, z
46
47     def q(self, encoded):
48         unrolled = encoded.view(-1, self.feature_volume)
49         return self.q_mean(unrolled), self.q_logvar(unrolled)
50
51     def z(self, mean, logvar):
52         std = torch.exp(0.5 * logvar)
53         eps = torch.randn_like(std)
54         return mean + eps * std
55
56     def _conv(self, channel_size, kernel_num):
57         return nn.Sequential(
58             nn.Conv2d(channel_size, kernel_num, kernel_size=4, stride=2, padding=1),
59             nn.BatchNorm2d(kernel_num),
60             nn.LeakyReLU(0.2),
61         )
62
63     def _deconv(self, channel_num, kernel_num):
64         return nn.Sequential(
65             nn.ConvTranspose2d(channel_num, kernel_num, kernel_size=4, stride=2, padding=1),
66             nn.BatchNorm2d(kernel_num),
67             nn.LeakyReLU(0.2),
68         )
69

```

۳.۴.۲ نتایج طبقه‌بندی با KNN روی فضای پنهان

در ادامه، نتایج اعمال KNN روی ویژگی‌های استخراج‌شده (بردار μ) از مدل‌های آموزش‌دیده برای ابعاد پنهان ۶۴ و ۱۰۰ آورده شده است.

جدول ۱۴: گزارش عملکرد مدل KNN + Dense VAE (فضای پنهان 64 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.43	0.60	0.50	1000
automobile	0.54	0.48	0.51	1000
bird	0.29	0.43	0.34	1000
cat	0.30	0.22	0.26	1000
deer	0.32	0.40	0.36	1000
dog	0.44	0.29	0.35	1000
frog	0.40	0.50	0.45	1000
horse	0.55	0.40	0.46	1000
ship	0.55	0.56	0.56	1000
truck	0.60	0.34	0.43	1000
Accuracy		0.42		10000
Macro Avg	0.44	0.42	0.42	10000
Weighted Avg	0.44	0.42	0.42	10000

جدول ۱۵: گزارش عملکرد مدل KNN + Dense VAE (فضای پنهان 100 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.43	0.60	0.50	1000
automobile	0.51	0.45	0.48	1000
bird	0.29	0.44	0.35	1000
cat	0.29	0.23	0.26	1000
deer	0.33	0.38	0.35	1000
dog	0.42	0.25	0.32	1000
frog	0.39	0.49	0.43	1000
horse	0.51	0.39	0.44	1000
ship	0.54	0.54	0.54	1000
truck	0.58	0.34	0.43	1000
Accuracy		0.41		10000
Macro Avg	0.43	0.41	0.41	10000
Weighted Avg	0.43	0.41	0.41	10000

جدول ۱۶: گزارش عملکرد مدل KNN + CNN VAE (فضای پنهان 64 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.39	0.62	0.48	1000
automobile	0.75	0.27	0.40	1000
bird	0.24	0.51	0.33	1000
cat	0.38	0.17	0.24	1000
deer	0.28	0.55	0.37	1000
dog	0.51	0.18	0.26	1000
frog	0.34	0.43	0.38	1000
horse	0.77	0.28	0.42	1000
ship	0.49	0.65	0.56	1000
truck	0.82	0.12	0.22	1000
Accuracy		0.38		10000
Macro Avg	0.50	0.38	0.36	10000
Weighted Avg	0.50	0.38	0.36	10000

جدول ۱۷: گزارش عملکرد مدل KNN + CNN VAE (فضای پنهان 100 بعدی)

Class	Precision	Recall	F1-Score	Support
airplane	0.38	0.61	0.47	1000
automobile	0.78	0.23	0.36	1000
bird	0.23	0.52	0.32	1000
cat	0.38	0.16	0.23	1000
deer	0.27	0.55	0.36	1000
dog	0.48	0.16	0.24	1000
frog	0.36	0.41	0.38	1000
horse	0.80	0.24	0.37	1000
ship	0.45	0.63	0.52	1000
truck	0.81	0.09	0.16	1000
Accuracy		0.36		10000
Macro Avg	0.49	0.36	0.34	10000
Weighted Avg	0.49	0.36	0.34	10000

۴.۴.۲ تحلیل و مقایسه نتایج

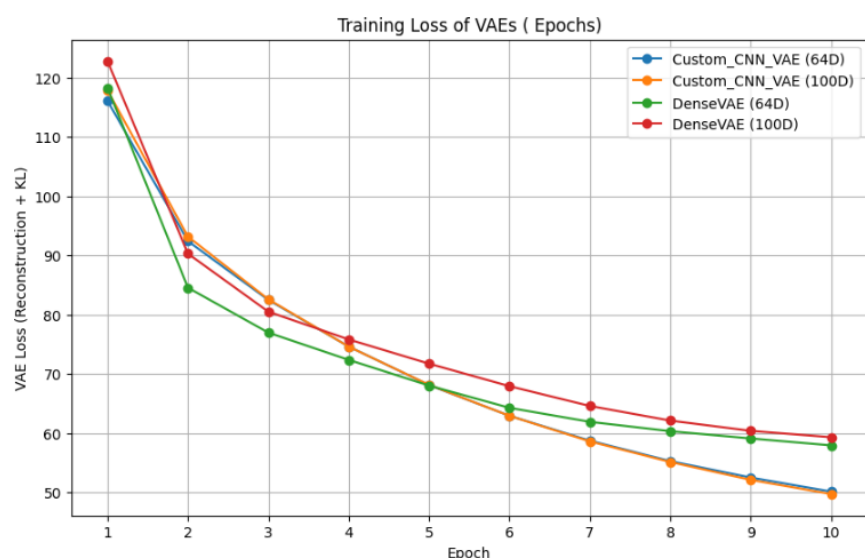
بررسی نتایج به دست آمده حاوی نکات مهمی درباره ماهیت VAE ها و تناسب آن ها با طبقه‌بند KNN است.

۱. مقایسه Dense VAE با Dense AE در نتایج قبلی، خودرمزگذار Dense معمولی به دقت حدود 0.44 رسیده بود، اما مدل Dense VAE در بهترین حالت (۶۴ بعدی) به دقت 0.42 و در ۱۰۰ بعدی به 0.41 رسیده است. این افت اندک به دلیل وجود جمله واگرایی کولبک-لایبلر (KL Divergence) در تابع زیان VAE است که به عنوان یک تنظیم‌کننده قوی عمل کرده و توزیع ویژگی‌ها را به توزیع نرمال استاندارد نزدیک می‌کند. در نتیجه مرز بین کلاس‌ها تا حدی کاهش یافته و عملکرد KNN که مبتنی بر فاصله اقلیدسی است کاهش می‌یابد.

۲. مقایسه Dense VAE و CNN VAE مدل CNN VAE دقت‌های پایین‌تری (0.38 برای ۶۴ بعدی و 0.36 برای ۱۰۰ بعدی) ثبت کرده است. اگرچه این مدل‌ها در بازسازی تصاویر عملکرد بسیار بهتری دارند، اما فضای ویژگی ایجاد شده توسط CNN بسیار انتزاعی و غیرخطی است و فاصله اقلیدسی مورد استفاده در KNN نمی‌تواند ساختار معنایی این فضا را به خوبی مدل کند.

۳. تأثیر ابعاد فضای پنهان در هر دو مدل، افزایش بعد فضای پنهان از ۶۴ به ۱۰۰ بهبودی ایجاد نکرده و حتی باعث افت دقت شده است. در VAE ها بخشی از ابعاد اضافی ممکن است صرف مدل‌سازی نویز توزیع پیشین شود و اطلاعات تمایزدهنده کلاس‌ها را افزایش ندهد.

در مجموع، هر چند VAE ها در یادگیری توزیع داده‌ها و بازسازی تصاویر بسیار قدرتمند هستند اما در تنظیمات بدون نظارت لزوماً فضای ویژگی مناسبی برای طبقه‌بندی مبتنی بر فاصله مانند KNN تولید نمی‌کنند. بنابراین استفاده از مدل‌های شرطی مانند Conditional VAE یا به‌کارگیری طبقه‌بند قوی‌تر مانند SVM یا شبکه‌های کاملاً متصل می‌تواند عملکرد بهتری ایجاد کند. در شکل زیر ۴ نمودار برای نمایش مقدار خطا بر حسب اپوک نشان داده شده است.



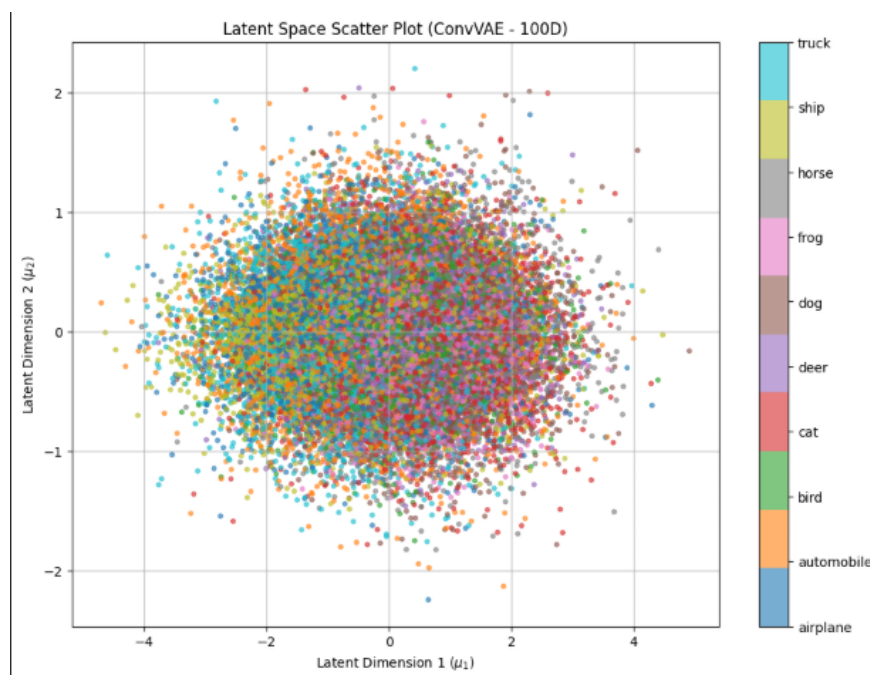
شکل ۱۹: نمودارهای خطا بر حسب اپوک برای مدل‌های خود رمزگذار متغیر

مقایسه معماری‌ها مدل‌های مبتنی بر کانولوشن (CNN_VAE) در هر دو بعد ۶۴ و ۱۰۰، در نهایت به مقدار زیان کمتری نسبت به مدل‌های Dense رسیده‌اند. این موضوع تأیید می‌کند که لایه‌های کانولوشن به دلیل حفظ روابط مکانی در تصاویر CIFAR-10، توانایی بسیار بالاتری در بازسازی دقیق تصویر و استخراج ویژگی‌های کلیدی دارند.

تأثیر ابعاد فضای پنهان در مدل‌های Dense، افزایش بعد از ۶۴ به ۱۰۰ منجر به افزایش جزئی در مقدار زیان شده است (نمودار قرمز بالاتر از سبز)، که می‌تواند نشان‌دهنده پیچیدگی بیشتر در بهینه‌سازی فضای ۱۰۰ بعدی در ساختار تمام‌متصل باشد. در مقابل، در مدل‌های CNN، هر دو بعد ۶۴ و ۱۰۰ رفتار بسیار نزدیکی به هم دارند و با اختلافی ناچیز در انتهای اپوک دهم همگرا شده‌اند، که نشان‌دهنده پایداری بیشتر معماری کانولوشنی نسبت به تغییر ابعاد است.

۵.۲ کاوش در فضای latent

شکل زیر توزیع ویژگی‌ها را در فضای latent نشان می‌دهد. شکل زیر فقط در دو بعد اول این فضا ترسیم شده است:



شکل ۲۰: ترسیم نمودار دو بعدی از ویژگی‌های داده‌ها در فضای latent

۱.۵.۲ تحلیل نمودار پراکندگی فضای پنهان مدل ConvVAE (فضای ۱۰۰ بعدی)

این نمودار نشان‌دهنده توزیع داده‌های آموزشی CIFAR-10 در فضای پنهان (Latent Space) مدل Convolutional VAE است. بر اساس کد ارائه شده، مختصات رسم شده تنها مربوط به دو بُعد اول بردار میانگین (یعنی μ_1 و μ_2) از یک فضای پنهان ۱۰۰ بعدی است. در ادامه رفتار این فضا از زوایای مختلف تحلیل شده است:

۱. تأثیر واگرایی کولبک-لایلر (KL Divergence) و مرکزیت داده‌ها مهم‌ترین مشاهده بصری در این نمودار، تجمع گروی و متراکم تمامی نقاط حول مبدأ مختصات $(0, 0)$ است. این رفتار مستقیماً ناشی از ترم واگرایی KL در تابع زیان (Loss Function) مدل VAE است. این ترم، توزیع فضای پنهان $q(z|x)$ را مجبور می‌کند تا حد امکان به توزیع پیشین (Prior) که یک توزیع نرمال استاندارد $\mathcal{N}(0, 1)$ است، نزدیک شود. به همین دلیل، مقادیر میانگین (μ) استخراج‌شده برای تصاویر مختلف، همگی در محدوده‌ای نزدیک به صفر (بین -4 تا 4 برای μ_1 و -2 تا 2 برای μ_2) فشرده شده‌اند و فضا پیوستگی بالایی دارد.

۲. هم‌پوشانی شدید کلاس‌ها (فقدان جدایی‌پذیری خطی) همان‌طور که از رنگ‌بندی نقاط (بر اساس ۱۰ کلاس مختلف CIFAR-10) مشخص است، هیچ‌گونه مرزبندی واضح یا خوشه‌بندی (Clustering) تفکیک‌پذیری بین کلاس‌ها دیده نمی‌شود و رنگ‌ها کاملاً در هم آمیخته‌اند. دلایل این پدیده عبارتند از:

- ماهیت بدون ناظر (Unsupervised) بودن VAE: شبکه‌ی VAE بدون اطلاع از برچسب کلاس‌ها آموزش می‌بیند و تنها به دنبال بازسازی پیکسل‌های تصویر است. بنابراین، ممکن است دو تصویر از کلاس‌های متفاوت (مثلاً «سگ» و «اسب») که پس‌زمینه‌ی سبز مشابهی دارند، در فضای پنهان در کنار یکدیگر قرار گیرند.
- فقدان حاشیه (Margin): برخلاف روش‌های نظارت‌شده (مثل Contrastive Learning یا شبکه‌های دارای Cross-Entropy Loss) که سعی در دور کردن کلاس‌های متفاوت دارند، VAE برای تولید یک فضای پیوسته (جهت تولید تصاویر جدید) تلاش می‌کند، نه یک فضای تفکیک‌شده.

۳. محدودیت تصویرسازی دو بُعد از صد بُعد (Dimensionality Projection) کد رسم نمودار (`z_train_features[:, 0]`) و (`z_train_features[:, 1]`) نشان می‌دهد که ما تنها در حال مشاهده‌ی ۲ بُعد اول از ۱۰۰ بُعد استخراج‌شده هستیم. این یک برش بسیار ساده‌انگارانه از فضای داده‌هاست. در حالی که این دو بُعد به شدت در هم تنیده به نظر می‌رسند، ممکن است در ۹۸ بُعد دیگر، الگوهای تمایزدهنده‌ای وجود داشته باشد. برای مشاهده بهتر خوشه‌ها در فضاهای با ابعاد بالا، استفاده از الگوریتم‌های کاهش ابعاد مانند t-SNE یا UMAP روی کل بردار ۱۰۰ بعدی، بسیار منطقی‌تر از انتخاب صرفاً دو ویژگی اول است.

۴. ارتباط تصویر با افت دقت طبقه‌بند KNN این نمودار دلیل اصلی افت عملکرد الگوریتم KNN (با دقت 0.38) روی ویژگی‌های ConvVAE را به وضوح توضیح می‌دهد. الگوریتم KNN بر پایه فاصله اقلیدسی عمل می‌کند. وقتی نمونه‌های کلاس‌های مختلف تا این حد در فضای پنهان فشرده و در هم تنیده شده‌اند، یافتن همسایه‌های نزدیک که متعلق به همان کلاس هدف باشند، به شدت دشوار می‌شود. این فشردگی که برای تولید تصاویر (Generation) یک مزیت است، برای وظیفه‌ی طبقه‌بندی (Classification) یک چالش بزرگ محسوب می‌شود.

۲.۵.۲ تحلیل تولید منیفولد (Manifold Generation) و پیمایش فضای پنهان

بخشی از این تصویر به دست آمده از نتیجه کد به صورت زیر است:



شکل ۲۱: تصویر تولید منیفولد

تصویر و کد ارائه شده، نمایشی از چگونگی عملکرد شبکه رمزگشا (Decoder) مدل ConvVAE در تبدیل بردارهای انتزاعی فضای پنهان به تصاویر معنادار است. این خروجی نشان‌دهنده روند تولید تصویر و تأثیر تغییرات مختصات فضای پنهان بر ویژگی‌های بصری خروجی است. در ادامه جزئیات این فرآیند تحلیل شده است:

۱. منطق کد و نحوه پیمایش فضا در کد نوشته شده، یک شبکه (Grid) دو بعدی 10×10 با استفاده از تابع `np.linspace` بین مقادیر -2.0 تا 2.0 ایجاد شده است. با توجه به اینکه فضای پنهان این مدل 100 بعدی است، کد ابتدا یک بردار تماماً صفر ایجاد می‌کند ($z_i = 0$ برای $i > 2$). سپس تنها دو بُعد اول این بردار (z_1 و z_2) را با مقادیر این شبکه دو بعدی مقداردهی می‌کند. این بردار به عنوان ورودی به Decoder داده می‌شود تا تصویر متناظر با آن نقطه از فضا تولید شود.

۲. پیوستگی و تغییرات هموار (Smooth Interpolation) مهم‌ترین ویژگی مشاهده شده در تصویر خروجی، تغییرات بسیار نرم و پیوسته رنگ‌ها و ساختارهای کلی (مانند تغییر از تونالیته تیره به روشن یا تغییر رنگ زمینه) با حرکت در طول سطرها و ستون‌ها است. این رفتار نشان‌دهنده موفقیت مدل VAE در ایجاد یک فضای پنهان پیوسته است. برخلاف Autoencoderهای معمولی که فضای پنهان گسسته‌ای دارند، استفاده از توزیع نرمال و ترم KL Divergence در VAE تضمین می‌کند که نقاط نزدیک به هم در فضای پنهان، تصاویر شبیه به هم تولید کنند.

۳. دلیل محو بودن و عدم وضوح تصاویر تولید شده تصاویر موجود در این منیفولد بسیار تار (Blurry) هستند و شکل اشیاء مشخصی از کلاس‌های CIFAR-10 (مانند ماشین، سگ و...) در آن‌ها دیده نمی‌شود. این مسئله دو دلیل عمده دارد:

- صفر بودن 98 بُعد دیگر: در این کد، برای رسم یک تصویر دو بعدی، 98 ویژگی دیگر فضای پنهان روی عدد صفر تنظیم شده‌اند. در یک مدل 100 بعدی، اطلاعات حیاتی مربوط به لبه‌ها، بافت‌ها و شکل دقیق اشیاء در تمام 100 بُعد پخش شده است. با صفر کردن آن‌ها، ما تنها در حال مشاهده یک «برش بسیار نازک» از این فضای پیچیده هستیم که طبیعتاً فاقد جزئیات کافی برای بازسازی یک تصویر کامل است.

- ذات مدل‌های VAE: مدل‌های VAE به دلیل استفاده از خطای بازسازی میانگین مربعات (MSE)، به طور ذاتی تمایل به تولید تصاویر محوتر نسبت به مدل‌هایی مانند GAN دارند.

۴. نتیجه‌گیری این آزمایش به خوبی نشان می‌دهد که مدل ConvVAE توانسته است ویژگی‌های کلان تصاویر (مانند رنگ‌بندی کلی و روشنایی) را در ابعاد اولیه توزیع نرمال خود (محدوده ۲- تا ۲) کدگذاری کند. با حرکت در این فضای هندسی، Decoder قادر است تصاویر جدیدی تولید کند که ترکیبی پیوسته از ویژگی‌های یادگرفته شده هستند. نمونه‌هایی از تصاویر تولید شده توسط فضای latent به صورت زیر آورده شده است:



شکل ۲۲: نمونه تصاویر تولید شده

تصاویر بیشتر در داخل نوتبوک به صورت کامل آورده شده است. همانطور که مشاهده می‌شود تصاویر تولیدی آنچنان خوب نیستند. با توجه به تحقیقاتی که ما داشتیم، تلاش‌های زیادی برای استفاده از این ساختار برای CIFAR-10 انجام شده است اما نتایج آنچنان مطلوب نبودند. برای Fashion-Mnist این موضوع حل شده است.

۳.۵.۲ کد کامل

```

1  # -*- coding: utf-8 -*-
2  """ML_Hw4_Q2.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/17-tXMIariW1DcMKgVozurDhpCt1tsVLt
8  """
9
10 from keras.datasets import cifar10
11 (x_train , y_train), (x_test , y_test) = cifar10.load_data()
12
13 """# Flatten Input 3 Channel
14 # StandardScaler transform on the dataset and making subset
15

```



```
۱۶
۱۷ """
۱۸
۱۹ import torch
۲۰ SEED = 42
۲۱ torch.manual_seed(SEED)
۲۲ if torch.cuda.is_available():
۲۳     torch.cuda.manual_seed_all(SEED)
۲۴
۲۵ import numpy as np
۲۶ from keras.datasets import cifar10
۲۷ from sklearn.decomposition import PCA
۲۸ from sklearn.manifold import Isomap
۲۹ from sklearn.neighbors import KNeighborsClassifier
۳۰ from sklearn.model_selection import RandomizedSearchCV
۳۱ from sklearn.pipeline import Pipeline
۳۲ from sklearn.preprocessing import StandardScaler
۳۳ from sklearn.metrics import accuracy_score
۳۴
۳۵ print(f"the shape of the x_train is {x_train.shape}")
۳۶ print(f"the shape of the x_test is {x_test.shape}")
۳۷
۳۸ x_train_R = x_train.reshape(x_train.shape[0], -1)
۳۹ x_test_R = x_test.reshape(x_test.shape[0], -1)
۴۰ print("-----")
۴۱ print("After reshaping the data to Vector:")
۴۲ print(f"the shape of the x_train is {x_train_R.shape}")
۴۳ print(f"the shape of the x_test is {x_test_R.shape}")
۴۴
۴۵
۴۶ scaler = StandardScaler()
۴۷ x_train_scaled = scaler.fit_transform(x_train_R)
۴۸ x_test_scaled = scaler.transform(x_test_R)
۴۹
۵۰
۵۱ n_train_samples = 2000
۵۲ n_test_samples = 500
۵۳
۵۴ x_train_sub = x_train_scaled[:n_train_samples]
۵۵ y_train_sub = y_train[:n_train_samples].ravel()
```



```
۵۶
۵۷ x_test_sub = x_test_scaled[:n_test_samples]
۵۸ y_test_sub = y_test[:n_test_samples].ravel()
۵۹ print("-----")
۶۰ print("Data ready")
۶۱
۶۲ class_names = [
۶۳     'airplane',
۶۴     'automobile',
۶۵     'bird',
۶۶     'cat',
۶۷     'deer',
۶۸     'dog',
۶۹     'frog',
۷۰     'horse',
۷۱     'ship',
۷۲     'truck'
۷۳ ]
۷۴
۷۵ """#Using Fixed Parametrs for PCA, Isomap and KNN"""
۷۶
۷۷ from sklearn.pipeline import Pipeline
۷۸ from sklearn.decomposition import PCA
۷۹ from sklearn.manifold import Isomap
۸۰ from sklearn.neighbors import KNeighborsClassifier
۸۱ from sklearn.model_selection import RandomizedSearchCV
۸۲ from sklearn.metrics import accuracy_score
۸۳ from sklearn.metrics import classification_report
۸۴
۸۵
۸۶ print("--- Fixed Parameters (Subsets) ---")
۸۷
۸۸ # PCA + KNN
۸۹ # Added random_state=42 to PCA for reproducible results
۹۰ pca_fixed = PCA(n_components=100, random_state=42)
۹۱ knn_pca_fixed = KNeighborsClassifier(n_neighbors=7, weights='uniform', metric='euclidean')
۹۲
۹۳ x_train_sub_pca = pca_fixed.fit_transform(x_train_sub)
۹۴ x_test_sub_pca = pca_fixed.transform(x_test_sub)
۹۵
```



```
۹۶ knn_pca_fixed.fit(x_train_sub_pca, y_train_sub)
۹۷ y_pred_pca = knn_pca_fixed.predict(x_test_sub_pca)
۹۸ print("Fixed PCA Acc:", accuracy_score(y_test_sub, knn_pca_fixed.predict(x_test_sub_pca)))
۹۹ print("PCA Classification Report:")
۱۰۰ print(classification_report(y_test_sub, y_pred_pca, target_names=class_names))
۱۰۱
۱۰۲
۱۰۳ # ISOMAP + KNN
۱۰۴ iso_fixed = Isomap(n_components=50, n_neighbors=15)
۱۰۵ knn_iso_fixed = KNeighborsClassifier(n_neighbors=7, weights='uniform')
۱۰۶
۱۰۷ x_train_sub_iso = iso_fixed.fit_transform(x_train_sub)
۱۰۸ x_test_sub_iso = iso_fixed.transform(x_test_sub)
۱۰۹
۱۱۰ knn_iso_fixed.fit(x_train_sub_iso, y_train_sub)
۱۱۱ y_pred_iso = knn_iso_fixed.predict(x_test_sub_iso)
۱۱۲ print("Fixed ISOMAP Acc:", accuracy_score(y_test_sub, knn_iso_fixed.predict(x_test_sub_iso)))
۱۱۳ print("ISOMAP Classification Report:")
۱۱۴ print(classification_report(y_test_sub, y_pred_iso, target_names=class_names))
۱۱۵
۱۱۶ """#Using Random search for choosing Parametrs"""
۱۱۷
۱۱۸ print("--- Random Search (Subsets) ---")
۱۱۹
۱۲۰ # --- PCA ---
۱۲۱ pca_pipeline = Pipeline([
۱۲۲     ('pca', PCA(random_state=42)),
۱۲۳     ('knn', KNeighborsClassifier())
۱۲۴ ])
۱۲۵
۱۲۶ pca_param_dist = {
۱۲۷     'pca__n_components': [50, 100, 150, 200], # Removed very large values
۱۲۸     'knn__n_neighbors': [5, 7, 10, 15],
۱۲۹     'knn__weights': ['uniform', 'distance'], # Added distance weighting
۱۳۰     'knn__metric': ['manhattan', 'euclidean'] # Used different distance metrics
۱۳۱ }
۱۳۲
۱۳۳ pca_random_search = RandomizedSearchCV(
۱۳۴     pca_pipeline, param_distributions=pca_param_dist,
۱۳۵     n_iter=15, cv=3, n_jobs=-1, random_state=42
```

```
۱۳۶ )
۱۳۷
۱۳۸ pca_random_search.fit(x_train_sub, y_train_sub)
۱۳۹ y_pred_pca_random = pca_random_search.predict(x_test_sub)
۱۴۰ print("Best PCA Params:", pca_random_search.best_params_)
۱۴۱ print("PCA Test Acc:", accuracy_score(y_test_sub, pca_random_search.predict(x_test_sub)))
۱۴۲ print("PCA Classification Report:")
۱۴۳ print(classification_report(y_test_sub, y_pred_pca_random, target_names=class_names))
۱۴۴
۱۴۵ # --- ISOMAP ---
۱۴۶ iso_pipeline = Pipeline([
۱۴۷     ('isomap', Isomap()),
۱۴۸     ('knn', KNeighborsClassifier())
۱۴۹ ])
۱۵۰
۱۵۱ iso_param_dist = {
۱۵۲     'isomap__n_components': [30, 50, 100],
۱۵۳     'isomap__n_neighbors': [10, 15, 20], # Increased Isomap neighbors for better global structure
        ↪ discovery
۱۵۴     'knn__n_neighbors': [5, 7, 10, 15],
۱۵۵     'knn__weights': ['distance', 'uniform']
۱۵۶ }
۱۵۷
۱۵۸ iso_random_search = RandomizedSearchCV(
۱۵۹     iso_pipeline, param_distributions=iso_param_dist,
۱۶۰     n_iter=10, cv=3, n_jobs=-1, random_state=42
۱۶۱ )
۱۶۲ iso_random_search.fit(x_train_sub, y_train_sub)
۱۶۳ y_pred_iso_R = iso_random_search.predict(x_test_sub)
۱۶۴ print("\nBest ISOMAP Params:", iso_random_search.best_params_)
۱۶۵ print("ISOMAP Test Acc:", accuracy_score(y_test_sub, iso_random_search.predict(x_test_sub)))
۱۶۶ print("ISOMAP Classification Report:")
۱۶۷ print(classification_report(y_test_sub, y_pred_iso_R, target_names=class_names))
۱۶۸
۱۶۹ """#Using all training Data for PCA + KNN"""
۱۷۰
۱۷۱ print("---- FULL Training Data: PCA + KNN ----")
۱۷۲
۱۷۳ # Use PCA to keep 90% of the variance (or use a fixed number like 50)
۱۷۴ pca_full = PCA(n_components=50, random_state=42)
```

```
۱۷۵
۱۷۶ print("Fitting PCA on 50,000 samples...")
۱۷۷ # Fit on all standardized training data
۱۷۸ x_train_full_pca = pca_full.fit_transform(x_train_scaled)
۱۷۹ x_test_full_pca = pca_full.transform(x_test_scaled)
۱۸۰
۱۸۱ print(f"Reduced features from 3072 to {x_train_full_pca.shape[1]}")
۱۸۲
۱۸۳ # Train KNN on the full dataset
۱۸۴ knn_full = KNeighborsClassifier(n_neighbors=15, weights='uniform', metric='euclidean', n_jobs=-1)
۱۸۵
۱۸۶ print("Training KNN on 50,000 samples...")
۱۸۷ knn_full.fit(x_train_full_pca, y_train.ravel())
۱۸۸
۱۸۹ print("Predicting on Test Data...")
۱۹۰ y_pred_full = knn_full.predict(x_test_full_pca)
۱۹۱
۱۹۲ print("Full Dataset Accuracy:", accuracy_score(y_test.ravel(), y_pred_full))
۱۹۳ print("PCA Classification Report:")
۱۹۴ print(classification_report(y_test.ravel(), y_pred_full, target_names=class_names))
۱۹۵
۱۹۶ print(f"shape of the x_train:{x_train.shape}")
۱۹۷ print(f"shape of the x_test:{x_test.shape}")
۱۹۸
۱۹۹ import numpy as np
۲۰۰ import torch
۲۰۱ import torch.nn as nn
۲۰۲ import torch.optim as optim
۲۰۳ from torch.utils.data import TensorDataset, DataLoader
۲۰۴ from keras.datasets import cifar10
۲۰۵ from sklearn.neighbors import KNeighborsClassifier
۲۰۶ from sklearn.metrics import classification_report
۲۰۷ import matplotlib.pyplot as plt
۲۰۸
۲۰۹
۲۱۰ x_train_t1 = np.transpose(x_train, (0, 3, 1, 2)).astype('float32') / 0.255
۲۱۱ x_test_t1 = np.transpose(x_test, (0, 3, 1, 2)).astype('float32') / 0.255
۲۱۲
۲۱۳
۲۱۴ y_train_f1 = y_train.flatten()
```

```
۲۱۵ y_test_f1 = y_test.flatten()
۲۱۶
۲۱۷ class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
۲۱۸                 'dog', 'frog', 'horse', 'ship', 'truck']
۲۱۹
۲۲۰ train_dataset = TensorDataset(torch.tensor(x_train_t1), torch.tensor(y_train_f1))
۲۲۱ test_dataset = TensorDataset(torch.tensor(x_test_t1), torch.tensor(y_test_f1))
۲۲۲
۲۲۳
۲۲۴ train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
۲۲۵ test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)
۲۲۶
۲۲۷ device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
۲۲۸ print(f"Using device: {device}")
۲۲۹
۲۳۰ """#Autoencoder"""
۲۳۱
۲۳۲ import numpy as np
۲۳۳ import torch
۲۳۴ import torch.nn as nn
۲۳۵ import torch.nn.functional as F
۲۳۶ import torch.optim as optim
۲۳۷ import torch.nn.init as init
۲۳۸ from torch.utils.data import DataLoader
۲۳۹ import torchvision
۲۴۰ import torchvision.transforms as transforms
۲۴۱ from tqdm import tqdm
۲۴۲ from sklearn.neighbors import KNeighborsClassifier
۲۴۳ from sklearn.metrics import classification_report
۲۴۴
۲۴۵ def xavier_initialize(model):
۲۴۶     modules = [
۲۴۷         m for n, m in model.named_modules() if
۲۴۸         'conv' in n or 'linear' in n or 'fc_' in n or 'project' in n or 'q_' in n
۲۴۹     ]
۲۵۰     parameters = [
۲۵۱         p for m in modules for p in m.parameters() if p.dim() >= 2
۲۵۲     ]
۲۵۳     for p in parameters:
۲۵۴         init.xavier_normal_(p)
```



```
۲۵۵
۲۵۶ # =====
۲۵۷ # 2. Architectures (Dense)
۲۵۸ # =====
۲۵۹ class DenseAutoencoder(nn.Module):
۲۶۰     def __init__(self, latent_dim):
۲۶۱         super(DenseAutoencoder, self).__init__()
۲۶۲         self.encoder = nn.Sequential(
۲۶۳             nn.Linear(32 * 32 * 3, 512),
۲۶۴             nn.ReLU(),
۲۶۵             nn.Linear(512, 256),
۲۶۶             nn.ReLU(),
۲۶۷             nn.Linear(256, latent_dim)
۲۶۸         )
۲۶۹         self.decoder = nn.Sequential(
۲۷۰             nn.Linear(latent_dim, 256),
۲۷۱             nn.ReLU(),
۲۷۲             nn.Linear(256, 512),
۲۷۳             nn.ReLU(),
۲۷۴             nn.Linear(512, 32 * 32 * 3),
۲۷۵             nn.Sigmoid()
۲۷۶         )
۲۷۷
۲۷۸     def encode(self, x):
۲۷۹         if len(x.shape) > 2:
۲۸۰             x = x.view(x.size(0), -1)
۲۸۱         return self.encoder(x)
۲۸۲
۲۸۳     def forward(self, x):
۲۸۴         z = self.encode(x)
۲۸۵         out = self.decoder(z)
۲۸۶         return out
۲۸۷
۲۸۸ def extract_features(model, dataloader):
۲۸۹     model.eval()
۲۹۰     features, labels = [], []
۲۹۱     with torch.no_grad():
۲۹۲         for data, target in dataloader:
۲۹۳             data = data.to(device)
۲۹۴             z = model.encode(data) #
```



```
۲۹۵         features.append(z.cpu().numpy())
۲۹۶         labels.append(target.cpu().numpy())
۲۹۷     return np.vstack(features), np.concatenate(labels)
۲۹۸
۲۹۹
۳۰۰     # =====
۳۰۱     # 4. Training Loop
۳۰۲     # =====
۳۰۳     latent_dims = [64, 100]
۳۰۴
۳۰۵     def train_and_evaluate(model_class, model_name, epochs=10, lr=5e-4, weight_decay=1e-5,
↪ is_dense=False):
۳۰۶         trained_models = {}
۳۰۷         histories = {}
۳۰۸
۳۰۹         for latent_dim in latent_dims:
۳۱۰             print(f"\n{'='*60}")
۳۱۱             print(f"Training {model_name} with Latent Dimension = {latent_dim}")
۳۱۲             print(f"{'='*60}")
۳۱۳
۳۱۴             history_key = f"{model_name} ({latent_dim}D)"
۳۱۵             histories[history_key] = []
۳۱۶
۳۱۷             model = model_class(latent_dim).to(device)
۳۱۸             xavier_initialize(model)
۳۱۹             optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
۳۲۰
۳۲۱             for epoch in range(1, epochs + 1):
۳۲۲                 model.train()
۳۲۳                 data_stream = tqdm(enumerate(train_loader, 1), total=len(train_loader))
۳۲۴                 running_loss = 0.0
۳۲۵
۳۲۶                 for batch_index, (x, _) in data_stream:
۳۲۷                     x = x.to(device)
۳۲۸                     optimizer.zero_grad()
۳۲۹
۳۳۰                     x_reconstructed = model(x)
۳۳۱
۳۳۲                     if is_dense:
۳۳۳                         x_reconstructed = x_reconstructed.view(-1, 3, 32, 32)
```



```
۳۳۴
۳۳۵
۳۳۶     loss = F.mse_loss(x_reconstructed, x, reduction='sum')/x.size(0)
۳۳۷
۳۳۸     loss.backward()
۳۳۹     optimizer.step()
۳۴۰
۳۴۱     running_loss += loss.item()
۳۴۲
۳۴۳     data_stream.set_description(
۳۴۴         f'Epoch: {epoch} | Loss: {loss.item():4f.}'
۳۴۵     )
۳۴۶
۳۴۷
۳۴۸     epoch_loss = running_loss / len(train_loader)
۳۴۹     histories[history_key].append(epoch_loss)
۳۵۰
۳۵۱     trained_models[latent_dim] = model
۳۵۲
۳۵۳     print(f"\n[Extracting Features & Training KNN...] Model: {model_name} | Latent Dim:
↳ {latent_dim}D")
۳۵۴     z_train, aligned_y_train = extract_features(model, train_loader)
۳۵۵     z_test, aligned_y_test = extract_features(model, test_loader)
۳۵۶
۳۵۷     knn = KNeighborsClassifier(n_neighbors=7, n_jobs=-1)
۳۵۸     knn.fit(z_train, aligned_y_train)
۳۵۹
۳۶۰     y_pred = knn.predict(z_test)
۳۶۱     print(f"\nClassification Report for {model_name} (Latent Dim: {latent_dim}):")
۳۶۲     print(classification_report(aligned_y_test, y_pred, target_names=class_names))
۳۶۳
۳۶۴     return trained_models, histories
۳۶۵
۳۶۶     # =====
۳۶۷     # 5. Execute and Plot
۳۶۸     # =====
۳۶۹     dense_models, dense_history = train_and_evaluate(DenseAutoencoder, "DenseAutoencoder", epochs=10,
↳ is_dense=True)
۳۷۰
۳۷۱     class ConvAutoencoder(nn.Module):
```



```
۳۷۲
۳۷۳     def __init__(self, latent_dim):
۳۷۴
۳۷۵         super(ConvAutoencoder, self).__init__()
۳۷۶
۳۷۷
۳۷۸
۳۷۹         self.encoder_cnn = nn.Sequential(
۳۸۰
۳۸۱             nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
۳۸۲
۳۸۳             nn.BatchNorm2d(32),
۳۸۴
۳۸۵             nn.ReLU(),
۳۸۶
۳۸۷
۳۸۸
۳۸۹             nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
۳۹۰
۳۹۱             nn.BatchNorm2d(64),
۳۹۲
۳۹۳             nn.ReLU(),
۳۹۴
۳۹۵
۳۹۶
۳۹۷             nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
۳۹۸
۳۹۹             nn.BatchNorm2d(128),
۴۰۰
۴۰۱             nn.ReLU(),
۴۰۲
۴۰۳         )
۴۰۴
۴۰۵         self.encoder_fc = nn.Linear(128 * 4 * 4, latent_dim)
۴۰۶
۴۰۷
۴۰۸
۴۰۹         self.decoder_fc = nn.Linear(latent_dim, 128 * 4 * 4)
۴۱۰
۴۱۱         self.decoder_cnn = nn.Sequential(
```



```
۴۱۲
۴۱۳         nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
۴۱۴
۴۱۵         nn.BatchNorm2d(64),
۴۱۶
۴۱۷         nn.ReLU(),
۴۱۸
۴۱۹
۴۲۰
۴۲۱         nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1),
۴۲۲
۴۲۳         nn.BatchNorm2d(32),
۴۲۴
۴۲۵         nn.ReLU(),
۴۲۶
۴۲۷
۴۲۸
۴۲۹         nn.ConvTranspose2d(32, 3, kernel_size=4, stride=2, padding=1),
۴۳۰
۴۳۱         nn.Sigmoid()
۴۳۲
۴۳۳     )
۴۳۴
۴۳۵
۴۳۶
۴۳۷     def encode(self, x):
۴۳۸
۴۳۹         x = self.encoder_cnn(x)
۴۴۰
۴۴۱         x = x.view(x.size(0), -1)
۴۴۲
۴۴۳         z = self.encoder_fc(x)
۴۴۴
۴۴۵         return z
۴۴۶
۴۴۷
۴۴۸
۴۴۹     def forward(self, x):
۴۵۰
۴۵۱         x = self.encoder_cnn(x)
```



```
۴۵۲
۴۵۳     x = x.view(x.size(0), -1)
۴۵۴
۴۵۵     z = self.encoder_fc(x)
۴۵۶
۴۵۷
۴۵۸
۴۵۹     z = self.decoder_fc(z)
۴۶۰
۴۶۱     z = z.view(z.size(0), 128, 4, 4)
۴۶۲
۴۶۳     out = self.decoder_cnn(z)
۴۶۴
۴۶۵     return out
۴۶۶
۴۶۷ cnn_models, cnn_history = train_and_evaluate(ConvAutoencoder, "CNNAutoencoder", epochs=10,
۴۶۸     ↪ is_dense=False)
۴۶۹
۴۶۹ import matplotlib.pyplot as plt
۴۷۰ histories = {**cnn_history, **dense_history}
۴۷۱
۴۷۲ plt.figure(figsize=(10, 6))
۴۷۳ for key, loss_values in histories.items():
۴۷۴     plt.plot(range(1, 11), loss_values, marker='o', label=key)
۴۷۵
۴۷۶ plt.title('Training Loss of Autoencoders ( Epochs)')
۴۷۷ plt.xlabel('Epoch')
۴۷۸ plt.ylabel('AE Loss (Reconstruction MSE)')
۴۷۹ plt.xticks(range(1, 11))
۴۸۰ plt.legend()
۴۸۱ plt.grid(True)
۴۸۲ plt.show()
۴۸۳
۴۸۴ """#Variational Auto Encoder"""
۴۸۵
۴۸۶ import numpy as np
۴۸۷ import torch
۴۸۸ import torch.nn as nn
۴۸۹ import torch.nn.functional as F
۴۹۰ import torch.optim as optim
```

```
۴۹۱ import torch.nn.init as init
۴۹۲ from torch.utils.data import DataLoader
۴۹۳ import torchvision
۴۹۴ import torchvision.transforms as transforms
۴۹۵ from tqdm import tqdm
۴۹۶ from sklearn.neighbors import KNeighborsClassifier
۴۹۷ from sklearn.metrics import classification_report
۴۹۸
۴۹۹ class DenseVAE(nn.Module):
۵۰۰     def __init__(self, z_size):
۵۰۱         super(DenseVAE, self).__init__()
۵۰۲         self.z_size = z_size
۵۰۳
۵۰۴         self.encoder = nn.Sequential(
۵۰۵             nn.Linear(32 * 32 * 3, 512),
۵۰۶             nn.ReLU(),
۵۰۷             nn.Linear(512, 256),
۵۰۸             nn.ReLU()
۵۰۹         )
۵۱۰         self.fc_mu = nn.Linear(256, z_size)
۵۱۱         self.fc_logvar = nn.Linear(256, z_size)
۵۱۲
۵۱۳         self.decoder = nn.Sequential(
۵۱۴             nn.Linear(z_size, 256),
۵۱۵             nn.ReLU(),
۵۱۶             nn.Linear(256, 512),
۵۱۷             nn.ReLU(),
۵۱۸             nn.Linear(512, 32 * 32 * 3),
۵۱۹             nn.Sigmoid()
۵۲۰         )
۵۲۱
۵۲۲     def encode(self, x):
۵۲۳
۵۲۴         if len(x.shape) > 2:
۵۲۵             x = x.view(x.size(0), -1)
۵۲۶         h = self.encoder(x)
۵۲۷         return self.fc_mu(h), self.fc_logvar(h)
۵۲۸
۵۲۹     def reparameterize(self, mu, logvar):
۵۳۰         std = torch.exp(5.0 * logvar)
```



```
۵۳۱         eps = torch.randn_like(std)
۵۳۲         return mu + eps * std
۵۳۳
۵۳۴     def forward(self, x):
۵۳۵         if len(x.shape) > 2:
۵۳۶             x = x.view(x.size(0), -1)
۵۳۷         mu, logvar = self.encode(x)
۵۳۸         z = self.reparameterize(mu, logvar)
۵۳۹         out = self.decoder(z)
۵۴۰         return out, mu, logvar
۵۴۱
۵۴۲     class VAE(nn.Module):
۵۴۳         def __init__(self, z_size, image_size=32, channel_num=3, kernel_num=128):
۵۴۴             super().__init__()
۵۴۵             self.image_size = image_size
۵۴۶             self.channel_num = channel_num
۵۴۷             self.kernel_num = kernel_num
۵۴۸             self.z_size = z_size
۵۴۹
۵۵۰
۵۵۱             self.encoder = nn.Sequential(
۵۵۲                 self._conv(channel_num, kernel_num // 4),
۵۵۳                 self._conv(kernel_num // 4, kernel_num // 2),
۵۵۴                 self._conv(kernel_num // 2, kernel_num),
۵۵۵             )
۵۵۶
۵۵۷             self.feature_size = image_size // 8
۵۵۸             self.feature_volume = kernel_num * (self.feature_size ** 2)
۵۵۹
۵۶۰             self.q_mean = nn.Linear(self.feature_volume, z_size)
۵۶۱             self.q_logvar = nn.Linear(self.feature_volume, z_size)
۵۶۲             self.project = nn.Linear(z_size, self.feature_volume)
۵۶۳
۵۶۴
۵۶۵             self.decoder = nn.Sequential(
۵۶۶                 self._deconv(kernel_num, kernel_num // 2),
۵۶۷                 self._deconv(kernel_num // 2, kernel_num // 4),
۵۶۸                 self._deconv(kernel_num // 4, channel_num),
۵۶۹                 nn.Sigmoid()
۵۷۰             )
```




```
۵۷۱
۵۷۲     def forward(self, x):
۵۷۳         mean, logvar, z = self.encode(x)
۵۷۴         z_projected = self.project(z).view(
۵۷۵             -1, self.kernel_num,
۵۷۶             self.feature_size,
۵۷۷             self.feature_size,
۵۷۸         )
۵۷۹         x_reconstructed = self.decoder(z_projected)
۵۸۰         return (mean, logvar), x_reconstructed
۵۸۱
۵۸۲     def encode(self, x):
۵۸۳         encoded = self.encoder(x)
۵۸۴         mean, logvar = self.q(encoded)
۵۸۵         z = self.z(mean, logvar)
۵۸۶         return mean, logvar, z
۵۸۷
۵۸۸     def q(self, encoded):
۵۸۹         unrolled = encoded.view(-1, self.feature_volume)
۵۹۰         return self.q_mean(unrolled), self.q_logvar(unrolled)
۵۹۱
۵۹۲     def z(self, mean, logvar):
۵۹۳         std = torch.exp(5.0 * logvar)
۵۹۴         eps = torch.randn_like(std)
۵۹۵         return mean + eps * std
۵۹۶
۵۹۷     def _conv(self, channel_size, kernel_num):
۵۹۸         return nn.Sequential(
۵۹۹             nn.Conv2d(channel_size, kernel_num, kernel_size=4, stride=2, padding=1),
۶۰۰             nn.BatchNorm2d(kernel_num),
۶۰۱             nn.LeakyReLU(2.0),
۶۰۲         )
۶۰۳
۶۰۴     def _deconv(self, channel_num, kernel_num):
۶۰۵         return nn.Sequential(
۶۰۶             nn.ConvTranspose2d(channel_num, kernel_num, kernel_size=4, stride=2, padding=1),
۶۰۷             nn.BatchNorm2d(kernel_num),
۶۰۸             nn.LeakyReLU(2.0),
۶۰۹         )
۶۱۰
```



```
۶۱۱ def xavier_initialize(model):
۶۱۲     modules = [
۶۱۳         m for n, m in model.named_modules() if
۶۱۴         'conv' in n or 'linear' in n or 'fc_' in n or 'project' in n or 'q_' in n
۶۱۵     ]
۶۱۶     parameters = [
۶۱۷         p for m in modules for p in m.parameters() if p.dim() >= 2
۶۱۸     ]
۶۱۹     for p in parameters:
۶۲۰         init.xavier_normal_(p)
۶۲۱
۶۲۲ latent_dims = [64 , 100]
۶۲۳ def extract_features(model, dataloader, is_dense=False):
۶۲۴     model.eval()
۶۲۵     features, labels = [], []
۶۲۶     with torch.no_grad():
۶۲۷         for data, target in dataloader:
۶۲۸             data = data.to(device)
۶۲۹             encoded_out = model.encode(data)
۶۳۰
۶۳۱
۶۳۲             if is_dense:
۶۳۳                 mu, _ = encoded_out
۶۳۴             else:
۶۳۵                 mu, _, _ = encoded_out
۶۳۶
۶۳۷             features.append(mu.cpu().numpy())
۶۳۸             labels.append(target.cpu().numpy())
۶۳۹
۶۴۰     return np.vstack(features), np.concatenate(labels)
۶۴۱
۶۴۲
۶۴۳ def train_and_evaluate(model_class, model_name, epochs=10, lr=5e-4, weight_decay=1e-5, beta=0.2,
→ is_dense=False):
۶۴۴     trained_models = {}
۶۴۵     histories = {}
۶۴۶
۶۴۷     for latent_dim in latent_dims:
۶۴۸         print(f"\n{'='*60}")
۶۴۹         print(f"Training {model_name} with Latent Dimension = {latent_dim}")
```



```
۶۵۰     print(f"{'='*60}")
۶۵۱
۶۵۲
۶۵۳     history_key = f"{model_name} ({latent_dim}D)"
۶۵۴     histories[history_key] = []
۶۵۵
۶۵۶     if is_dense:
۶۵۷         model = model_class(z_size=latent_dim).to(device)
۶۵۸     else:
۶۵۹         model = model_class(z_size=latent_dim).to(device)
۶۶۰
۶۶۱     xavier_initialize(model)
۶۶۲     optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
۶۶۳
۶۶۴     for epoch in range(1, epochs + 1):
۶۶۵         model.train()
۶۶۶         data_stream = tqdm(enumerate(train_loader, 1), total=len(train_loader))
۶۶۷
۶۶۸         running_total = 0.0
۶۶۹
۶۷۰         for batch_index, (x, _) in data_stream:
۶۷۱             x = x.to(device)
۶۷۲             optimizer.zero_grad()
۶۷۳
۶۷۴             outputs = model(x)
۶۷۵
۶۷۶             if is_dense:
۶۷۷                 x_reconstructed, mean, logvar = outputs
۶۷۸                 x_reconstructed = x_reconstructed.view(-1, 3, 32, 32)
۶۷۹             else:
۶۸۰                 (mean, logvar), x_reconstructed = outputs
۶۸۱
۶۸۲             reconstruction_loss = F.mse_loss(x_reconstructed, x, reduction='sum') / x.size(0)
۶۸۳             kl_divergence_loss = -0.5 * torch.sum(1 + logvar - mean.pow(2) - logvar.exp(),
۶۸۴                 ↪ dim=1).mean()
۶۸۵
۶۸۶             total_loss = reconstruction_loss + (beta * kl_divergence_loss)
۶۸۷
۶۸۸             total_loss.backward()
۶۸۹             optimizer.step()
```



```
۶۸۹
۶۹۰         running_total += total_loss.item()
۶۹۱
۶۹۲         data_stream.set_description(
۶۹۳             f'Epoch: {epoch} | '
۶۹۴             f'loss => total: {total_loss.item():2f.} / '
۶۹۵             f're: {reconstruction_loss.item():2f.} / '
۶۹۶             f'kl: {kl_divergence_loss.item():2f.}'
۶۹۷         )
۶۹۸
۶۹۹
۷۰۰         epoch_loss = running_total / len(train_loader)
۷۰۱         histories[history_key].append(epoch_loss)
۷۰۲
۷۰۳     trained_models[latent_dim] = model
۷۰۴
۷۰۵     print(f"\n[Extracting Features & Training KNN...] Model: {model_name} | Latent Dim:
↪ {latent_dim}D")
۷۰۶     z_train, aligned_y_train = extract_features(model, train_loader, is_dense=is_dense)
۷۰۷     z_test, aligned_y_test = extract_features(model, test_loader, is_dense=is_dense)
۷۰۸
۷۰۹     knn = KNeighborsClassifier(n_neighbors=7, n_jobs=-1)
۷۱۰     knn.fit(z_train, aligned_y_train)
۷۱۱
۷۱۲     y_pred = knn.predict(z_test)
۷۱۳     print(f"\nClassification Report for {model_name} (Latent Dim: {latent_dim}):")
۷۱۴     print(classification_report(aligned_y_test, y_pred, target_names=class_names))
۷۱۵
۷۱۶     return trained_models, histories
۷۱۷
۷۱۸ dense_models, dense_history = train_and_evaluate(DenseVAE, "DenseVAE", epochs=10, lr=5e-4,
↪ beta=0.15, is_dense=True)
۷۱۹
۷۲۰ cnn_models, cnn_history = train_and_evaluate(VAE, "Custom_CNN_VAE", epochs=10, lr=5e-4,
↪ beta=0.15, is_dense=False)
۷۲۱
۷۲۲ histories = {**cnn_history, **dense_history}
۷۲۳ plt.figure(figsize=(10, 6))
۷۲۴ for key, loss_values in histories.items():
۷۲۵     plt.plot(range(1, 11), loss_values, marker='o', label=key)
```

```

۷۲۶
۷۲۷ plt.title('Training Loss of VAEs ( Epochs)')
۷۲۸ plt.xlabel('Epoch')
۷۲۹ plt.ylabel('VAE Loss (Reconstruction + KL)')
۷۳۰ plt.xticks(range(1, 11))
۷۳۱ plt.legend()
۷۳۲ plt.grid(True)
۷۳۳ plt.show()
۷۳۴
۷۳۵ """#Exploring in latent Space"""
۷۳۶
۷۳۷ target_latent_dim = 100
۷۳۸ model = cnn_models[target_latent_dim]
۷۳۹ model.eval()
۷۴۰
۷۴۱ import matplotlib.pyplot as plt
۷۴۲
۷۴۳ z_train_features, train_labels = extract_features(model, train_loader, is_dense=False)
۷۴۴
۷۴۵
۷۴۶ plt.figure(figsize=(11, 8))
۷۴۷ scatter = plt.scatter(z_train_features[:, 0], z_train_features[:, 1], c=train_labels,
۷۴۸ ↪ cmap='tab10', alpha=0.6, s=10)
۷۴۹
۷۵۰ cbar = plt.colorbar(scatter, ticks=range(10))
۷۵۱ cbar.ax.set_yticklabels(class_names)
۷۵۲ # -----
۷۵۳
۷۵۴ plt.title(f"Latent Space Scatter Plot (ConvVAE - {target_latent_dim}D)")
۷۵۵ plt.xlabel(r"Latent Dimension 1 ( $\mu_1$ )")
۷۵۶ plt.ylabel(r"Latent Dimension 2 ( $\mu_2$ )")
۷۵۷ plt.grid(True)
۷۵۸ plt.show()
۷۵۹
۷۶۰ import matplotlib.pyplot as plt
۷۶۱ import numpy as np
۷۶۲ import torch
۷۶۳
۷۶۴ n = 10
    
```



```
۷۶۵ img_dim = 32
۷۶۶ scale = 0.2
۷۶۷ figsize = 15
۷۶۸
۷۶۹ # 100
۷۷۰ target_latent_dim = 100
۷۷۱ model = cnn_models[target_latent_dim] #
۷۷۲
۷۷۳ figure = np.zeros((img_dim * n, img_dim * n, 3))
۷۷۴
۷۷۵ grid_x = np.linspace(-scale, scale, n)
۷۷۶ grid_y = np.linspace(-scale, scale, n)[::-1]
۷۷۷
۷۷۸ model.eval()
۷۷۹
۷۸۰ with torch.no_grad():
۷۸۱     for i, yi in enumerate(grid_y):
۷۸۲         for j, xi in enumerate(grid_x):
۷۸۳
۷۸۴             z_sample = torch.zeros(1, target_latent_dim).to(device)
۷۸۵
۷۸۶
۷۸۷             z_sample[0, 0] = xi
۷۸۸             z_sample[0, 1] = yi
۷۸۹
۷۹۰
۷۹۱             z_projected = model.project(z_sample).view(-1, model.kernel_num, model.feature_size,
۷۹۲                 ↪ model.feature_size)
۷۹۳             x_decoded = model.decoder(z_projected)
۷۹۴
۷۹۵
۷۹۶             img = x_decoded[0].cpu().numpy()
۷۹۷             img = np.transpose(img, (1, 2, 0))
۷۹۸             img = np.clip(img, 0, 1)
۷۹۹
۸۰۰             figure[i * img_dim : (i + 1) * img_dim,
۸۰۱                 j * img_dim : (j + 1) * img_dim, :] = img
۸۰۲
۸۰۳ plt.figure(figsize=(figsize, figsize))
۸۰۴ plt.imshow(figure)
```



```
۸۰۴ plt.title(f"Manifold generation (ConvVAE - {target_latent_dim}D)")
۸۰۵ plt.axis('off')
۸۰۶ plt.show()
۸۰۷
۸۰۸ import matplotlib.pyplot as plt
۸۰۹ import torch
۸۱۰ import numpy as np
۸۱۱
۸۱۲ target_latent_dim = 100
۸۱۳ model = cnn_models[target_latent_dim]
۸۱۴ model.eval()
۸۱۵
۸۱۶ fig, axs = plt.subplots(10, 6, figsize=(15, 25))
۸۱۷
۸۱۸ with torch.no_grad():
۸۱۹     sixty_images_latent = torch.randn(60, target_latent_dim).to(device)
۸۲۰
۸۲۱
۸۲۲ z_projected = model.project(sixty_images_latent).view(-1, model.kernel_num,
↪ model.feature_size, model.feature_size)
۸۲۳ sixty_images = model.decoder(z_projected)
۸۲۴ # -----
۸۲۵
۸۲۶ print("Shape before permute:", sixty_images.shape)
۸۲۷
۸۲۸
۸۲۹ sixty_images = sixty_images.permute(0, 2, 3, 1).cpu().numpy()
۸۳۰
۸۳۱
۸۳۲ sixty_images = np.clip(sixty_images, 0, 1)
۸۳۳
۸۳۴ for i in range(10):
۸۳۵     for j in range(6):
۸۳۶         idx = i * 6 + j
۸۳۷         axs[i][j].imshow(sixty_images[idx])
۸۳۸         axs[i][j].axis('off')
۸۳۹
۸۴۰ plt.tight_layout()
۸۴۱ plt.show()
۸۴۲
```

```

۸۴۳ fig, axs = plt.subplots(10, 6, figsize=(10, 15))
۸۴۴
۸۴۵ with torch.no_grad():
۸۴۶     sixty_images_latent = torch.randn(60, target_latent_dim).to(device)
۸۴۷     z_projected = model.project(sixty_images_latent).view(-1, model.kernel_num,
        ↳ model.feature_size, model.feature_size)
۸۴۸     sixty_images = model.decoder(z_projected)
۸۴۹
۸۵۰     sixty_images = sixty_images.permute(0, 2, 3, 1).cpu().numpy()
۸۵۱     sixty_images = np.clip(sixty_images, 0, 1)
۸۵۲
۸۵۳ for i in range(10):
۸۵۴     for j in range(6):
۸۵۵         idx = i * 6 + j
۸۵۶
۸۵۷         axs[i][j].imshow(sixty_images[idx], interpolation='nearest')
۸۵۸         axs[i][j].axis('off')
۸۵۹
۸۶۰ plt.tight_layout()
۸۶۱ plt.show()
    
```

۶.۲ مدل های انتشار (Diffusion Models)

۱.۶.۲ مدل های انتشار (Diffusion Models) و مقایسه آن ها با VAE

در سال های اخیر، کلاس جدیدی از شبکه های مولد به نام مدل های انتشار (Diffusion Models) معرفی شده اند که در زمینه تولید تصاویر با کیفیت بالا، به استاندارد جدید (State-of-the-Art) تبدیل شده اند. این مدل ها توانسته اند بر بسیاری از محدودیت های مدل های پیشین مانند GAN ها (ناپایداری در آموزش) و VAE ها (تولید تصاویر محو) غلبه کنند.

نحوه عملکرد مدل های انتشار (Diffusion Models) ایده ی اصلی مدل های انتشار، الهام گرفته از فیزیک ترمودینامیک غیرتعادلی است. این مدل ها فرآیند تولید داده را به صورت یک فرآیند مارکوفی دو مرحله ای (رفت و برگشت) مدل سازی می کنند:

۱. فرآیند رفت (Forward Process): در این مرحله، به تدریج و طی T گام زمانی، نویز گاوسی به تصویر اصلی (x_0) اضافه می شود تا در نهایت به یک نویز خالص (x_T) تبدیل گردد. توزیع احتمال انتقال به صورت زیر تعریف می شود:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$$

که در آن β_t واریانس نویز در گام t است.

۲. فرآیند برگشت (Reverse Process): شبکه عصبی (معمولاً یک معماری U-Net) آموزش می‌بیند تا فرآیند رفت را معکوس کند؛ یعنی با دریافت یک نویز خالص (x_T)، به صورت گام به گام نویز را حذف کرده و تصویر اصلی را بازسازی کند:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

شبکه با کمینه‌سازی خطای میانگین مربعات (MSE) بین نویز واقعی اضافه شده و نویز پیش‌بینی شده، آموزش می‌بیند.

مدل‌های انتشار پنهان (Latent Diffusion Models) برای رفع مشکل کندی فرآیند تولید در مدل‌های انتشار استاندارد، معماری جدیدی به نام LDM معرفی شد (که پایه و اساس Stable Diffusion است). در این رویکرد، به جای اعمال فرآیند انتشار روی فضای پیکسلی، ابتدا یک Autoencoder (مشابه VAE) تصویر را به یک فضای پنهان فشرده (z) نگاشت می‌کند و فرآیند انتشار در این فضای پنهان با ابعاد کمتر انجام می‌شود.

مقایسه جامع VAE و Diffusion Models اگرچه هر دو مدل بر پایه تئوری احتمالات و استنتاج متغیر (Variational Inference) بنا شده‌اند، تفاوت‌های بنیادینی دارند:

- کیفیت و وضوح تصاویر (Fidelity): مدل‌های VAE به دلیل استفاده از تابع زیان مبتنی بر بازسازی (مانند MSE) و فشرده‌سازی اطلاعات در یک گلوگاه (Bottleneck) منفرد، معمولاً تصاویری محو (Blurry) تولید می‌کنند. در مقابل، مدل‌های Diffusion به دلیل فرآیند تدریجی کاهش نویز در صدها گام، قادر به تولید جزئیات بسیار دقیق (High-Fidelity) هستند.
- ساختار فضای پنهان (Latent Space): در VAE، فضای پنهان ابعاد بسیار کمتری نسبت به تصویر ورودی دارد و برای کارهایی مانند پیمایش در فضا (Interpolation) عالی است. اما در مدل‌های انتشار استاندارد، فضای متغیرهای پنهان دارای ابعادی برابر با تصویر اصلی است و فشرده‌سازی ابعادی (بدون لایه‌های میانی LDM) رخ نمی‌دهد.
- سرعت استنتاج و تولید (Sampling Speed): تولید تصویر در VAE تنها با یک بار عبور از شبکه رمزگشا انجام می‌شود و بسیار سریع است. اما Diffusion نیازمند عبور مکرر (از ده‌ها تا هزاران بار) از شبکه برای حذف نویز است که فرآیند تولید را بسیار کندتر می‌کند.